

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CÔNG NGHỆ TP.HCM



LẬP TRÌNH
TRÊN THIẾT BỊ DI ĐỘNG

Biên soạn:

ThS. Bùi Phú Khuyến

Website: <http://www.hutech.edu.vn>

LẬP TRÌNH TRÊN THIẾT BỊ DI ĐỘNG

Ấn bản 2024

MỤC LỤC

MỤC LỤC	I
HƯỚNG DẪN.....	VI
BÀI 1: TỔNG QUAN VỀ LẬP TRÌNH TRÊN THIẾT BỊ DI ĐỘNG	1
1.1 TỔNG QUAN VỀ THIẾT BỊ DI ĐỘNG THÔNG MINH	1
1.1.1 Khái niệm thiết bị di động thông minh	1
1.1.2 Các hệ điều hành di động phổ biến	4
1.1.3 Sự phát triển và tương lai của thiết bị di động.....	7
1.2 KHÁI QUÁT VỀ LẬP TRÌNH ĐA NỀN TẢNG	9
1.2.1 Định nghĩa lập trình đa nền tảng	9
1.2.2 Giới thiệu về công cụ và ngôn ngữ lập trình đa nền tảng	9
1.3 CÁC CÔNG NGHỆ PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG ĐA NỀN TẢNG PHỔ BIẾN HIỆN NAY	10
1.3.1 So sánh chi tiết giữa Flutter, React Native và Xamarin	11
1.3.2 Lựa chọn công nghệ phù hợp	11
TÓM TẮT	12
BÀI TẬP	13
BÀI 2: CƠ BẢN VỀ NGÔN NGỮ DART.....	14
2.1 GIỚI THIỆU NGÔN NGỮ DART.....	14
2.1.1 Dart là gì?	14
2.1.2 Lịch sử phát triển của Dart	15
2.1.3 Ưu điểm của Dart.....	15
2.1.4 Những khái niệm quan trọng	16
2.2 CÀI ĐẶT MÔI TRƯỜNG PHÁT TRIỂN DART	17
2.2.1 Dart SDK.....	17
2.2.2 DartPad	17
2.2.3 Chương trình Dart đơn giản	18
2.3 CÚ PHÁP CƠ BẢN	20
2.3.1 Biến – Variables	20
2.3.2 Toán tử - Operators	22
2.3.3 Chú thích - Comments	29
2.3.4 Siêu dữ liệu – Metadata	30
2.3.5 Thư viện – Libraries	32
2.3.6 Từ khoá – Keywords	33
2.4 KIỂU DỮ LIỆU	34
2.4.1 Kiểu dữ liệu tổng hợp.....	34
2.4.2 Records	39
2.4.3 Generics	39
2.4.4 Typedefs	42

2.5 PATTERNS TRONG DART	43
2.5.1 Giới thiệu	43
2.5.2 Các dạng Pattern	43
2.6 HÀM	45
2.6.1 Hàm main().....	45
2.6.2 Khai báo hàm cơ bản	45
2.6.3 Sử dụng cú pháp rút gọn với =>	46
2.6.4 Hàm với tên tham số (Named parameters).....	47
2.6.5 Hàm với tham số tùy chọn (Optional positional parameters)	48
2.6.6 Hàm ẩn danh (Anonymous functions)	49
2.7 CẤU TRÚC LẶP VÀ RÈ NHÁNH	51
2.7.1 Cấu trúc lặp.....	51
2.7.2 Cấu trúc rẽ nhánh	52
2.8 LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG.....	54
2.8.1 Kiến thức cơ bản	54
2.8.2 Tính kế thừa	61
2.8.3 Abstract, Interface và Mixin	62
2.9 XỬ LÝ LỖI.....	66
2.10 XỬ LÝ ĐỒNG THỜI	70
2.10.1 Giới thiệu	70
2.10.2 Future và Stream	71
2.10.3 Async và Await.....	73
2.10.4 Một số thao tác thường gặp	73
TÓM TẮT	75
BÀI TẬP	75
BÀI 3: FLUTTER VÀ CÁC WIDGET CƠ BẢN	77
3.1 TỔNG QUAN VỀ FLUTTER	77
3.1.1 Giới thiệu	77
3.1.2 Kiến trúc ứng dụng.....	78
3.1.3 Thiết lập môi trường phát triển	79
3.1.4 Cấu trúc dự án Flutter.....	83
3.2 WIDGET VÀ LAYOUT	84
3.2.1 Giới thiệu Widget	84
3.2.2 Phân loại Widgets.....	84
3.2.3 Layout Widgets.....	85
3.3 STATELESS VÀ STATEFULL WIDGETS	85
3.3.1 Stateless Widgets.....	85
3.3.2 Statefull Widgets.....	86
3.3.3 Quản lý trạng thái	88
3.4 CÁC WIDGET CƠ BẢN.....	92
3.4.1 Scaffold	92

3.4.2 Text	92
3.4.3 Row.....	93
3.4.4 Column	94
3.4.5 Stack	94
3.4.6 Container	95
3.4.7 Align.....	96
3.4.8 Padding.....	96
3.4.9 SizedBox	97
3.4.10 IconButton	97
3.5 ASSET VÀ MEDIA	98
3.5.1 Xử lý Assets và Images	98
3.5.2 Xử lý Video.....	99
3.6 NAVIGATING VÀ ROUTING	99
3.6.1 Sử dụng Navigator	100
3.6.2 Sử dụng Named Routes.....	100
3.6.3 Sử dụng Router.....	101
TÓM TẮT	102
BÀI TẬP	102
BÀI 4: THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG NÂNG CAO	103
4.1 DESIGN & THEMING	103
4.1.1 Material Design	103
4.1.2 Cupertino Design.....	106
4.2 NAVIGATION & LAYOUT CONTROL.....	108
4.2.1 BottomNavigationBar.....	108
4.2.2 ListView	109
4.2.3 GridView	109
4.2.4 SingleChildScrollView.....	110
4.2.5 Positioned	111
4.2.6 Flexible	111
4.2.7 Expanded.....	112
4.2.8 Rotated Box	113
4.3 INTERACTIVITY	113
4.3.1 Buttons.....	113
4.3.2 Gestures	115
4.3.3 Input & Form	116
4.3.4 Visual Feedback	117
4.4 ACCESSIBILITY & INTERNATIONALIZATION	120
4.4.1 Acessibility	120
4.4.2 Internationalization	121
4.5 ANIMATIONS & TRANSITIONS	121
TÓM TẮT	123

BÀI TẬP	123
BÀI 5: THƯ VIỆN FLUTTER	125
5.1 PUB.DEV	125
5.1.1 Giới thiệu	125
5.1.2 Tích hợp thư viện vào Flutter	126
5.2 THƯ VIỆN PHỔ BIẾN	128
5.2.1 Quản lý trạng thái (State Management)	128
5.2.2 Giao tiếp mạng (Networking).....	130
5.2.3 Lưu trữ dữ liệu.....	130
5.2.4 Hiển thị và tương tác hình ảnh (Image Interaction)	132
5.2.5 Thông báo và quản lý quyền (Notifications and Permissions)	133
5.2.6 Định vị và bản đồ (Location and Maps)	133
5.2.7 Tương tác với các chức năng hệ thống (System Interaction)	134
TÓM TẮT	136
BÀI TẬP	136
BÀI 6: LẬP TRÌNH MẠNG VÀ CƠ SỞ DỮ LIỆU.....	138
6.1 XỬ LÝ JSON	138
6.1.1 Tổng quan về JSON	138
6.1.2 Serialize	140
6.1.3 Deserialize	141
6.2 LÀM VIỆC VỚI REST APIS	142
6.2.1 Giới thiệu về REST APIs	142
6.2.2 Xử lý dữ liệu	142
6.3 SHARED PREFERENCES.....	146
6.3.1 Giới thiệu	146
6.3.2 Cách sử dụng	147
6.4 SQFLITE	148
6.4.1 Giới thiệu	148
6.4.2 Cách sử dụng	148
6.5 FIREBASE.....	153
6.5.1 Giới thiệu Firebase	153
6.5.2 Firebase Core	153
6.5.3 Firebase Realtime Database & Firestore	154
6.5.4 FlutterFire UI.....	157
TÓM TẮT	158
BÀI TẬP	158
BÀI 7: XỬ LÝ TRẠNG THÁI VÀ LOGIC ỨNG DỤNG	160
7.1 GIỚI THIỆU TỔNG QUAN	160
7.2 PROVIDER.....	164
7.2.1 Giới thiệu	164

7.2.2 Cách sử dụng.....	164
7.3 BLOC	166
7.3.1 Giới thiệu	166
7.3.2 Cách sử dụng.....	167
7.4 GET	169
7.4.1 Giới thiệu	169
7.4.2 Cách sử dụng.....	169
TÓM TẮT	171
BÀI TẬP	171
BÀI 8: XUẤT BẢN VÀ PHÂN PHỐI ỨNG DỤNG	173
8.1 QUY TRÌNH XUẤT BẢN	173
8.1.1 Quản lý phiên bản ứng dụng.....	173
8.1.2 Xuất bản Android	175
8.1.3 Xuất bản iOS	177
8.2 QUY TRÌNH PHÂN PHỐI	178
8.2.1 Google Play Console	178
8.2.2 App Store Connect - Apple	183
TÓM TẮT	185
BÀI TẬP	185
TÀI LIỆU THAM KHẢO	187

HƯỚNG DẪN

MÔ TẢ MÔN HỌC

Môn học trang bị các kiến thức về cách thức phát triển ứng dụng trên nền tảng thiết bị di động sử dụng Flutter. Bao gồm các nhóm kiến thức: giới thiệu các nền tảng hay hệ điều hành trên thiết bị di động thông minh, các công cụ và môi trường để phát triển ứng dụng trên thiết bị di động thông minh đa nền tảng.

NỘI DUNG MÔN HỌC

- Bài 1: Tổng quan về lập trình trên thiết bị di động: Kiến thức cơ bản về thiết bị di động và khái niệm cơ bản về môi trường và cách thức lập trình trên thiết bị di động.
- Bài 2: Cơ bản về ngôn ngữ Dart: Giới thiệu về ngôn ngữ lập trình Dart, cú pháp cơ bản và cách thức cài đặt môi trường phát triển Dart.
- Bài 3: Flutter và các widget cơ bản: Tổng quan về Flutter, kiến trúc ứng dụng Flutter, và giới thiệu các widget cơ bản như Scaffold, Text, Row, Column, Stack, Container.
- Bài 4: Thiết kế giao diện người dùng nâng cao: Trình bày về Design & Theming, Navigation & Layout Control và các widget nâng cao như BottomNavigationBar, ListView, GridView, SingleChildScrollView.
- Bài 5: Thư viện Flutter: Giới thiệu về pub.dev, cách tích hợp thư viện vào Flutter và các thư viện phổ biến như quản lý trạng thái, giao tiếp mạng, lưu trữ dữ liệu.
- Bài 6: Lập trình mạng và cơ sở dữ liệu: Xử lý JSON, làm việc với REST APIs, sử dụng Shared Preferences, SQLite và Firebase trong ứng dụng Flutter.
- Bài 7: Xử lý trạng thái và logic ứng dụng: Kiến thức về quản lý trạng thái trong Flutter, nguyên lý hoạt động của các thư viện quản lý trạng thái phổ biến như Provider, BLoC và Get và cách sử dụng các thư viện này trong ứng dụng Flutter.
- Bài 8: Xuất bản và phân phối ứng dụng: Quy trình xuất bản ứng dụng trên 2 hệ điều hành chính là Android và iOS, quản lý phiên bản ứng dụng và các công cụ phân phối ứng dụng như Google Play Console và App Store Connect.

KIẾN THỨC TIỀN ĐỀ

Sinh viên cần có kiến thức về lập trình hướng đối tượng, lập trình giao diện người dùng, và cơ sở dữ liệu quan hệ. Kiến thức cơ bản về ngôn ngữ lập trình Dart và Flutter sẽ giúp sinh viên tiếp cận nhanh hơn với các bài học trong môn học này.

YÊU CẦU MÔN HỌC

Người học phải tham dự đầy đủ các buổi học và làm bài tập đầy đủ ở nhà. Ngoài ra, trước mỗi buổi học, người học cần đọc trước bài học trong giáo trình để nắm các nội dung chính. Sau mỗi giờ học, người học cần đọc thêm tài liệu do giảng viên yêu cầu. Do giới hạn của giáo trình không thể cung cấp hết và chi tiết tất cả các thành phần trong lập trình Flutter, vì vậy người học cần đọc thêm các tài liệu tham khảo được liệt kê ở phần cuối của giáo trình.

CÁCH TIẾP NHẬN NỘI DUNG MÔN HỌC

Để học tốt môn này, người học cần ôn tập các bài đã học, trả lời các câu hỏi và làm nhiều bài tập; đọc trước bài mới và tự tìm kiếm các thông tin liên quan đến bài học.

Đối với mỗi bài học, người học đọc trước mục tiêu và tóm tắt bài học, sau đó đọc nội dung bài học. Kết thúc mỗi bài học người đọc cần phải làm tối thiểu các bài tập yêu cầu.

Ngoài ra đây là môn học trong đó có một số bài sẽ có phần thực hành là viết chương trình, do đó người học cần phải làm những bài tập phần thực hành tương ứng với những bài học.

PHƯƠNG PHÁP ĐÁNH GIÁ MÔN HỌC

Môn học được đánh giá gồm:

- Điểm quá trình (50%): Hình thức và cách đánh giá do giảng viên dạy lý thuyết quyết định được phê duyệt của bộ môn.

- Điểm thi cuối kỳ (50%): Hình thức làm đồ án môn học và chấm thi vấn đáp dựa trên nội dung đồ án môn học kết hợp với lý thuyết trong các bài học. Danh sách đồ án do giảng viên quyết định được bộ môn kiểm duyệt và cung cấp vào đầu khóa học.

BÀI 1: TỔNG QUAN VỀ LẬP TRÌNH TRÊN THIẾT BỊ DI ĐỘNG

Học xong bài này người học cần nắm được các nội dung sau.

- Các hệ điều hành di động phổ biến: Hiểu rõ về Android và iOS, bao gồm kiến trúc, lịch sử phát triển và tính năng nổi bật của từng hệ điều hành.
- Lập trình đa nền tảng: Nhận thức được lợi ích và khi nào nên chọn lập trình đa nền tảng so với lập trình bản địa.
- Công nghệ phát triển ứng dụng đa nền tảng: Hiểu biết về Flutter, React Native và Xamarin, kèm theo khả năng so sánh hiệu quả giữa các công nghệ này.
- Lựa chọn công nghệ phù hợp: Phát triển kỹ năng phân tích nhu cầu dự án để lựa chọn công nghệ phát triển ứng dụng phù hợp.
- Khả năng tự học và nghiên cứu: Khuyến khích tự học và nghiên cứu để cập nhật các công nghệ mới và thích ứng với các xu hướng công nghệ hiện đại.

1.1 TỔNG QUAN VỀ THIẾT BỊ DI ĐỘNG THÔNG MINH

1.1.1 Khái niệm thiết bị di động thông minh

1.1.1.1 Định nghĩa

Thiết bị di động thông minh (smart devices) là các thiết bị điện tử có khả năng xử lý và lưu trữ thông tin, kết nối mạng và chạy các ứng dụng phần mềm để thực hiện nhiều chức năng khác nhau. Chúng thường được trang bị các hệ điều hành mạnh mẽ, màn hình cảm ứng và nhiều cảm biến khác nhau để cung cấp trải nghiệm người dùng

đa dạng và phong phú. Thiết bị di động thông minh không chỉ giới hạn ở điện thoại và máy tính bảng, mà còn bao gồm các thiết bị đeo thông minh và nhiều thiết bị IoT khác.

Thiết bị di động thông minh đã trở thành một phần không thể thiếu trong cuộc sống hàng ngày, hỗ trợ các hoạt động từ công việc, học tập, giải trí đến quản lý sức khỏe và nhà cửa. Khả năng kết nối mạng không dây cho phép các thiết bị này luôn sẵn sàng truy cập thông tin và dịch vụ trực tuyến, làm việc cùng nhau trong một hệ sinh thái thông minh.

1.1.1.2 Các loại thiết bị di động thông dụng

- **Điện thoại thông minh (Smartphones)**

Định nghĩa: Điện thoại thông minh là thiết bị di động cá nhân có khả năng thực hiện cuộc gọi và nhắn tin, đồng thời có thể chạy các ứng dụng phức tạp, truy cập internet, chụp ảnh và nhiều tính năng khác. Chúng là sự kết hợp giữa điện thoại di động và máy tính cá nhân.

Đặc điểm:

- Hệ điều hành: Điện thoại thông minh thường chạy trên các hệ điều hành như Android, iOS, cho phép cài đặt và chạy các ứng dụng từ cửa hàng ứng dụng.
- Kết nối: Hỗ trợ kết nối không dây như Wi-Fi, Bluetooth, mạng di động (4G/5G), GPS.
- Cảm biến: Được trang bị nhiều cảm biến như gia tốc kế, con quay hồi chuyển, cảm biến ánh sáng và cảm biến vân tay để cung cấp các tính năng bảo mật và trải nghiệm người dùng tốt hơn.
- Camera: Camera chất lượng cao cho phép chụp ảnh và quay video với độ phân giải cao.
- Màn hình cảm ứng: Màn hình lớn, cảm ứng đa điểm hỗ trợ thao tác chạm, vuốt và kéo thả.

- **Máy tính bảng (Tablets)**

Định nghĩa: Máy tính bảng là thiết bị di động có màn hình cảm ứng lớn hơn điện thoại thông minh, thường từ 7 inch trở lên và có thể thực hiện các tác vụ tương tự như một máy tính xách tay.

Đặc điểm:

- Hệ điều hành: Chạy các hệ điều hành như Android, iPadOS hoặc Windows, cho phép cài đặt và chạy nhiều ứng dụng phục vụ cho công việc và giải trí.
- Màn hình: Kích thước màn hình lớn hơn, thường từ 7 inch đến 12 inch hoặc hơn, cung cấp không gian làm việc rộng rãi và trải nghiệm giải trí tốt hơn.
- Pin: Pin dung lượng lớn hơn, cho phép thời gian sử dụng lâu hơn so với điện thoại thông minh.
- Phụ kiện: Hỗ trợ các phụ kiện như bút cảm ứng (stylus), bàn phím rời, giúp tăng hiệu quả sử dụng cho các công việc sáng tạo và văn phòng.

• **Thiết bị đeo thông minh (Wearables Device)**

Định nghĩa: Thiết bị đeo thông minh bao gồm các thiết bị như đồng hồ thông minh (smartwatches), vòng tay sức khỏe (fitness trackers), có thể đeo trên người và cung cấp thông tin cũng như khả năng kết nối với điện thoại thông minh.

Đặc điểm:

- Kết nối: Kết nối không dây với điện thoại thông minh qua Bluetooth hoặc Wi-Fi, đồng bộ hóa dữ liệu về sức khỏe và thông báo.
- Theo dõi sức khỏe: Theo dõi các chỉ số sức khỏe như nhịp tim, bước chân, giấc ngủ và lượng calo tiêu thụ.
- Thông báo: Hiển thị thông báo từ điện thoại như tin nhắn, cuộc gọi và các thông báo ứng dụng khác.
- Chống nước: Nhiều thiết bị đeo thông minh có khả năng chống nước, cho phép sử dụng khi bơi lội hoặc trong các điều kiện ẩm ướt.

• **Thiết bị kết nối thông minh khác (Other Smart Connected Devices)**

Định nghĩa: Bao gồm các thiết bị thông minh khác như smart TV, smart home devices (thiết bị nhà thông minh) như đèn thông minh, ổ khóa thông minh và nhiều thiết bị khác.

Đặc điểm:

- IoT (Internet of Things): Các thiết bị này thường tích hợp công nghệ IoT, cho phép chúng kết nối và tương tác với nhau qua mạng internet.
- Điều khiển từ xa: Người dùng có thể điều khiển các thiết bị này từ xa thông qua ứng dụng trên điện thoại thông minh hoặc bằng giọng nói thông qua các trợ lý ảo như Google Assistant, Amazon Alexa.
- Tự động hóa: Các thiết bị nhà thông minh có thể được lập trình để tự động thực hiện các tác vụ như bật/tắt đèn, điều chỉnh nhiệt độ, khóa/mở cửa theo thời gian hoặc cảm biến.

1.1.2 Các hệ điều hành di động phổ biến

1.1.2.1 Android

Định nghĩa: Android là hệ điều hành di động dựa trên nhân Linux, được phát triển bởi Google. Được thiết kế cho các thiết bị cảm ứng như điện thoại thông minh và máy tính bảng, Android cũng đã mở rộng sang các thiết bị khác như TV, ô tô và thiết bị đeo thông minh.

Lịch sử phát triển:

- Android được thành lập vào năm 2003 bởi Andy Rubin và được Google mua lại vào năm 2005.
- Phiên bản thương mại đầu tiên, Android 1.0, ra mắt vào năm 2008.
- Đến nay, Android đã trải qua nhiều phiên bản cập nhật, mỗi phiên bản đều có tên mã là một món tráng miệng, từ Cupcake (Android 1.5) đến Android 15 (Vanilla Ice Cream).

Đặc điểm:

- Mã nguồn mở: Android là một hệ điều hành mã nguồn mở, cho phép các nhà phát triển tùy chỉnh và xây dựng các phiên bản Android riêng của họ.
- Google Play Store: Nền tảng chính để phân phối và tải xuống các ứng dụng Android.
- Đa dạng thiết bị: Hỗ trợ nhiều loại thiết bị từ nhiều nhà sản xuất khác nhau, dẫn đến sự phong phú về thiết kế và giá cả.
- Tùy biến cao: Người dùng có thể tùy chỉnh giao diện, cài đặt các ứng dụng bên ngoài Google Play Store và thay đổi nhiều thiết lập hệ thống.

Ưu điểm:

- Sự đa dạng về thiết bị và giá cả, phù hợp với nhiều đối tượng người dùng.
- Khả năng tùy biến và cài đặt ứng dụng linh hoạt.
- Cộng đồng phát triển mạnh mẽ và phong phú.

Nhược điểm:

- Sự phân mảnh: Nhiều phiên bản Android cùng tồn tại làm khó khăn trong việc cập nhật và hỗ trợ.
- Bảo mật: Do tính mở và khả năng cài đặt ứng dụng từ nguồn không chính thống, nguy cơ bảo mật cao hơn.

1.1.2.2 iOS

Định nghĩa: iOS là hệ điều hành di động được phát triển bởi Apple, dành riêng cho các thiết bị của hãng như iPhone, iPad và iPod Touch. iOS nổi tiếng với sự ổn định, bảo mật cao và giao diện người dùng thân thiện.

Lịch sử phát triển:

- iOS được giới thiệu lần đầu tiên vào năm 2007 cùng với sự ra mắt của iPhone.
- Hệ điều hành này đã liên tục được cập nhật và cải tiến, với mỗi phiên bản mới mang lại nhiều tính năng và cải tiến bảo mật.

Đặc điểm:

- Độc quyền: Chỉ có sẵn trên các thiết bị của Apple.
- AppStore: Nền tảng duy nhất để phân phối và tải xuống các ứng dụng cho iOS.
- Giao diện người dùng: Được thiết kế thân thiện và dễ sử dụng, với các ứng dụng tích hợp chất lượng cao.
- Bảo mật: Tính năng bảo mật mạnh mẽ, bao gồm mã hóa dữ liệu, bảo mật sinh trắc học (Face ID, Touch ID).

Ưu điểm:

- Tính ổn định và hiệu năng cao.
- Bảo mật và bảo vệ quyền riêng tư mạnh mẽ.
- Hệ sinh thái Apple đồng nhất và tương thích cao giữa các thiết bị.

Nhược điểm:

- Độc quyền và giá cả cao, giới hạn đối tượng người dùng.
- Hạn chế trong việc tùy biến hệ điều hành và cài đặt ứng dụng từ bên ngoài AppStore.

1.1.2.3 Windows Phone

Định nghĩa: Windows Phone là hệ điều hành di động được phát triển bởi Microsoft, thiết kế cho các thiết bị di động như điện thoại thông minh. Windows Phone được ra mắt để cung cấp một sự thay thế cho các hệ điều hành di động hiện có như Android và iOS.

Lịch sử phát triển:

- Windows Phone được ra mắt lần đầu tiên vào năm 2010.
- Windows Phone 8.1 ra mắt vào năm 2014, mang lại nhiều cải tiến và tính năng mới.
- Tuy nhiên, Microsoft đã ngừng hỗ trợ Windows Phone vào năm 2017 do sự cạnh tranh khốc liệt từ Android và iOS.

Đặc điểm:

- Giao diện Metro: Giao diện người dùng với các ô vuông động (Live Tiles), cung cấp thông tin cập nhật trực tiếp trên màn hình chính.
- Hệ sinh thái Microsoft: Tích hợp tốt với các dịch vụ của Microsoft như Office, OneDrive và Outlook.
- Tính năng đồng bộ: Khả năng đồng bộ hóa mạnh mẽ với hệ điều hành Windows trên máy tính và các thiết bị khác của Microsoft.

Ưu điểm:

- Giao diện người dùng trực quan và dễ sử dụng.
- Tích hợp chặt chẽ với các dịch vụ của Microsoft.
- Hiệu năng mượt mà trên các thiết bị cấu hình trung bình.

Nhược điểm:

- Số lượng ứng dụng hạn chế so với Android và iOS.
- Thiếu sự hỗ trợ và cập nhật từ Microsoft sau khi ngừng phát triển.
- Ít sự lựa chọn về thiết bị.

1.1.3 Sự phát triển và tương lai của thiết bị di động

Thiết bị di động bắt đầu từ những chiếc điện thoại cơ bản với chức năng gọi điện và nhắn tin vào những năm 1980 và 1990. Đến những năm 2000, điện thoại thông minh ra đời với sự xuất hiện của iPhone (2007) và Android (2008), mang lại nhiều tính năng tiên tiến như truy cập internet, máy ảnh chất lượng cao và kho ứng dụng phong phú. Các thiết bị này nhanh chóng phát triển, tích hợp nhiều công nghệ như GPS, mạng di động 4G/5G và trở thành trung tâm điều khiển cho các thiết bị IoT. Sự phát triển của thiết bị di động đã chuyển từ việc đơn thuần phục vụ liên lạc sang việc cung cấp trải nghiệm đa dạng, đáp ứng nhu cầu công việc, giải trí và quản lý sức khỏe.

Tương lai của thiết bị di động

- **Mạng 5G và 6G**

- 5G: Công nghệ 5G đang dần trở thành tiêu chuẩn với tốc độ truy cập internet cực nhanh, độ trễ thấp và khả năng kết nối nhiều thiết bị cùng lúc. Điều này sẽ thúc đẩy sự phát triển của các ứng dụng mới như thực tế ảo (VR), thực tế tăng cường (AR) và Internet of Things (IoT) với quy mô lớn hơn.
- 6G: Trong tương lai, công nghệ 6G có thể sẽ xuất hiện, hứa hẹn mang lại tốc độ truyền tải dữ liệu nhanh gấp nhiều lần so với 5G và mở ra những khả năng mới trong viễn thông và kết nối toàn cầu.

- **Thiết Bị Gập và Cuộn**

- Thiết bị gập: Các công ty công nghệ lớn như Samsung và Huawei đã giới thiệu điện thoại thông minh có màn hình gập. Điều này không chỉ tạo ra trải nghiệm người dùng mới lạ mà còn mở rộng khả năng sử dụng của thiết bị di động trong nhiều tình huống khác nhau.
- Thiết bị cuộn: Thiết bị với màn hình cuộn cũng đang được nghiên cứu và phát triển, hứa hẹn mang lại trải nghiệm linh hoạt và tiện lợi hơn.

- **Trí tuệ nhân tạo (AI) và Học máy (Machine Learning)**

- AI Tích Hợp: Các thiết bị di động trong tương lai sẽ tích hợp nhiều hơn các công nghệ AI, giúp cải thiện trải nghiệm người dùng qua các trợ lý ảo, dịch vụ cá nhân hóa và khả năng xử lý ngôn ngữ tự nhiên.
- Học máy: Học máy sẽ được sử dụng để tối ưu hóa hiệu suất thiết bị, dự đoán nhu cầu người dùng và cải thiện bảo mật.

- **Tính bền vững và môi trường**

- Vật liệu thân thiện với môi trường: Các nhà sản xuất sẽ chú trọng hơn đến việc sử dụng vật liệu tái chế và thân thiện với môi trường trong sản xuất thiết bị di động.
- Tiết kiệm năng lượng: Công nghệ pin mới và các giải pháp tiết kiệm năng lượng sẽ được phát triển để kéo dài tuổi thọ pin và giảm thiểu tác động tiêu cực đến môi trường.

- **Bảo mật và quyền riêng tư:**

- Bảo mật nâng cao: Với sự gia tăng của các mối đe dọa bảo mật, các thiết bị di động trong tương lai sẽ được trang bị các biện pháp bảo mật nâng cao như xác thực sinh trắc học, mã hóa dữ liệu mạnh mẽ và khả năng phát hiện và ngăn chặn các cuộc tấn công mạng.
- Quyền riêng tư: Các công nghệ mới sẽ giúp bảo vệ quyền riêng tư của người dùng, cho phép họ kiểm soát dữ liệu cá nhân và đảm bảo rằng thông tin của họ được bảo vệ an toàn.

1.2 KHÁI QUÁT VỀ LẬP TRÌNH ĐA NỀN TẢNG

Lập trình đa nền tảng đã trở thành một xu hướng phổ biến trong phát triển ứng dụng di động, cho phép các nhà phát triển tạo ra các ứng dụng chạy được trên nhiều hệ điều hành khác nhau từ cùng một mã nguồn. Điều này không chỉ giúp tiết kiệm thời gian và công sức mà còn đảm bảo tính nhất quán và hiệu quả trong việc phát triển và bảo trì ứng dụng.

1.2.1 Định nghĩa lập trình đa nền tảng

Lập trình đa nền tảng (Cross-platform Development) là phương pháp phát triển ứng dụng mà trong đó một mã nguồn duy nhất có thể chạy trên nhiều hệ điều hành khác nhau như Android, iOS, Windows và các nền tảng khác. Các công cụ và framework lập trình đa nền tảng sử dụng ngôn ngữ lập trình và thư viện chung để xây dựng ứng dụng, sau đó biên dịch hoặc chuyển đổi mã nguồn thành mã máy tương ứng cho từng hệ điều hành.

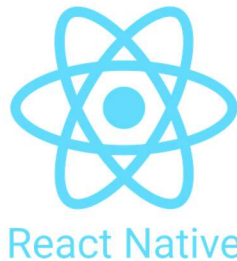
1.2.2 Giới thiệu về công cụ và ngôn ngữ lập trình đa nền tảng

1.2.2.1 Flutter



Flutter là một bộ công cụ phát triển giao diện người dùng (UI toolkit) mã nguồn mở được phát triển bởi Google. Nó cho phép tạo ra các ứng dụng Natively Compiled cho di động, web và máy tính để bàn từ một cơ sở mã duy nhất.

1.2.2.2 React Native



React Native là một framework mã nguồn mở được phát triển bởi Facebook, cho phép xây dựng các ứng dụng di động bằng JavaScript và React. React Native cho phép sử dụng các thành phần UI gốc của nền tảng, đảm bảo hiệu suất và trải nghiệm người dùng tốt.

1.2.2.3 Xamarin



Xamarin là một nền tảng phát triển ứng dụng đa nền tảng được Microsoft phát triển, cho phép xây dựng các ứng dụng di động bằng ngôn ngữ lập trình C#. Xamarin cho phép chia sẻ mã nguồn giữa các nền tảng, giúp tiết kiệm thời gian và công sức trong quá trình phát triển.

1.3 CÁC CÔNG NGHỆ PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG ĐA NỀN TẢNG PHỔ BIẾN HIỆN NAY

Trong những năm gần đây, nhu cầu phát triển ứng dụng di động đa nền tảng đã tăng mạnh, kéo theo sự xuất hiện của nhiều công nghệ và công cụ hỗ trợ lập trình đa nền tảng. Các công cụ này giúp nhà phát triển tiết kiệm thời gian và công sức bằng

cách sử dụng cùng một mã nguồn để triển khai ứng dụng trên nhiều hệ điều hành khác nhau. Ba công nghệ phổ biến hiện nay là Flutter, React Native và Xamarin.

1.3.1 So sánh chi tiết giữa Flutter, React Native và Xamarin

Lập trình đa nền tảng cho phép tạo ra các ứng dụng chạy được trên nhiều hệ điều hành khác nhau từ cùng một mã nguồn, tiết kiệm thời gian và công sức phát triển. Dưới đây là bảng so sánh chi tiết giữa ba công nghệ phổ biến: Flutter, React Native và Xamarin.

Tiêu chí	Flutter	React Native	Xamarin
Ngôn ngữ lập trình	Dart	Javascript	C#
Hiệu suất	Biên dịch trực tiếp thành mã máy, hiệu suất cao	Sử dụng các thành phần UI gốc, hiệu suất gần gốc	Truy cập đầy đủ API gốc, hiệu suất cao
Hệ thống Widget	Hệ thống widget phong phú, tùy biến cao	Component-based, sử dụng các component UI gốc	Xamarin.Forms cho phép chia sẻ mã nguồn UI
Hot Reload	Có	Có	Có
Ưu điểm	Tốc độ phát triển nhanh, giao diện đẹp, cộng đồng hỗ trợ mạnh mẽ	Sử dụng JavaScript phổ biến, tích hợp tốt với hệ sinh thái React, hiệu suất cao	Chia sẻ mã nguồn lớn, sử dụng C#, tích hợp tốt với dịch vụ Microsoft
Nhược điểm	Dart chưa phổ biến rộng rãi, kích thước ứng dụng lớn	Yêu cầu kiến thức lập trình gốc, cộng đồng không đồng đều	Kích thước ứng dụng lớn, cộng đồng nhỏ hơn

Bảng so sánh 3 công nghệ Flutter, React Native và Xamarin

1.3.2 Lựa chọn công nghệ phù hợp

Việc lựa chọn công nghệ phù hợp để phát triển ứng dụng di động đa nền tảng là một quyết định quan trọng, ảnh hưởng trực tiếp đến hiệu suất, trải nghiệm người dùng và tiến độ phát triển dự án. Mỗi công nghệ như Flutter, React Native và Xamarin đều có ưu điểm và hạn chế riêng. Để đưa ra lựa chọn đúng đắn, nhà phát triển cần xem xét

các yếu tố như yêu cầu dự án, kỹ năng của đội ngũ phát triển, cộng đồng hỗ trợ và khả năng mở rộng trong tương lai. Việc đánh giá kỹ lưỡng các tiêu chí này sẽ giúp chọn được công nghệ tối ưu, đảm bảo thành công cho dự án.

- **Xác định yêu cầu dự án**

- Hiệu suất: Chọn Flutter hoặc Xamarin nếu yêu cầu hiệu suất cao.
- Ngôn ngữ lập trình: Chọn React Native nếu đội ngũ phát triển quen thuộc với JavaScript, chọn Xamarin nếu sử dụng C#.
- Giao diện người dùng: Chọn Flutter nếu yêu cầu giao diện người dùng đẹp và nhất quán trên nhiều nền tảng.

- **Đánh giá cộng đồng hỗ trợ**

- Cộng đồng: Flutter và React Native có cộng đồng lớn và tích cực.
- Hỗ trợ: Xamarin có sự hỗ trợ mạnh mẽ từ Microsoft, Flutter được Google giám sát phát triển và được Facebook hỗ trợ với React Native.

- **Khả năng mở rộng và bảo trì**

- Mã nguồn mở: Cả ba công nghệ đều là mã nguồn mở.
- Tính linh hoạt: Flutter và React Native dễ dàng tích hợp với các thư viện bên ngoài và có tính tùy biến cao.

TÓM TẮT

Bài học giới thiệu tổng quan về thiết bị di động cá nhân, điển hình là điện thoại di động và ngày nay thường được trang bị cấu hình mạnh nên được gọi là điện thoại thông minh. Do thiết bị này hiện tại khá phổ biến nên việc phát triển ứng dụng trên thiết bị này được yêu cầu khá nhiều. Bài học giới thiệu các hệ điều hành dành cho thiết bị di động phổ biến hiện nay là Android, iOS (và cả Windows Phone).

BÀI TẬP

Câu 1: Hãy cho biết các nền tảng cho thiết bị di động thông minh hiện nay? Với mỗi nền tảng hãy cho biết đặc điểm, ưu và khuyết điểm.

Câu 2: Liệt kê các nền tảng phát triển ứng dụng di động phổ biến hiện nay và so sánh sự khác biệt chính giữa chúng.

Câu 3: Điều gì làm cho Flutter trở thành một lựa chọn phổ biến cho việc phát triển ứng dụng đa nền tảng? So sánh với các nền tảng khác như React Native và Xamarin.

Câu 4: Liệt kê các ngôn ngữ lập trình chính được sử dụng để phát triển ứng dụng trên Android và giải thích tại sao chúng lại được chọn.

Câu 5: Liệt kê các ngôn ngữ lập trình chính được sử dụng để phát triển ứng dụng trên iOS.

Câu 6: Hãy thảo luận về những thách thức mà Windows Phone đã phải đối mặt và nguyên nhân dẫn đến sự sụt giảm thị phần của nó.

Câu 7: Khám phá các ngôn ngữ và công cụ để phát triển ứng dụng web trên thiết bị di động.

Câu 8: Nghiên cứu về nhu cầu nguồn nhân lực lập trình viên trên thiết bị di động hiện nay và những kỹ năng được yêu cầu nhiều nhất.

BÀI 2: CƠ BẢN VỀ NGÔN NGỮ DART

Bài học này cung cấp một cái nhìn toàn diện về ngôn ngữ lập trình Dart, ngôn ngữ chính được sử dụng trong phát triển ứng dụng với Flutter. Qua bài học, người học sẽ hiểu rõ về cú pháp, kiểu dữ liệu và các cấu trúc điều khiển trong Dart. Ngoài ra, bài học cũng sẽ giới thiệu các khái niệm lập trình hướng đối tượng và cách xử lý lỗi ngoại lệ, cùng với kỹ thuật lập trình bất đồng bộ, đóng vai trò quan trọng trong việc phát triển ứng dụng hiện đại. Sau bài học này, người học sẽ:

- Thành thạo cú pháp và sử dụng các kiểu dữ liệu trong Dart.
- Có khả năng thiết kế và triển khai các hàm, lớp và đối tượng.
- Áp dụng thành thạo các kỹ thuật lập trình hướng đối tượng và xử lý lỗi.
- Sử dụng các kỹ thuật bất đồng bộ để xây dựng ứng dụng hiệu quả và mượt mà.

2.1 GIỚI THIỆU NGÔN NGỮ DART

2.1.1 Dart là gì?



Dart là một ngôn ngữ lập trình mạnh mẽ, mã nguồn mở được phát triển bởi Google. Được thiết kế để xây dựng các ứng dụng hiện đại với hiệu suất cao, Dart được sử dụng chủ yếu trong phát triển giao diện người dùng cho các ứng dụng di động, web và máy tính để bàn. Ngôn ngữ Dart đặc biệt nổi bật khi kết hợp với Flutter để tạo ra các ứng dụng đa nền tảng với giao diện người dùng đẹp mắt và mượt mà.

2.1.2 Lịch sử phát triển của Dart

Dart được giới thiệu lần đầu tiên vào năm 2011 bởi Google với mục tiêu thay thế JavaScript và cung cấp một môi trường lập trình hiệu quả hơn cho các ứng dụng web. Phiên bản 1.0 của Dart ra mắt vào tháng 11 năm 2013, cung cấp một ngôn ngữ mạnh mẽ và linh hoạt cho các nhà phát triển. Qua các năm, Dart liên tục được cải tiến với nhiều tính năng mới và cải thiện hiệu suất.

Các phiên bản quan trọng:

- Dart 2.0 (2018): Mang lại nhiều cải tiến về cú pháp, hiệu suất và khả năng biên dịch, đánh dấu một bước tiến lớn trong việc làm cho Dart trở thành một ngôn ngữ lập trình đa năng và hiện đại.
- Dart 2.12 (2021): Giới thiệu hệ thống null safety, cải thiện tính an toàn và hiệu quả của mã nguồn.
- Dart 3.0 (2023): Mang đến nhiều tính năng mới như pattern matching, records và class modifiers, củng cố vị thế của Dart trong cộng đồng lập trình.
- Dart 3.4 (2024): Cải thiện hiệu suất và thêm nhiều tính năng mới cho ngôn ngữ Dart, bao gồm cả tính năng Macros và hỗ trợ Wasm.

2.1.3 Ưu điểm của Dart

- Hiệu suất cao: Dart có khả năng biên dịch thành mã máy (Native Code) hoặc mã JavaScript, cung cấp hiệu suất gần với mã gốc.
- Cú pháp dễ học: Dart có cú pháp đơn giản và dễ hiểu, giúp các nhà phát triển dễ dàng tiếp cận và làm quen.
- Tính đồng nhất: Dart cung cấp một môi trường lập trình nhất quán với các công cụ tích hợp mạnh mẽ, từ đó giúp giảm bớt sự phức tạp trong phát triển và bảo trì ứng dụng.
- Khả năng đa nền tảng: Dart kết hợp với Flutter cho phép phát triển ứng dụng chạy mượt mà trên nhiều nền tảng như Android, iOS, web và máy tính để bàn từ cùng một mã nguồn duy nhất.

- Thư viện phong phú: Dart có một hệ thống thư viện phong phú và mạnh mẽ, cung cấp nhiều tính năng hữu ích giúp đẩy nhanh quá trình phát triển ứng dụng.
- Cộng đồng hỗ trợ mạnh mẽ: Dart có một cộng đồng phát triển lớn và tích cực, cung cấp nhiều tài liệu, khóa học và hỗ trợ qua các diễn đàn và mạng xã hội.

2.1.4 Những khái niệm quan trọng

2.1.4.1 Khái niệm cơ bản

- Biến (**Variables**): Dart hỗ trợ nhiều loại biến, từ kiểu dữ liệu nguyên thủy (như `int`, `double`, `bool`) đến các kiểu phức tạp hơn như **List**, **Map**. Biến trong Dart có thể được khai báo với từ khóa `var`, `final` hoặc `const`.
- Hàm (**Functions**): Hàm trong Dart có thể được khai báo bằng từ khóa `function`. Dart hỗ trợ cả hàm không tên (anonymous functions) và hàm mũi tên (**arrow functions**) giúp viết mã ngắn gọn hơn.
- Lớp và Đối tượng (**Classes and Objects**): Dart là ngôn ngữ lập trình hướng đối tượng, mọi thứ trong Dart đều là đối tượng. Lớp (`class`) là bản thiết kế cho đối tượng, giúp định nghĩa các thuộc tính và phương thức của đối tượng.
- Thừa kế (**Inheritance**): Dart hỗ trợ tính thừa kế, cho phép tạo ra các lớp con kế thừa từ lớp cha, giúp tái sử dụng mã và tổ chức mã nguồn tốt hơn.
- Mixin: **Mixin** cho phép một lớp kế thừa từ nhiều lớp khác nhau, giúp chia sẻ hành vi giữa các lớp mà không cần thừa kế.

2.1.4.2 Quy tắc trình bày mã nguồn

- Quy tắc đặt tên (**Naming Conventions**): Dart sử dụng quy tắc **camelCase** cho tên biến và hàm, **PascalCase** cho tên lớp. Ví dụ: `myVariable`, `myFunction`, `MyClass`.
- Dấu chấm phẩy (**Semicolons**): Mỗi câu lệnh trong Dart cần kết thúc bằng dấu chấm phẩy.

- Dấu nháy (**Braces**): Dart yêu cầu sử dụng dấu nháy `{}` để bao quanh khối mã, như trong hàm, điều kiện `if` hoặc vòng lặp.
- Thụt lề (**Indentation**): Sử dụng khoảng cách hoặc tab để thụt lề mã, giúp mã nguồn dễ đọc và bảo trì.
- Dấu chú thích (**Comments**): Dart hỗ trợ cả chú thích đơn dòng (`//`) và chú thích đa dòng (`/* */`), giúp giải thích mã và ghi chú các phần quan trọng.

2.2 CÀI ĐẶT MÔI TRƯỜNG PHÁT TRIỂN DART

2.2.1 Dart SDK

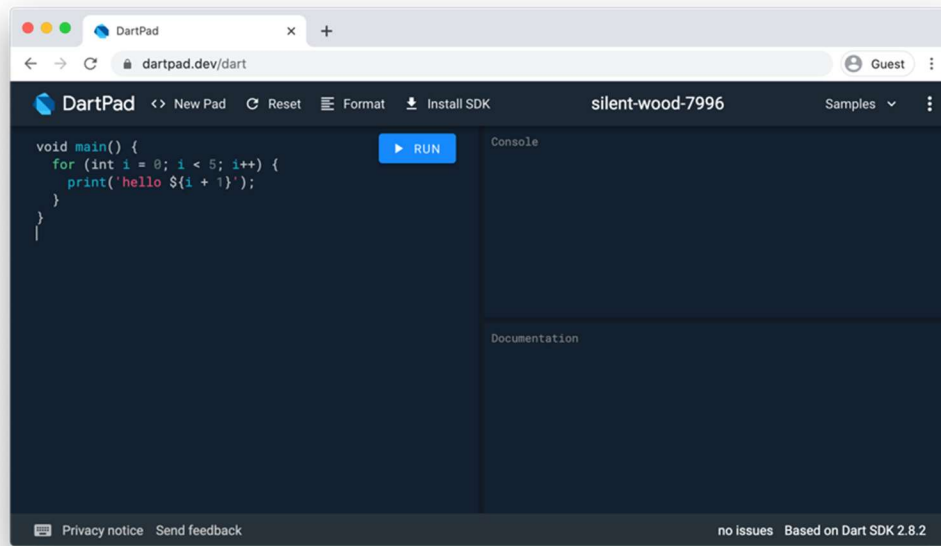
Dart SDK (Software Development Kit) là bộ công cụ cần thiết để phát triển ứng dụng với Dart. Dart SDK bao gồm:

- Dart VM: Máy ảo
- Dart để chạy các chương trình Dart.
- Dart Compiler: Trình biên dịch Dart để biên dịch mã Dart thành mã máy hoặc JavaScript.
- Các công cụ phát triển: Bao gồm các công cụ như Dart Analyzer để kiểm tra mã nguồn, Dart Formatter để định dạng mã và Dart DevTools để gỡ lỗi và tối ưu hóa ứng dụng.

Để cài đặt Dart SDK, người dùng có thể truy cập trang <https://dart.dev> và làm theo hướng dẫn cài đặt cho hệ điều hành của mình (Windows, macOS hoặc Linux).

2.2.2 DartPad

DartPad là một công cụ trực tuyến cho phép người dùng viết và chạy mã Dart trực tiếp trên trình duyệt mà không cần cài đặt bất kỳ phần mềm nào. DartPad rất hữu ích cho việc học và thử nghiệm mã Dart nhanh chóng. Người dùng có thể truy cập DartPad tại <https://dartpad.dev>.



DartPad hỗ trợ nhiều tính năng như:

- Viết mã và chạy trực tiếp: Thử nghiệm mã Dart ngay lập tức.
- Chia sẻ mã: Tạo liên kết chia sẻ mã với người khác.
- Codelabs: Truy cập các hướng dẫn và bài tập lập trình mẫu.

2.2.3 Chương trình Dart đơn giản

```
import 'dart:io';  
  
// Định nghĩa lớp Person  
class Person {  
  // Thuộc tính  
  String name;  
  int age;  
  
  // Hàm khởi tạo  
  Person(this.name, this.age);  
  
  // Phương thức để in thông tin  
  void printInfo() {  
    print('Name: $name');  
    print('Age: $age');  
  }  
}
```

```
// Hàm chính của chương trình
void main() {
    // Nhập tên từ người dùng
    stdout.write('Enter your name: ');
    String name = stdin.readLineSync()!;

    // Nhập tuổi từ người dùng
    stdout.write('Enter your age: ');
    int age = int.parse(stdin.readLineSync()!);

    // Tạo đối tượng Person
    Person person = Person(name, age);

    // Gọi phương thức để in thông tin
    person.printInfo();

    // Sử dụng hàm để tính tổng hai số
    int sum = addNumbers(10, 20);
    print('Sum of 10 and 20 is: $sum');
}

// Hàm để tính tổng hai số
int addNumbers(int a, int b) {
    return a + b;
}
```

Giải thích chương trình:

- Import thư viện: **import 'dart:io';** để sử dụng các hàm nhập/xuất cơ bản.
- Định nghĩa lớp **Person**: Lớp này có hai thuộc tính **name** và **age**, một hàm khởi tạo để gán giá trị cho các thuộc tính và một phương thức **printInfo** để in thông tin của đối tượng.
- Hàm **main**: Đây là điểm bắt đầu của chương trình.
 - + Sử dụng **stdout.write** và **stdin.readLineSync** để nhập tên và tuổi từ người dùng.
 - + Tạo đối tượng **Person** với các thông tin vừa nhập.
 - + Gọi phương thức **printInfo** để in thông tin của đối tượng.

- + Gọi hàm `addNumbers` để tính tổng hai số và in kết quả.
- Hàm `addNumbers`: Hàm này nhận hai tham số kiểu `int` và trả về tổng của chúng.

2.3 CÚ PHÁP CƠ BẢN

2.3.1 Biến – Variables

2.3.1.1 Khai báo biến

Biến trong Dart được sử dụng để lưu trữ các tham chiếu tới các đối tượng. Ví dụ về cách tạo biến và khởi tạo:

```
// Biến name lưu trữ tham chiếu tới đối tượng String với giá trị 'Khuyên'.  
var name = 'Khuyên';
```

Dart tự động suy luận kiểu cho biến `name` là `String`. Tuy nhiên, ta có thể thay đổi kiểu của biến bằng cách chỉ định cụ thể:

```
// Biến name có kiểu Object  
Object name = 'Khuyên';
```

Hoặc khai báo rõ ràng kiểu dữ liệu:

```
// Biến name có kiểu String  
String name = 'Khuyên';
```

2.3.1.2 Null safety

Dart thực thi tính năng `Null Safety`, giúp ngăn chặn lỗi `Null Dereference`. Với `Null Safety`, Dart phát hiện các lỗi tiềm ẩn tại thời gian biên dịch. Để khai báo biến có thể nhận giá trị `null`, thêm ký hiệu `?` vào sau kiểu dữ liệu:

```
String? name; // Kiểu dữ liệu có thể null  
String name = 'Khuyên'; // Kiểu dữ liệu không thể null
```

Biến không thể `null` phải được khởi tạo trước khi sử dụng:

```
int lineCount = 0; // Khởi tạo giá trị ban đầu
```

Ví dụ về việc sử dụng biến có thể `null`:

```
int? lineCount;  
assert(lineCount == null); // Biến lineCount có giá trị ban đầu là null
```

Lưu ý: Hàm `assert()` được sử dụng để kiểm tra các điều kiện trong quá trình phát triển (development). Nếu điều kiện trong `assert()` là `false`, chương trình sẽ ném ra lỗi và dừng lại.

2.3.1.3 Biến kiểu late

Khi ta chắc chắn biến sẽ được khởi tạo trước khi sử dụng nhưng Dart không thể xác định điều đó, ta có thể sử dụng từ khóa `late`:

```
late String description;  
  
void main() {  
  description = 'Trường HUTECH!';  
  print(description); // In ra: Trường HUTECH!  
}
```

Biến `late` cũng có thể được khởi tạo dạng Lazy Initialization:

```
late String temperature = readThermometer(); // Khởi tạo dạng Lazy
```

2.3.1.4 Biến final và const

- **Final:** Biến `final` chỉ được gán một lần và không thể thay đổi giá trị sau khi gán:

```
final name = 'Khuyen'; // Biến final  
name = 'Cuong'; // Lỗi: không thể thay đổi giá trị của biến final
```

- **Const:** Biến `const` là hằng số tại thời gian biên dịch và không thể thay đổi giá trị:

```
const bar = 1000000; // Đơn vị áp suất (dynes/cm2)  
const double atm = 1.01325 * bar; // Áp suất tiêu chuẩn
```

Biến `const` có thể được sử dụng để tạo giá trị hằng số và khai báo hàm tạo hằng số:

```
const Object i = 3;  
const list = [i as int]; // Sử dụng ép kiểu  
const map = {if (i is int) i: 'int'}; // Sử dụng is và collection if
```

```
const set = {if (list is List<int>) ...list}; // Sử dụng spread
```

Dưới đây là ví dụ tổng hợp về các quy tắc trên:

```
import 'dart:io';

void main() {
  // Khai báo biến
  var name = 'Khuyên'; // Dart suy luận kiểu String
  int age = 25; // Khai báo rõ ràng kiểu int

  // Sử dụng null safety
  String? nullableName;
  nullableName = null; // Được phép vì kiểu có thể null

  // Sử dụng biến kiểu late
  late String description;
  description = 'Lập trình Dart'; // Khởi tạo sau khi khai báo

  // Sử dụng final và const
  final String country = 'Vietnam'; // Biến final
  const int year = 2024; // Biến const

  // In giá trị biến
  print('Name: $name');
  print('Age: $age');
  print('Description: $description');
  print('Country: $country');
  print('Year: $year');

  // Kiểm tra giá trị ban đầu của biến nullable
  int? lineCount;
  assert(lineCount == null); // Biến lineCount có giá trị ban đầu là null
}
```

2.3.2 Toán tử - Operators

Dart hỗ trợ một loạt các toán tử, từ toán tử cơ bản như cộng trừ nhân chia đến các toán tử **logic** và toán tử thao tác **bit**. Các toán tử này được sử dụng để thực hiện các phép tính và thao tác trên các giá trị và biến trong chương trình.

2.3.2.1 Toán tử số học

Dart hỗ trợ các toán tử số học cơ bản:

Toán tử	Ý nghĩa
+	Cộng
-	Trừ
*	Nhân
/	Chia
~/	Chia (lấy phần nguyên)
%	Chia (lấy phần dư)
-expr	Phủ định (đổi dấu biểu thức)

Bảng toán tử số học trong Dart

Ví dụ minh họa:

```
assert(2 + 3 == 5);
assert(2 - 3 == -1);
assert(2 * 3 == 6);
assert(5 / 2 == 2.5); // Kết quả là double
assert(5 ~/ 2 == 2);  // Kết quả là int
assert(5 % 2 == 1);   // Phần dư là 1
```

2.3.2.1 Toán tử tăng và giảm

Toán tử	Ý nghĩa
++var	var = var + 1 (giá trị của biểu thức là var + 1)
var++	var = var + 1 (giá trị của biểu thức là var)
--var	var = var - 1 (giá trị của biểu thức là var - 1)
var--	var = var - 1 (giá trị của biểu thức là var)

Bảng toán tử số học trong Dart

Ví dụ minh họa:

```
int a;
```

```

int b;

a = 0;
b = ++a; // Tăng a trước khi b nhận giá trị của nó.
assert(a == b); // 1 == 1

a = 0;
b = a++; // Tăng a sau khi b nhận giá trị của nó.
assert(a != b); // 1 != 0

a = 0;
b = --a; // Giảm a trước khi b nhận giá trị của nó.
assert(a == b); // -1 == -1

a = 0;
b = a--; // Giảm a sau khi b nhận giá trị của nó.
assert(a != b); // -1 != 0

```

2.3.2.2 Toán tử so sánh và quan hệ

Toán tử	Ý nghĩa
==	Bằng
!=	Không bằng
>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng

Bảng toán tử so sánh, quan hệ trong Dart

Ví dụ minh họa:

```

assert(2 == 2);
assert(2 != 3);
assert(3 > 2);
assert(2 < 3);
assert(3 >= 3);
assert(2 <= 3);

```

2.3.2.3 Toán tử kiểm tra kiểu

Toán tử	Ý nghĩa
<code>as</code>	Ép kiểu
<code>is</code>	Kiểm tra kiểu
<code>!is</code>	Kiểm tra không phải kiểu

Bảng toán tử kiểm tra kiểu trong Dart

Ví dụ minh họa:

```
if (employee is Person) {
  // Kiểm tra kiểu
  employee.firstName = 'Khuyên';
}
```

2.3.2.4 Toán tử gán

Toán tử	Ý nghĩa
<code>=</code>	Gán giá trị
<code>?=</code>	Gán giá trị nếu biến hiện tại là null
<code>+=, -=, *=, ...</code>	Gán kết hợp với toán tử khác

Bảng toán tử gán trong Dart

Ví dụ minh họa:

```
var a = 2; // Gán giá trị
a *= 3;    // Gán và nhân: a = a * 3
assert(a == 6);
```

2.3.2.5 Toán tử logic

Toán tử	Ý nghĩa
<code>!expr</code>	Đảo ngược biểu thức (biến <code>false</code> thành <code>true</code> và ngược lại)
<code> </code>	OR logic
<code>&&</code>	AND logic

Bảng toán tử logic trong Dart

Ví dụ minh họa:

```
if (!done && (col == 0 || col == 3)) {
  // ...Thực hiện hành động nào đó...
}
```

2.3.2.6 Toán tử thao tác bit

Toán tử	Ý nghĩa
&	AND bit
	OR bit
^	XOR bit
~expr	Phủ định bit
<<	Dịch trái
>>	Dịch phải
>>>	Dịch phải không dấu

Bảng toán tử kiểm tra kiểu trong Dart

Ví dụ minh họa:

```
final value = 0x22;
final bitmask = 0x0f;

assert((value & bitmask) == 0x02); // AND
assert((value & ~bitmask) == 0x20); // AND NOT
assert((value | bitmask) == 0x2f); // OR
assert((value ^ bitmask) == 0x2d); // XOR

assert((value << 4) == 0x220); // Dịch trái
assert((value >> 4) == 0x02); // Dịch phải
```

2.3.2.7 Toán tử điều kiện

Toán tử	Ý nghĩa
expr1 ? expr2 : expr3	Nếu expr1 đúng, trả về expr2; nếu không, trả về expr3

<code>expr1 ?? expr2</code>	Nếu <code>expr1</code> không <code>null</code> , trả về giá trị của nó; nếu không, trả về <code>expr2</code>
-----------------------------	--

Bảng toán tử điều kiện trong Dart

Ví dụ minh họa:

```
var visibility = isPublic ? 'public' : 'private';
String playerName(String? name) => name ?? 'Guest';
```

2.3.2.8 Toán tử cascade

Toán tử cascade (`..`, `?..`) cho phép ta thực hiện một chuỗi các thao tác trên cùng một đối tượng. Ngoài việc truy cập các thành viên của đối tượng, ta cũng có thể gọi các phương thức của đối tượng đó. Điều này giúp tiết kiệm bước tạo biến tạm thời và cho phép viết mã linh hoạt hơn.

Xem xét đoạn mã sau:

```
var paint = Paint()
  ..color = Colors.black
  ..strokeCap = StrokeCap.round
  ..strokeWidth = 5.0;
```

Constructor `Paint()` trả về một đối tượng `Paint`. Đoạn mã sử dụng toán tử cascade để thao tác trên đối tượng này, bỏ qua bất kỳ giá trị nào có thể được trả về.

Ví dụ trên tương đương với mã sau:

```
var paint = Paint();
paint.color = Colors.black;
paint.strokeCap = StrokeCap.round;
paint.strokeWidth = 5.0;
```

Nếu đối tượng mà toán tử cascade thao tác có thể `null`, hãy sử dụng cascade có kiểm tra `null` (`?..`) cho thao tác đầu tiên. Bắt đầu với `?..` đảm bảo rằng không có thao tác nào trong chuỗi cascade được thực hiện trên đối tượng `null`.

```
querySelector('#confirm') // Lấy một đối tượng.
  ?..text = 'Xác nhận' // Sử dụng các thành viên của nó.
  ..classes.add('important')
  ..onClick.listen((e) => window.alert('Đã xác nhận!'))
```

```
..scrollIntoView();
```

Đoạn mã trên tương đương với:

```
var button = querySelector('#confirm');
button?.text = 'Xác nhận';
button?.classes.add('important');
button?.onClick.listen((e) => window.alert('Đã xác nhận!'));
button?.scrollIntoView();
```

Ta cũng có thể lồng các cascade. Ví dụ:

```
final addressBook = (AddressBookBuilder()
  ..name = 'Khuyên'
  ..email = 'bp.khuyen@hutech.edu.vn'
  ..phone = (PhoneNumberBuilder()
    ..number = '+8433093935'
    ..label = 'Di động')
  .build())
.build();
```

Hãy cẩn thận khi sử dụng cascade trên một hàm trả về giá trị. Ví dụ, đoạn mã sau sẽ thất bại:

```
var sb = StringBuffer();
sb.write('foo')
  ..write('bar'); // Lỗi: phương thức 'write' không được định nghĩa cho 'void'.
```

2.3.2.9 Các toán tử khác

Ta có thể xem một số toán tử còn lại trong một số ví dụ sau

Toán tử	Tên	Ý nghĩa
()	Function application	Biểu thị một lệnh gọi hàm/phương thức
[]	Subscript access	Ví dụ: <code>fooList[1]</code> truy cập <code>fooList</code> tại phần tử có chỉ mục <code>1</code>
?[]	Conditional subscript access	Giống như [], nhưng toán hạng bên trái có thể là <code>null</code> .
.	Member access	Tham chiếu đến một thuộc tính của biểu thức/lớp; ví dụ: <code>foo.bar</code> chọn thuộc tính <code>bar</code> từ biểu thức <code>foo</code>

?.	Conditional member access	Giống như ., nhưng toán hạng bên trái có thể là null .
!	Non-null assertion operator	Ép kiểu một biểu thức thành không-null , ném một ngoại lệ tại runtime nếu ép kiểu thất bại. Ví dụ: foo!.bar xác nhận foo không-null và truy cập thuộc tính bar

Bảng thể hiện một số toán tử khác

2.3.3 Chú thích - Comments

Dart hỗ trợ các loại chú thích một dòng, nhiều dòng và chú thích tài liệu.

- **Chú thích một dòng**

Sử dụng `//` để bắt đầu một comment một dòng. Comment này chỉ tồn tại trên một dòng và thường được sử dụng để giải thích ý nghĩa của một dòng code cụ thể. Ví dụ:

```
// Kiểm tra xem giá trị có phải là số dương
bool isPositive(int number) {
  return number > 0;
}
```

- **Chú thích nhiều dòng**

Sử dụng `/* */` cho comment nhiều dòng, cho phép ta viết comment trải dài qua nhiều dòng. Điều này hữu ích khi cần giải thích chi tiết hơn hoặc vô hiệu hóa tạm thời một đoạn code lớn mà không muốn xóa nó. Ví dụ:

```
/*
  Hàm này sẽ tính toán và trả về tổng của hai số.
  Đây là ví dụ về cách sử dụng comment nhiều dòng.
*/
int add(int a, int b) {
  return a + b;
}
```

- **Chú thích tài liệu**

Sử dụng `///` hoặc `/** */` để viết comment tài liệu, những comment này thường được dùng để tạo tài liệu tự động với DartDoc. Chúng cho phép tích hợp liên kết, mã nhúng và định dạng phong phú. Ví dụ:

```
/// Hàm tính tổng của hai số.  
///  
/// Trả về tổng của [a] và [b].  
int sum(int a, int b) {  
    return a + b;  
}
```

2.3.4 Siêu dữ liệu – Metadata

Siêu dữ liệu (metadata) được sử dụng để cung cấp thông tin bổ sung về mã nguồn. Một chú thích siêu dữ liệu bắt đầu với ký tự `@`, theo sau là một tham chiếu đến một hằng số tại thời gian biên dịch (như `@deprecated`) hoặc một cuộc gọi đến một **constructor** hằng số.

Siêu dữ liệu giúp mã nguồn dễ đọc hơn và cung cấp thông tin rõ ràng về các yếu tố trong chương trình. Bằng cách sử dụng siêu dữ liệu, ta có thể cung cấp hướng dẫn cụ thể cho người dùng và công cụ phân tích mã, giúp cải thiện quá trình phát triển và bảo trì mã nguồn.

- **Các chú thích có sẵn**

Dart cung cấp 4 chú thích siêu dữ liệu mặc định: `@Deprecated`, `@deprecated`, `@override` và `@pragma`. Dưới đây là một ví dụ về việc sử dụng chú thích `@Deprecated`:

```
class Television {  
    /// Sử dụng [turnOn] để bật nguồn thay thế.  
    @Deprecated('Sử dụng turnOn thay thế')  
    void activate() {  
        turnOn();  
    }  
  
    /// Bật nguồn TV.  
    void turnOn() {  
        // Mã bật nguồn TV  
    }  
}
```

Trong ví dụ này, phương thức `activate` được đánh dấu là đã lỗi thời (**deprecated**) và người dùng được khuyến khích sử dụng phương thức `turnOn` thay thế. Ta có thể sử

dùng **@deprecated** nếu không muốn chỉ định thông báo nhưng việc cung cấp thông báo cụ thể với **@Deprecated** sẽ hữu ích hơn cho người dùng.

- **Định nghĩa chú thích tùy chỉnh**

Ta cũng có thể tự định nghĩa các chú thích siêu dữ liệu của riêng mình. Ví dụ, ta có thể tạo một chú thích **@Todo** để ghi chú những công việc cần làm:

```
class Todo {  
  final String who;  
  final String what;  
  
  const Todo(this.who, this.what);  
}
```

Và sử dụng chú thích này như sau:

```
@Todo('Khuyến', 'Thực hiện chức năng này')  
void doSomething() {  
  print('Làm gì đó');  
}
```

Trong ví dụ trên, chú thích **@Todo** cung cấp thông tin về người thực hiện (who) và nhiệm vụ cần làm (what).

- **Vị trí sử dụng siêu dữ liệu**

Siêu dữ liệu có thể được đặt trước các thành phần trong mã Dart như: Thư viện (library), Lớp (class), Kiểu dữ liệu (typedef), Tham số kiểu (type parameter), Constructor, Factory, Hàm (function), Trường (field), Tham số (parameter), Biến (variable), Chỉ thị import hoặc export

Ví dụ, ta có thể sử dụng chú thích **@override** để ghi đè một phương thức từ lớp cha:

```
class Parent {  
  void speak() {  
    print('Parent is speaking');  
  }  
}  
  
class Child extends Parent {  
  @override
```

```
void speak() {  
  print('Child is speaking');  
}  
}
```

Trong ví dụ trên, chú thích `@override` chỉ ra rằng phương thức `speak` trong lớp `Child` đang ghi đè phương thức `speak` từ lớp `Parent`.

2.3.5 Thư viện – Libraries

Trong Dart, việc sử dụng các thư viện giúp mã nguồn của ta trở nên mô-đun và dễ chia sẻ. Các thư viện không chỉ cung cấp API mà còn giúp bảo vệ mã nguồn của ta: các định danh bắt đầu bằng dấu gạch dưới (`_`) chỉ có thể nhìn thấy bên trong thư viện đó. Mỗi tệp Dart (cùng với các phần của nó) được xem như là một thư viện, ngay cả khi tệp đó không sử dụng chỉ thị `library`.

- **Sử dụng thư viện**

Để sử dụng một thư viện trong Dart, ta sử dụng từ khóa `import`. Việc này cho phép ta truy cập các lớp, hàm và các thành phần khác từ thư viện đó trong mã.

Ví dụ, các ứng dụng web Dart thường sử dụng thư viện `dart:html`, có thể được `import` như sau:

```
import 'dart:html';
```

Tham số bắt buộc duy nhất cho `import` là **URI** chỉ định thư viện. Đối với các thư viện tích hợp, **URI** có dạng đặc biệt `dart:.`. Đối với các thư viện khác, ta có thể sử dụng đường dẫn hệ thống tệp hoặc dạng `package:.`. Ví dụ:

```
import 'package:test/test.dart';
```

- **Đặt tên tiền tố cho thư viện**

Khi ta nhập nhiều thư viện có thể gây xung đột về tên, ta có thể sử dụng tiền tố để phân biệt. Ví dụ, nếu `library1` và `library2` đều có lớp `Element`, ta có thể sử dụng tiền tố như sau:

```
import 'package:lib1/lib1.dart';  
import 'package:lib2/lib2.dart' as lib2;  
  
// Sử dụng Element từ lib1.
```

```
Element element1 = Element();

// Sử dụng Element từ lib2.
lib2.Element element2 = lib2.Element();
```

- **Import một phần của thư viện**

Nếu ta chỉ cần một phần của thư viện, ta có thể chọn lọc **import** các thành phần cụ thể. Ví dụ:

```
// Chỉ import foo.
import 'package:lib1/lib1.dart' show foo;

// Import tất cả các thành phần ngoại trừ foo.
import 'package:lib2/lib2.dart' hide foo;
```

- **Tải thư viện dạng Lazy Loading**

Lazy Loading cho phép ta tải thư viện khi cần thiết, giúp giảm thời gian khởi động ban đầu của ứng dụng. Ta có thể sử dụng kỹ thuật này để tải các thành phần ít sử dụng hoặc tùy chọn. Ta sử dụng từ khóa **deferred as** để tải thư viện dạng **Lazy Loading**:

```
import 'package:greetings/hello.dart' deferred as hello;
```

Khi cần sử dụng thư viện, ta gọi hàm **loadLibrary()** để tải thư viện.

```
Future<void> greet() async {
  await hello.loadLibrary();
  hello.printGreeting();
}
```

Trong ví dụ trên, từ khóa **await** sẽ tạm dừng thực thi cho đến khi thư viện được tải xong. Ta có thể gọi **loadLibrary()** nhiều lần mà không gặp vấn đề gì, vì thư viện chỉ được tải một lần duy nhất.

2.3.6 Từ khoá – Keywords

Có một số từ khóa (keywords) được ngôn ngữ Dart dành riêng để sử dụng. Những từ khóa này không thể được sử dụng làm đặt định danh (tên biến, hàm, lớp, v.v.) trừ khi có ghi chú khác. Ngay cả khi được phép, việc sử dụng từ khóa làm định danh có thể gây nhầm lẫn cho các nhà phát triển khác khi đọc mã của mình và nên tránh.

abstract ²	as ²	assert	async ³
-----------------------	-----------------	--------	--------------------

await ¹	base ³	break	case
catch	class	const	continue
covariant ²	default	deferred ²	do
dynamic ²	else	enum	export ²
extends	extension ²	external ²	factory ²
false	final (var)	final (class)	finally
for	Function ²	get ²	hide ³
if	implements ²	import ²	in
interface ²	is	late ²	library ²
mixin ²	new	null	of ³
on ³	operator ²	part ²	required ²
rethrow	return	sealed ³	set ²
show ³	static ²	super	switch
sync ³	this	throw	true
try	type ²	typedef ²	var
void	when ³	with	while
yield ¹			

Bảng thể hiện một số từ khoá trong Dart

Ghi chú:

[1] Từ khóa này có thể được sử dụng làm định danh tùy thuộc vào ngữ cảnh.

[2] Từ khóa này không thể được sử dụng làm tên của một kiểu dữ liệu (như lớp, mixin, enum, kiểu mở rộng hoặc kiểu alias), tên của một mở rộng (extension) hoặc làm tiền tố import. Nó có thể được sử dụng làm định danh trong tất cả các trường hợp khác.

[3] Từ khóa này có thể được sử dụng làm định danh mà không có hạn chế.

2.4 KIỂU DỮ LIỆU

2.4.1 Kiểu dữ liệu tổng hợp

Dart cung cấp một hệ thống kiểu dữ liệu phong phú và đa dạng, giúp lập trình viên xây dựng các ứng dụng mạnh mẽ và hiệu quả. Trong phần này, ta sẽ tìm hiểu về các kiểu dữ liệu thông dụng trong Dart, bao gồm số nguyên, số thực, chuỗi, boolean, danh sách, cấu trúc dữ liệu Set, Map và các kiểu đặc biệt khác.

- **Kiểu số (Numbers)**

Dart có 2 loại kiểu số chính: số nguyên (int) và số thực (double).

Số nguyên (int): Đại diện cho các giá trị số nguyên không có phần thập phân. Ví dụ:

```
var x = 10;  
var hex = 0xDEADBEEF;
```

Số thực (double): Đại diện cho các giá trị số thực với phần thập phân. Ví dụ:

```
var y = 1.1;  
var exponents = 1.42e5;
```

Ta cũng có thể khai báo biến dưới dạng `num`. Khi đó, biến có thể chứa cả giá trị nguyên và giá trị thực.

```
num x = 1; // x có thể chứa cả giá trị int và double  
x += 2.5;
```

Ngoài ra ta có thể xem một số ví dụ về cách chuyển đổi giữa chuỗi và số

```
// Chuỗi -> số nguyên  
var one = int.parse('1');  
assert(one == 1);  
  
// Chuỗi -> số thực  
var onePointOne = double.parse('1.1');  
assert(onePointOne == 1.1);  
  
// Số nguyên -> chuỗi  
String oneAsString = 1.toString();  
assert(oneAsString == '1');  
  
// Số thực -> chuỗi  
String piAsString = 3.14159.toStringAsFixed(2);  
assert(piAsString == '3.14');
```

- **Chuỗi (String)**

Chuỗi trong Dart là một tập hợp các ký tự UTF-16. Ta có thể sử dụng dấu nháy đơn hoặc dấu nháy kép để tạo chuỗi. Ví dụ:

```
var s1 = 'Dấu nháy đơn cho chuỗi.';  
var s2 = "Dấu nháy kép cũng được sử dụng.";
```

Ta có thể chèn giá trị của một biểu thức vào trong chuỗi bằng cách sử dụng cú pháp `${expression}`. Nếu biểu thức chỉ là một biến đơn, ta có thể bỏ qua dấu ngoặc nhọn.

```
var name = 'Khuyên';
print(Xin chào, $name!);
```

Ta cũng có thể tạo chuỗi nhiều dòng bằng cách sử dụng ba dấu nháy đơn hoặc ba dấu nháy kép.

```
var s1 = '''
Đây là một chuỗi nhiều dòng
sử dụng ba dấu nháy đơn.
''';

var s2 = """Đây cũng là một chuỗi nhiều dòng
sử dụng ba dấu nháy kép.""";
```

Ở một số trường hợp, ta có thể tạo chuỗi với dữ liệu thô (tức không có bất kỳ ký tự đặc biệt nào được xử lý trong chuỗi) này, hãy thêm ký tự `r` trước chuỗi. Ví dụ:

```
var s = r'Đây là một chuỗi thô, ngay cả \n cũng không được xử lý.';
```

Trong Dart, ta có thể kết hợp các chuỗi bằng cách sử dụng các chuỗi liên tiếp hoặc sử dụng toán tử `+`:

```
var s1 = 'Chuỗi '
        'liên tiếp'
        " hoạt động qua cả dòng mới.";
assert(s1 ==
        'Chuỗi liên tiếp hoạt động qua cả '
        'dòng mới.');
```

```
var s2 = 'Toán tử + ' + 'cũng hoạt động.';
assert(s2 == 'Toán tử + cũng hoạt động.');
```

- **Boolean (bool)**

Kiểu dữ liệu boolean chỉ có hai giá trị: `true` và `false`. Ví dụ:

```
bool isTrue = true;
bool isFalse = false;
```

Ta có thể sử dụng boolean trong các câu lệnh điều kiện:

```
if (isTrue) {
```

```
print('Đúng!');  
} else {  
  print('Sai!');  
}
```

Dart không cho phép sử dụng các giá trị không phải boolean trong các điều kiện như các ngôn ngữ khác. Thay vào đó, ta phải kiểm tra giá trị cụ thể:

```
// Kiểm tra chuỗi rỗng  
var fullName = '';  
assert(fullName.isEmpty);  
  
// Kiểm tra giá trị bằng 0  
var hitPoints = 0;  
assert(hitPoints == 0);  
  
// Kiểm tra giá trị null  
var unicorn = null;  
assert(unicorn == null);  
  
// Kiểm tra giá trị NaN  
var iMeantToDoThis = 0 / 0;  
assert(iMeantToDoThis.isNaN);
```

- **Danh sách (List)**

Danh sách trong Dart được biểu thị bằng các giá trị hoặc biểu thức được phân tách bằng dấu phẩy, nằm trong dấu ngoặc vuông ([]).

```
var list = [1, 2, 3];
```

Dart tự động suy luận rằng list có kiểu `List<int>`. Nếu ta cố gắng thêm các đối tượng không phải là số nguyên vào danh sách này, sẽ xảy ra lỗi trong quá trình phân tích hoặc thực thi.

Ta có thể thêm dấu phẩy sau mục cuối cùng trong danh sách để tránh lỗi sao chép-dán.

```
var list = [  
  'Car',  
  'Boat',  
  'Plane',  
];
```

Danh sách sử dụng chỉ số bắt đầu từ 0. Ta có thể lấy độ dài của danh sách bằng thuộc tính `.length` và truy cập các giá trị của danh sách bằng toán tử chỉ số (`[]`):

```
var list = [1, 2, 3];
assert(list.length == 3);
assert(list[1] == 2);

list[1] = 1;
assert(list[1] == 1);
```

Để tạo một danh sách là hằng số tại thời điểm biên dịch, thêm từ khóa `const` trước danh sách:

```
var constantList = const [1, 2, 3];
constantList[1] = 1; // Dòng này sẽ gây ra lỗi.
```

- **Cấu trúc Set**

Cấu trúc Set là một tập hợp các phần tử không trùng lặp. Ví dụ:

```
var set = {1, 2, 3};
set.add(4);
set.add(1); // Không thêm phần tử trùng lặp
print(set); // {1, 2, 3, 4}
```

- **Cấu trúc Map**

Map là một tập hợp các cặp khóa-giá trị (key-value), cho phép lưu trữ và truy xuất giá trị theo khóa. Ví dụ:

```
var map = {'name': 'Khuyên', 'age': 26};
map['language'] = 'Tiếng Việt';
print(map); // {'name': 'Khuyên', 'age': 26, 'language': 'Tiếng Việt'}
```

- **Runes (Chuỗi Unicode 32-bit)**

Trong Dart, rune là các điểm mã UTF-32 của một chuỗi. Unicode xác định một giá trị số duy nhất cho mỗi chữ cái, chữ số và ký hiệu được sử dụng trong tất cả các hệ thống chữ viết của thế giới. Bởi vì chuỗi Dart là một chuỗi các đơn vị mã UTF-16, việc thể hiện các giá trị Unicode 32 bit trong một chuỗi đòi hỏi cú pháp đặc biệt. Cách thông thường để thể hiện điểm mã Unicode là `\uXXXX`, trong đó `XXXX` là giá trị thập lục phân 4 chữ số. Ví dụ: ký tự trái tim (♥) là `\u2665`. Để chỉ định nhiều hơn hoặc ít hơn 4 chữ

số hex, đặt giá trị trong dấu ngoặc nhọn. Ví dụ: biểu tượng cảm xúc cười (😊) là `\u{1f600}`.

Ta xem một chương trình minh họa sau:

```
Runes input = Runes( '\u2665 \u{1f605} \u{1f60e} \u{1f47b}');  
print(String.fromCharCode(input));
```

Kết quả của chương trình là:

♥ ☐ ☐ ☐

2.4.2 Records

Records trong Dart là một kiểu dữ liệu tổng hợp, không đổi (immutable) và ẩn danh (anonymous). Tương tự như các kiểu dữ liệu tổng hợp khác, records cho phép ta gộp nhiều đối tượng thành một đối tượng duy nhất. Tuy nhiên, không giống như các kiểu tổng hợp khác, records có kích thước cố định, không đồng nhất (heterogeneous) và được đánh kiểu rõ ràng.

Records là các giá trị thực; ta có thể lưu trữ chúng trong biến, lồng nhau, truyền chúng vào và ra khỏi các hàm và lưu trữ chúng trong các cấu trúc dữ liệu như danh sách Lists, Maps và Sets.

Biểu thức records là danh sách các trường có tên hoặc vị trí, phân cách bằng dấu phẩy và được bao quanh bởi dấu ngoặc đơn:

```
var record = ('first', a: 2, b: true, 'last');
```

2.4.3 Generics

Generics trong Dart là một cách để tạo ra các lớp, hàm và cấu trúc dữ liệu có thể làm việc với nhiều loại dữ liệu khác nhau. Các lợi ích của Generics bao gồm:

- *Kiểm tra lỗi trong quá trình viết mã:* Khi sử dụng Generics, Dart có thể kiểm tra lỗi ngay trong quá trình viết mã nguồn, giúp phát hiện lỗi sớm hơn.
- *Giảm trùng lặp mã:* Ta không cần viết lại mã nguồn cho mỗi loại dữ liệu khác nhau. Thay vào đó, ta chỉ cần viết một lần và sử dụng cho nhiều loại dữ liệu.

- *Cải thiện hiệu suất*: Khi sử dụng Generics, mã nguồn được tối ưu hóa tốt hơn, giúp cải thiện hiệu suất của chương trình.

- **Sử dụng Generics cho Class**

Ta có thể khai báo Generics cho một lớp bằng cách sử dụng cú pháp `<T>` ngay sau tên lớp. Ví dụ:

```
class Box<T> {  
    T? value;  
  
    Box(this.value);  
  
    T getValue() => value!;  
    void setValue(T value) => this.value = value;  
}
```

Trong ví dụ trên, `T` là một tham số kiểu đại diện cho loại dữ liệu mà ta sẽ xác định sau, đồng nghĩa với việc ta có thể sử dụng `Box` với bất kỳ loại dữ liệu nào:

```
void main() {  
    var intBox = Box<int>(123);  
    print(intBox.getValue()); // Output: 123  
  
    var stringBox = Box<String>("Hello");  
    print(stringBox.getValue()); // Output: Hello  
}
```

- **Sử dụng Generics cho Function**

Tương tự như lớp, ta cũng có thể khai báo Generics cho một hàm bằng cách sử dụng cú pháp `<T>` ngay trước kiểu trả về của hàm. Ví dụ:

```
T getFirst<T>(List<T> items) {  
    return items.first;  
}  
  
void main() {  
    var numbers = [1, 2, 3];  
    var firstNumber = getFirst<int>(numbers);  
    print(firstNumber); // Output: 1  
  
    var words = ["apple", "banana", "cherry"];
```

```
var firstWord = getFirst<String>(words);  
print(firstWord); // Output: apple  
}
```

• Sử dụng Generics cho Collections

Ta có thể sử dụng Generics với các Collections như List, Set và Map để xác định loại dữ liệu mà chúng lưu trữ:

```
void main() {  
  var intList = <int>[1, 2, 3];  
  var stringSet = <String>{'apple', 'banana', 'cherry'};  
  var intToStringMap = <int, String>{  
    1: 'one',  
    2: 'two',  
    3: 'three'  
  };  
  
  print(intList); // Output: [1, 2, 3]  
  print(stringSet); // Output: {apple, banana, cherry}  
  print(intToStringMap); // Output: {1: one, 2: two, 3: three}  
}
```

Lưu ý khi sử dụng Generics

Khi sử dụng Generics, ta có thể chỉ định rõ loại dữ liệu mà một collection chứa. Điều này giúp kiểm tra lỗi ngay trong quá trình biên dịch, tránh việc thêm các loại dữ liệu không phù hợp vào collection. Ví dụ, nếu ta muốn danh sách chỉ chứa các chuỗi (String), ta có thể khai báo như sau:

```
void main() {  
  var names = <String>[];  
  names.addAll(['Seth', 'Kathy', 'Lars']);  
  names.add(42); // Lỗi: Không thể thêm số vào danh sách Generic chuỗi  
}
```

Trong ví dụ trên, danh sách names được khai báo chỉ chứa các chuỗi (String). Nếu cố gắng thêm một số (number) vào danh sách này, Dart sẽ báo lỗi ngay lập tức. Điều này giúp phát hiện lỗi sớm và đảm bảo tính nhất quán của dữ liệu trong chương trình.

2.4.4 Typedefs

Typedefs trong Dart là một cách ngắn gọn để tạo các **alias** (bí danh) cho các loại dữ liệu. Điều này giúp mã nguồn trở nên rõ ràng và dễ đọc hơn, đặc biệt khi làm việc với các loại dữ liệu phức tạp.

Typedefs giúp đơn giản hóa cách biểu diễn các loại dữ liệu. Ví dụ, để tạo một bí danh cho **List<int>**, ta sử dụng cú pháp **typedef** như sau:

```
typedef IntList = List<int>;

void main() {
  IntList il = [1, 2, 3];
  print(il); // Output: [1, 2, 3]
}
```

Trong ví dụ trên, **IntList** là một **alias** cho **List<int>**. Điều này giúp mã nguồn trở nên ngắn gọn và dễ hiểu hơn.

Bên cạnh đó, ta cũng có thể khai báo **typedefs** với tham số kiểu. Điều này cho phép tạo ra các **alias** phức tạp hơn. Ví dụ:

```
typedef ListMapper<X> = Map<X, List<X>>;

void main() {
  Map<String, List<String>> m1 = {}; // Cách viết dài.
  ListMapper<String> m2 = {}; // Ngắn gọn và rõ ràng hơn.

  print(m1); // Output: {}
  print(m2); // Output: {}
}
```

Trước phiên bản Dart 2.13, **typedefs** chỉ được sử dụng cho các loại hàm. Tuy nhiên, kể từ phiên bản Dart 2.13 trở đi, **typedefs** đã được mở rộng để hỗ trợ nhiều loại dữ liệu hơn. Dù vậy, **typedefs** cho các hàm vẫn rất hữu ích trong nhiều trường hợp:

```
typedef Compare<T> = int Function(T a, T b);

int sort(int a, int b) => a - b;
```

```
void main() {  
  assert(sort is Compare<int>); // Kết quả: true!  
}
```

Trong ví dụ này, `Compare<T>` là một **alias** cho hàm `int Function(T a, T b)`. Điều này giúp mã nguồn trở nên rõ ràng và dễ hiểu hơn khi làm việc với các hàm phức tạp.

2.5 PATTERNS TRONG DART

2.5.1 Giới thiệu

Patterns trong Dart là một loại cú pháp dùng để kiểm tra và trích xuất dữ liệu từ các giá trị. Chúng cho phép kiểm tra xem một giá trị có dạng mong muốn hay không và sau đó trích xuất các phần của giá trị đó một cách ngắn gọn và dễ hiểu. Dart hỗ trợ patterns từ phiên bản Dart 3.0 trở lên.

Patterns có thể được sử dụng trong nhiều tình huống khác nhau như khai báo biến, gán giá trị và trong các vòng lặp hoặc cấu trúc điều khiển như switch-case.

2.5.2 Các dạng Pattern

- **Logical-or Pattern:** Kiểm tra nếu một trong các **subpattern** khớp.

```
var isPrimary = switch (color) {  
  Color.red || Color.yellow || Color.blue => true,  
  _ => false  
};
```

Ví dụ này kiểm tra xem `color` có phải là màu đỏ, vàng hay xanh không. Nếu đúng, giá trị trả về là `true`.

- **Logical-or Pattern:** Kiểm tra nếu tất cả các **subpattern** khớp.

```
switch ((1, 2)) {  
  case (var a, var b) && (var c, var d):  
    print('a = $a, b = $b, c = $c, d = $d');  
}
```

Ví dụ này kiểm tra nếu cả hai cặp giá trị (1, 2) đều khớp với mẫu (var a, var b) và (var c, var d). Tuy nhiên, cú pháp này không hợp lệ vì không thể có hai **subpattern** trùng nhau.

- **Relational Pattern:** So sánh giá trị với một hằng số.

```
String asciiCharType(int char) {  
  const space = 32;  
  const zero = 48;  
  const nine = 57;  
  
  return switch (char) {  
    < space => 'control',  
    == space => 'space',  
    > space && < zero => 'punctuation',  
    >= zero && <= nine => 'digit',  
    _ => ''  
  };  
}
```

Ví dụ này phân loại các ký tự ASCII dựa trên giá trị của chúng.

- **Cast Pattern:** Ép kiểu giá trị trước khi trích xuất

```
(num, Object) record = (1, 's');  
var (i as int, s as String) = record;
```

Ví dụ này ép kiểu các giá trị trong **record** thành **int** và **String**.

- **Null-check Pattern:** Kiểm tra giá trị không phải là **null** trước khi trích xuất

```
String? maybeString = 'nullable with base type String';  
switch (maybeString) {  
  case var s?:  
    print('Non-null string: $s');  
}
```

Ngoài ra còn một số dạng khác như **Constant Pattern**, **Variable Pattern**, **List Pattern**, **Map Pattern**, **Record Pattern**... cũng có cách hoạt động tương tự.

2.6 HÀM

Dart là một ngôn ngữ thuần hướng đối tượng, vì vậy ngay cả các hàm cũng là đối tượng và có kiểu `Function`. Điều này có nghĩa là các hàm có thể được gán cho các biến hoặc truyền làm tham số cho các hàm khác. Ta cũng có thể gọi một thể hiện (instance) của một lớp Dart như thể nó là một hàm.

2.6.1 Hàm `main()`

Mọi ứng dụng Dart đều phải có hàm `main()` là điểm khởi đầu của ứng dụng. Hàm `main()` trả về `void` và có thể có tham số tùy chọn `List<String>` để nhận các đối số dòng lệnh. Ví dụ:

```
void main() {
  print('Hello, World!');
}

void main(List<String> arguments) {
  print(arguments); // In ra các đối số dòng lệnh trong danh sách

  assert(arguments.length == 2);
  assert(int.parse(arguments[0]) == 1);
  assert(arguments[1] == 'test');
}
```

2.6.2 Khai báo hàm cơ bản

Hàm là một khối lệnh thực hiện một tác vụ gì đó, khối lệnh này được dùng nhiều lần nên gom chúng lại thành một hàm. Trong Dart mọi thứ đều là đối tượng nên hàm cũng là một đối tượng (kế thừa `Function`). Đây là một khai báo hàm:

```
double tinhTong(var a, double b, double c) {
  return a + b + c;
}

//Hoặc
double tinhTong(var a, double b, double c) {
  return a + b + c;
}
```

```
//Gọi hàm  
var x = tinhTong(1,2,3);  
print(x); //6.0
```

Như vậy khi khai báo hàm ta có thể có các thành phần sau:

- Chỉ ra kiểu giá trị trả về của hàm (ví dụ **double**), giá trị trả về bằng biểu thức của lệnh **return**. Với Dart thiếu việc khai báo kiểu giá trị trả về hàm vẫn hoạt động tốt. Còn khi có chỉ rõ kiểu trả về thì giá trị trong biểu thức **return** phải trùng với kiểu khai báo hàm.
- Tham số được liệt kê sau tên hàm trong cặp **()** như trên có ba tham số **a,b,c**
- Hàm kết thúc khi chạy hết khối lệnh hoặc gặp lệnh **return**, giá trị hàm là biểu thức sau **return**, nếu thiếu giá trị hàm sẽ là **null**
- Gọi hàm thì viết tên hàm và truyền đúng tham số theo thứ tự khai báo

2.6.3 Sử dụng cú pháp rút gọn với =>

Đối với các hàm chỉ chứa duy nhất một biểu thức, ta có thể sử dụng cú pháp rút gọn với **=>**. Ví dụ:

```
double tinhTong(var a, var b) {  
    return a + b;  
}  
  
// Có thể viết lại thành  
double tinhTong(var a, var b) => a + b;
```

Nghĩa là: **=> expression** là cách viết gọn của **{ return expression; }**

Lưu ý: Chỉ có biểu thức (**expression**) đứng đằng sau **=>**, không áp dụng cho câu lệnh (**statement**).

Ví dụ: Ta không thể để **if statement** đằng sau **=>** được. Tuy nhiên ta có thể sử dụng toán tử ba ngôi:

condition ? expr1 : expr2

2.6.4 Hàm với tên tham số (Named parameters)

Khi gọi 1 hàm (**function**), ta có truyền giá trị thông qua tên biến **paramName: value**, việc này giúp việc đọc code dễ dàng hơn trong các trường hợp hàm (**function**) có quá nhiều tham số (**parameter**).

Để làm được thế, hàm phải khai báo tham số ở trong **{ }**.

Ví dụ: **{param1, param2,...}**

```
String createFullName({String surName, String midName, String firstName}) {  
    return surName + midName + firstName;  
}  
  
void main() {  
    var result = createFullName(surName: 'Bui ', midName: 'Phu ', firstName: 'Khuyen '  
    ');  
    print(result); // Output: Bui Phu Khuyen  
}
```

Để đặt giá trị mặc định cho một tham số, ta có thể sử dụng cú pháp **paramName = defaultValue**. Điều này giúp tránh các lỗi do tham số không được cung cấp giá trị khi gọi hàm. Ví dụ:

```
String createFullName({String surName = 'Bui ', String midName = 'Phu ', String  
firstName = 'Khuyen '}) {  
    return surName + midName + firstName;  
}  
  
void main() {  
    var result = createFullName();  
    print(result); // Output: Bui Phu Khuyen  
  
    var result2 = createFullName(surName: 'Nguyen ');  
    print(result2); // Output: Nguyen Phu Khuyen  
}
```

Để bắt buộc người gọi phải cung cấp giá trị cho tham số, ta có thể sử dụng từ khóa **required**. Điều này đảm bảo rằng hàm sẽ nhận được các giá trị cần thiết khi được gọi. Ví dụ:

```
String createFullName({required String surName, required String midName, required  
String firstName}) {
```

```
    return surName + midName + firstName;
}

void main() {
    var result = createFullName(surName: 'Bui ', midName: 'Phu ', firstName: 'Khuyen ');
    print(result); // Output: Bui Phu Khuyen

    // Gọi hàm mà không cung cấp đủ các giá trị bắt buộc sẽ gây lỗi biên dịch
    var result2 = createFullName(surName: 'Nguyen ');
}
```

2.6.5 Hàm với tham số tùy chọn (Optional positional parameters)

Các tham số tùy chọn trong hàm cho phép ta gọi hàm mà không cần cung cấp đầy đủ tất cả các tham số. Khi các tham số tùy chọn không được cung cấp, chúng sẽ nhận giá trị null theo mặc định. Để khai báo các tham số tùy chọn, ta sử dụng dấu ngoặc vuông [].

Ví dụ dưới đây minh họa cách sử dụng tham số tùy chọn trong hàm:

```
double sum(double a, [double? b, double? c]) {
    var result = a;
    if (b != null) {
        result += b;
    }
    result += (c != null) ? c : 0;
    return result;
}

void main() {
    print(sum(1));           // Output: 1.0
    print(sum(1, 2));        // Output: 3.0
    print(sum(1, 2, 3));     // Output: 6.0
}
```

Trong ví dụ trên:

- Hàm **sum** có một tham số bắt buộc **a** và hai tham số tùy chọn **b** và **c**.

- Khi gọi hàm `sum`, nếu `b` không được cung cấp, nó sẽ nhận giá trị `null` và không được thêm vào kết quả.
- Tương tự, nếu `c` không được cung cấp, nó sẽ nhận giá trị `null` và hàm sẽ thêm `0` vào kết quả thay vì giá trị của `c`.
- Nhờ vào các tham số tùy chọn, ta có thể linh hoạt gọi hàm `sum` với một, hai hoặc ba tham số mà không gặp lỗi.

Để cung cấp giá trị mặc định cho các tham số tùy chọn, ta có thể sử dụng cú pháp `=` để gán giá trị mặc định. Ví dụ:

```
double sum(double a, [double b = 0, double c = 0]) {  
    return a + b + c;  
}  
  
void main() {  
    print(sum(1));           // Output: 1.0  
    print(sum(1, 2));        // Output: 3.0  
    print(sum(1, 2, 3));     // Output: 6.0  
}
```

2.6.6 Hàm ẩn danh (Anonymous functions)

Trong Dart, ta có thể tạo các hàm không có tên gọi là hàm ẩn danh, hay còn gọi là `lambda` hoặc `closure`. Chúng thường được sử dụng trong các ngữ cảnh mà ta không cần sử dụng tên hàm, ví dụ như khi truyền hàm như tham số cho hàm khác.

Để tạo hàm ẩn danh, ta có thể sử dụng cú pháp tương tự như khi khai báo hàm có tên, nhưng bỏ qua phần kiểu trả về và tên hàm.

```
(var a, var b) {  
    return a + b;  
};
```

Hoặc sử dụng cú pháp mũi tên `() => {}`:

```
(var a, var b) => {  
    return a + b;  
};
```

Nếu hàm chỉ chứa một biểu thức, ta có thể viết gọn hơn bằng cách bỏ từ khóa `return` và dấu ngoặc nhọn:

```
(var a, var b) => a + b;
```

Khai báo trên là hàm ẩn danh, nhưng để sử dụng nó thì ra lệnh cho nó chạy luôn hoặc gán nó vào một biến rồi dùng biến gọi hàm.

```
//Chạy hàm ngay sau khi khai báo
var x = ((var a, var b) {
    return a + b;
})(5, 6);
print(x); // Output: 11

//Gán hàm ẩn danh vào một biến, sau đó sử dụng biến đó để gọi hàm
var ham = (var a, var b) {
    return a + b;
};
print(ham(10, 11)); // Output: 21
```

Hàm ẩn danh rất tiện dụng để làm tham số (callback) trong các hàm khác, thậm chí khai báo luôn một hàm ẩn danh ngay trong danh sách tham số của hàm.

```
void f1(var a, var b, Function printMessage) {
    var c = a + b;
    printMessage(c);
}

void main() {
    f1(1, 2, (x) {
        print('Tổng là: $x');
    }); // Output: Tổng là: 3

    f1(1, 2, (z) => print('SUM = $z')); // Output: SUM = 3
}
```

Trong ví dụ trên:

- Hàm `f1` nhận ba tham số: `a`, `b` và `printMessage`. Tham số `printMessage` là một hàm ẩn danh.

- Khi gọi `f1`, ta có thể truyền vào một hàm ẩn danh để xử lý kết quả của phép cộng `a + b`.

Bằng cách này, hàm ẩn danh giúp ta viết mã ngắn gọn, dễ đọc và linh hoạt hơn khi cần xử lý các tác vụ nhỏ, tạm thời trong mã nguồn của mình.

2.7 CẤU TRÚC LẶP VÀ Rẽ NHÁNH

2.7.1 Cấu trúc lặp

Trong Dart, ta có thể điều khiển luồng chạy của mã thông qua các cấu trúc lặp và các câu lệnh hỗ trợ:

- **Vòng lặp `for`**

Ta có thể lặp lại mã bằng cách sử dụng vòng lặp `for` tiêu chuẩn. Ví dụ:

```
// Ví dụ: In ra từ 1 đến 5
for (var i = 1; i <= 5; i++) {
  print(i);
}
// Kết quả: 1 2 3 4 5
```

Đôi khi ta không cần biết chỉ số hiện tại khi lặp qua một **Iterable** như **List** hoặc **Set**. Trong trường hợp đó, sử dụng vòng lặp `for-in` sẽ làm mã dễ đọc hơn:

```
// Ví dụ: In ra các tên trong danh sách
var names = ['An', 'Bình', 'Chi'];
for (var name in names) {
  print(name);
}
// Kết quả: An Bình Chi
```

- **Vòng lặp `while`**

Vòng lặp `while` sẽ lặp lại một đoạn mã cho đến khi điều kiện trở thành `false`:

```
// Ví dụ: In ra từ 1 đến 5
var i = 1;
while (i <= 5) {
  print(i);
  i++;
}
```

```
// Kết quả: 1 2 3 4 5
```

- **Vòng lặp do-while**

Vòng lặp **do-while** sẽ luôn thực hiện ít nhất một lần trước khi kiểm tra điều kiện:

```
// Ví dụ: In ra từ 1 đến 5
var i = 1;
do {
  print(i);
  i++;
} while (i <= 5);
// Kết quả: 1 2 3 4 5
```

- **Câu lệnh break và continue**

Câu lệnh **break** dùng để thoát khỏi vòng lặp ngay lập tức:

```
// Ví dụ: Thoát khỏi vòng lặp khi giá trị bằng 3
for (var i = 1; i <= 5; i++) {
  if (i == 3) {
    break;
  }
  print(i);
}
// Kết quả: 1 2
```

Câu lệnh **continue** dùng để bỏ qua phần còn lại của vòng lặp hiện tại và tiếp tục với lần lặp tiếp theo:

```
// Ví dụ: Bỏ qua giá trị 3 và tiếp tục vòng lặp
for (var i = 1; i <= 5; i++) {
  if (i == 3) {
    continue;
  }
  print(i);
}
// Kết quả: 1 2 4 5
```

2.7.2 Cấu trúc rẽ nhánh

Trong Dart, ta có thể điều khiển luồng chạy của chương trình bằng cách sử dụng các cấu trúc rẽ nhánh.

- **Câu lệnh if**

Câu lệnh `if` kiểm tra một điều kiện và thực thi một khối mã nếu điều kiện đó đúng. Ta cũng có thể sử dụng `else if` và `else` để kiểm tra thêm các điều kiện khác.

```
void main() {  
  bool isRaining = false;  
  bool isSnowing = true;  
  
  if (isRaining) {  
    print('Mang theo áo mưa.');  } else if (isSnowing) {  
    print('Mặc áo khoác.');  } else {  
    print('Hạ mui xe xuống.');  }  
}
```

- **Câu lệnh if-case**

Câu lệnh `if-case` cho phép ta kiểm tra và trích xuất dữ liệu từ một giá trị dựa trên mẫu (pattern).

```
void main() {  
  var pair = [3, 4];  
  
  if (pair case [int x, int y]) {  
    print('Cặp tọa độ là: $x, $y');  } else {  
    print('Dữ liệu không hợp lệ.');  }  
}  
// Kết quả: Cặp tọa độ là 3, 4
```

- **Câu lệnh switch-case**

Câu lệnh `switch` đánh giá một biểu thức và so sánh giá trị của nó với các trường hợp (case) khác nhau. Nếu giá trị khớp với một trường hợp, khối mã tương ứng sẽ được thực thi.

```
void main() {  
  var command = 'OPEN';
```

```
switch (command) {  
  case 'CLOSED':  
    print('Đóng cửa.');
```

```
    break;  
  case 'PENDING':  
    print('Đang chờ xử lý.');
```

```
    break;  
  case 'APPROVED':  
    print('Đã phê duyệt.');
```

```
    break;  
  case 'DENIED':  
    print('Bị từ chối.');
```

```
    break;  
  case 'OPEN':  
    print('Mở cửa.');
```

```
    break;  
  default:  
    print('Lệnh không xác định.');
```

```
}  
}
```

2.8 LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

2.8.1 Kiến thức cơ bản

Dart là một ngôn ngữ lập trình hướng đối tượng, nơi mọi thứ đều là đối tượng và các đối tượng này được tạo ra từ các lớp (class).

- **Sử dụng các thành viên của lớp**

Mỗi đối tượng trong Dart có thể có các thuộc tính (biến) và phương thức (hàm). Thuộc tính lưu trữ dữ liệu và phương thức thực hiện các hành động.

Sử dụng dấu chấm (.) để tham chiếu đến một thuộc tính hoặc phương thức:

```
class Point {  
  double x, y;  
  
  Point(this.x, this.y);  
  
  double distanceTo(Point other) {  
    var dx = x - other.x;
```



```
    var dy = y - other.y;
    return sqrt(dx * dx + dy * dy);
}
}

void main() {
    //Khởi tạo đối tượng Point
    var p = Point(2, 2);

    // Gọi phương thức distanceTo() trên p.
    double distance = p.distanceTo(Point(4, 4));
    print(distance); // In ra khoảng cách
}
```

Ở ví dụ trên, lớp **Point** có hai thuộc tính **x** và **y** và một phương thức **distanceTo** để tính khoảng cách từ điểm hiện tại đến một điểm khác. Trong hàm **main**, ta tạo một đối tượng của **Point** và sử dụng các thuộc tính và phương thức của nó.

- **Thuộc tính**

Dưới đây là cách khai báo thuộc tính trong Dart

```
class Point {
    double? x; // Thuộc tính instance x, ban đầu là null.
    double? y; // Thuộc tính instance y, ban đầu là null.
    double z = 0; // Thuộc tính instance z, ban đầu là 0.
}

void main() {
    var point = Point();
    point.x = 4; // Sử dụng setter method cho x.
    assert(point.x == 4); // Sử dụng getter method cho x.
    assert(point.y == null); // Giá trị mặc định là null.
}
```

Ở ví dụ trên, ta khai báo ba thuộc tính **x**, **y** và **z** trong lớp **Point**. Ban đầu, **x** và **y** có giá trị là **null**, trong khi **z** có giá trị là **0**. Trong hàm **main**, chúng ta tạo một đối tượng của **Point** và thiết lập giá trị cho thuộc tính **x**.

Khi ta khởi tạo một thuộc tính tại nơi nó được khai báo, giá trị sẽ được gán khi đối tượng được tạo, trước khi **constructor** và danh sách khởi tạo của nó thực thi. Do đó,

biểu thức khởi tạo (sau dấu =) của một thuộc tính **instance non-late** không thể truy cập với từ khoá **this**.

```
double initialX = 1.5;

class Point {
  double? x = initialX; // OK, có thể truy cập các khai báo không phụ thuộc vào `this`.

  // LỖI, không thể truy cập `this` trong khởi tạo non-late:
  double? y = this.x;

  // OK, có thể truy cập `this` trong khởi tạo `late`:
  late double? z = this.x;

  // OK, `this.fieldName` là khai báo tham số, không phải biểu thức:
  Point(this.x, this.y);
}
```

Ngoài ra, thuộc tính có thể là **final**, nghĩa là chúng phải được gán giá trị duy nhất một lần. Ta có thể khởi tạo thuộc tính **final** tại nơi khai báo, bằng **constructor** hoặc bằng danh sách khởi tạo của **constructor**.

```
class ProfileMark {
  final String name;
  // Thuộc tính được khởi tạo tại nơi khai báo
  final DateTime start = DateTime.now();

  ProfileMark(this.name); // Constructor
  ProfileMark.unnamed() : name = ''; // Constructor với initializer list
}
```

Ở đây, **name** là một thuộc tính **final** được khởi tạo thông qua **constructor** và **start** được khởi tạo với thời gian hiện tại khi đối tượng được tạo.

- **Phương thức khởi tạo**

Ta có thể tạo đối tượng bằng cách sử dụng phương thức khởi tạo (**constructors**). Tên **constructor** có thể là **ClassName** hoặc **ClassName.identifier**. Xét ví dụ minh họa sau:

```
class Point {
```

```
double x, y;

Point(this.x, this.y);
Point.fromJson(Map<String, double> json) : x = json['x']!, y = json['y']!;
}

void main() {
  var p1 = Point(2, 2); // Sử dụng constructor mặc định
  var p2 = Point.fromJson({'x': 1, 'y': 2}); // Sử dụng named constructor
}
```

- **Constructor** `Point` khởi tạo các thuộc tính `x` và `y` từ các tham số truyền vào.
- Named constructor `Point.fromJson` khởi tạo các thuộc tính `x` và `y` từ một kiểu dữ liệu `Map JSON`.

- **Thuộc tính và phương thức static**

Trong Dart, thuộc tính và phương thức tĩnh (**static**) được sử dụng để chia sẻ dữ liệu và chức năng giữa tất cả các đối tượng của một lớp. Điều này có nghĩa là ta không cần phải tạo ra một đối tượng để truy cập vào các thuộc tính hoặc phương thức này.

Thuộc tính tĩnh là thuộc tính cấp lớp, được sử dụng cho các giá trị chung hoặc các hằng số mà không thay đổi giữa các đối tượng khác nhau của lớp.

```
class Queue {
  static const int initialCapacity = 16; // Thuộc tính tĩnh
}

void main() {
  // Truy cập thuộc tính tĩnh mà không cần tạo đối tượng
  print(Queue.initialCapacity); // Kết quả: 16
}
```

Phương thức tĩnh không hoạt động trên một đối tượng cụ thể nào và không có quyền truy cập vào `this`. Chúng thường được sử dụng cho các chức năng không liên quan đến trạng thái của bất kỳ đối tượng cụ thể nào.

```
import 'dart:math';

class Point {
  double x, y;
```

```
Point(this.x, this.y);

// Phương thức tĩnh để tính khoảng cách giữa hai điểm
static double distanceBetween(Point a, Point b) {
    var dx = a.x - b.x;
    var dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

void main() {
    var a = Point(2, 2);
    var b = Point(4, 4);

    // Truy cập phương thức tĩnh mà không cần tạo đối tượng mới
    var distance = Point.distanceBetween(a, b);
    print(distance); // In ra khoảng cách giữa a và b
}
```

- **Phương thức getters, setter**

Trong Dart, **getters** và **setters** là các phương thức đặc biệt cung cấp quyền truy cập đọc và ghi vào các thuộc tính của một đối tượng. Ta có thể tạo thêm các thuộc tính bằng cách triển khai **getters** và **setters** thông qua các từ khóa **get** và **set**. Ví dụ:

```
class Rectangle {
    double left, top, width, height;

    Rectangle(this.left, this.top, this.width, this.height);

    // Định nghĩa hai thuộc tính tính toán: right và bottom.
    double get right => left + width; // Getter cho right
    set right(double value) => left = value - width; // Setter cho right

    double get bottom => top + height; // Getter cho bottom
    set bottom(double value) => top = value - height; // Setter cho bottom
}

void main() {
    var rect = Rectangle(3, 4, 20, 15);
    assert(rect.left == 3); // Kiểm tra giá trị của left
    rect.right = 12; // Thiết lập giá trị mới cho right, tự động cập nhật left
}
```

```
assert(rect.left == -8); // Kiểm tra lại giá trị của left sau khi cập nhật  
}
```

• Lớp Object

Lớp **Object** là lớp cơ sở của tất cả các lớp trong Dart. Điều này có nghĩa là mọi lớp, phương thức và thuộc tính mà ta định nghĩa đều mở rộng từ lớp này. Nhờ đó, mọi lớp đều có một số phương thức và thuộc tính chung được kế thừa từ **Object**.

Các thuộc tính và phương thức chung của lớp **Object**

- **hashCode**: Thuộc tính chứa mã băm (**hash code**) của đối tượng. Mã băm này được sử dụng để so sánh và quản lý các đối tượng trong các cấu trúc dữ liệu như **hash map**.
- **toString()**: Phương thức trả về chuỗi mô tả đối tượng. Phương thức này thường được ghi đè để cung cấp thông tin mô tả đối tượng cụ thể hơn.
- **==**: Toán tử so sánh hai đối tượng bằng cách so sánh mã băm của chúng. Ta có thể ghi đè phương thức này để định nghĩa cách so sánh đối tượng cụ thể.

Ví dụ:

```
class Product {  
  String name;  
  double price;  
  
  Product(this.name, this.price);  
  
  // Ghi đè phương thức toString() để cung cấp chuỗi mô tả đối tượng  
  @override  
  String toString() {  
    return 'Product: $name, Price: \${price.toStringAsFixed(2)}';  
  }  
  
  // Ghi đè phương thức hashCode để cung cấp mã băm cho đối tượng  
  @override  
  int get hashCode {  
    return name.hashCode ^ price.hashCode;  
  }  
  
  // Ghi đè toán tử == để so sánh hai đối tượng Product
```

```
@override
bool operator ==(Object other) {
    if (identical(this, other)) return true;
    if (other is! Product) return false;
    return other.name == name && other.price == price;
}

void main() {
    var product1 = Product('Laptop', 1500.00);
    var product2 = Product('Laptop', 1500.00);
    var product3 = Product('Smartphone', 800.00);

    print(product1); // Kết quả: Product: Laptop, Price: $1500.00
    print(product2); // Kết quả: Product: Laptop, Price: $1500.00
    print(product3); // Kết quả: Product: Smartphone, Price: $800.00

    print(product1 == product2); // Kết quả: true
    print(product1 == product3); // Kết quả: false

    print(product1.hashCode); // Kết quả: mã hash của product1
    print(product2.hashCode); // Kết quả: mã hash của product2
    print(product3.hashCode); // Kết quả: mã hash của product3
}
```

Giải thích:

- Phương thức **toString()**: Được ghi đè để trả về chuỗi mô tả chi tiết của đối tượng **Product**.
- Thuộc tính **hashCode**: Được ghi đè để trả về mã băm của đối tượng, kết hợp mã băm của tên và giá sản phẩm.
- Toán tử **==**: Được ghi đè để so sánh hai đối tượng **Product** dựa trên tên và giá của chúng.

Khi chạy đoạn mã trên, ta sẽ thấy kết quả của các phương thức và toán tử của lớp **Object** được hiển thị, cho thấy cách mà các đối tượng được mô tả và so sánh.

2.8.2 Tính kế thừa

- Tổng quan

Trong Dart, ta có thể sử dụng từ khóa **extends** để tạo một lớp con (**subclass**) kế thừa từ một lớp cha (**superclass**) và **super** để tham chiếu đến lớp cha. Xét ví dụ sau:

```
class Product {
  double price;
  int quantity;
  String name;

  Product(this.price, {this.quantity = 0});

  void showTotal() {
    print('Total Price is: ${price * quantity}');
  }
}

class Tablet extends Product {
  double width = 0;
  double height = 0;

  Tablet(double price) : super(price, quantity: 1) {
    this.name = "IPad Pro"; // Truy vấn thuộc tính từ lớp cha
  }

  @override
  void showTotal() {
    print('Name Tablet is: $name');
    super.showTotal(); // Gọi đến phương thức ở lớp cha
  }
}

void main() {
  var tablet = Tablet(600);
  tablet.showTotal();
  // Kết quả:
  // Name Tablet is: IPad Pro
  // Total Price is: 600
}
```

Giải thích ví dụ:

- Khởi tạo tại lớp con và truy vấn đến lớp cha:
 - Trong lớp **Tablet**, phương thức khởi tạo **Tablet(double price)** sử dụng **super(price, quantity: 1)** để gọi phương thức khởi tạo của lớp cha **Product**.
 - Sau khi phương thức khởi tạo của lớp cha chạy xong, thuộc tính **name** được thiết lập giá trị **"iPad Pro"**.
- Nạp chồng phương thức:
 - Lớp **Tablet** nạp chồng phương thức **showTotal()** của lớp cha bằng cách sử dụng từ khóa **@override**.
 - Trong phương thức nạp chồng, **super.showTotal()** được sử dụng để gọi đến phương thức **showTotal()** của lớp cha.

- **Phương thức noSuchMethod**

Để phát hiện hoặc phản hồi lại khi ta cố gắng sử dụng một phương thức hoặc thuộc tính không tồn tại, lúc này ta có thể nạp chồng phương thức **noSuchMethod()**:

```
class A {  
  @override  
  void noSuchMethod(Invocation invocation) {  
    print('You tried to use a non-existent member: ${invocation.memberName}');  
  }  
}  
  
void main() {  
  var a = A();  
  a.nonExistentMethod(); // Kết quả: You tried to use a non-existent member:  
  Symbol("nonExistentMethod")  
}
```

2.8.3 Abstract, Interface và Mixin

Dart hỗ trợ các khái niệm về lớp trừu tượng **abstract**, **interface** và **mixin** để giúp quản lý và tổ chức mã nguồn hiệu quả hơn trong lập trình hướng đối tượng.

- **Lớp trừu tượng abstract**

Lớp trừu tượng là lớp không thể được tạo ra đối tượng trực tiếp. Nó chỉ được sử dụng để làm cơ sở cho các lớp con kế thừa. Một phương thức trong lớp trừu tượng có thể chỉ khai báo tên mà không có nội dung, được gọi là phương thức trừu tượng. Các lớp kế thừa bắt buộc phải định nghĩa lại nội dung cho các phương thức trừu tượng này.

Xét ví dụ minh họa sau:

```
abstract class A {  
    // Khai báo các thuộc tính  
    var name = 'My Abstract Class';  
  
    // Khai báo phương thức trừu tượng  
    void displayInformation();  
}  
  
class B extends A {  
    @override  
    void displayInformation() {  
        print(this.name);  
    }  
}  
  
void main() {  
    var i = B();  
    i.displayInformation(); // Kết quả: My Abstract Class  
}
```

Giải thích ví dụ trên:

- Lớp **A** là lớp trừu tượng với một thuộc tính **name** và một phương thức trừu tượng **displayInformation()**.
- Lớp **B** kế thừa từ lớp **A** và định nghĩa lại phương thức **displayInformation()** để in ra giá trị của **name**.
- Khi tạo một đối tượng của lớp **B** và gọi phương thức **displayInformation()**, nó sẽ in ra "My Abstract Class".

- **Interface**

Trong Dart, mọi lớp đều là một **interface** mặc định. Khi một lớp triển khai một **interface**, lớp đó phải định nghĩa lại tất cả các phương thức và thuộc tính của **interface** đó. Để triển khai một **interface**, sử dụng từ khóa **implements**.

```
class B {
  String name = 'Class B';

  void displayInformation() {
    print('Information from B');
  }
}

class C implements B {
  @override
  String name;

  @override
  void displayInformation() {
    print('Information from C');
  }
}

void main() {
  var c = C();
  c.displayInformation(); // Kết quả: Information from C
}
```

Giải thích ví dụ trên:

- Lớp **B** có một thuộc tính **name** và một phương thức **displayInformation()**.
- Lớp **C** triển khai **interface** từ lớp **B** bằng từ khóa **implements** và phải định nghĩa lại tất cả các phương thức và thuộc tính của **B**.
- Khi tạo một đối tượng của lớp **C** và gọi phương thức **displayInformation()**, nó sẽ in ra "Information from C".

- **Mixin**

Mixin là một lớp có chứa các phương thức và thuộc tính dùng để gộp vào một lớp khác. Mixin không thể được sử dụng để tạo đối tượng trực tiếp. Để sử dụng mixin, sử dụng từ khóa `with`.

```
mixin M {  
  void showSomething() {  
    print('Print message ...');  
  }  
}  
  
class B {  
  String name = 'Class B';  
  
  void displayInformation() {  
    print('Information from B');  
  }  
}  
  
class C extends B with M {  
  @override  
  String name;  
  
  @override  
  void displayInformation() {  
    showSomething();  
  }  
}  
  
void main() {  
  var c = C();  
  c.displayInformation(); // Kết quả: Print message ...  
}
```

Giải thích ví dụ trên:

- Mixin `M` có một phương thức `showSomething()` in ra một thông báo.
- Lớp `C` kế thừa từ lớp `B` và sử dụng mixin `M` với từ khóa `with`.
- Trong phương thức `displayInformation()` của lớp `C`, nó gọi phương thức `showSomething()` từ mixin `M`.

- Khi tạo một đối tượng của lớp `C` và gọi phương thức `displayInformation()`, nó sẽ in ra "Print message ...".

2.9 XỬ LÝ LỖI

Trong Dart, ta có thể ném (**throw**) và bắt (**catch**) các ngoại lệ (**exceptions**). Ngoại lệ là các lỗi cho biết có điều gì đó bất ngờ đã xảy ra. Nếu ngoại lệ không được bắt, chương trình sẽ bị tạm dừng và thường bị kết thúc.

Khác với Java, tất cả các ngoại lệ trong Dart đều là ngoại lệ không kiểm tra. Điều này có nghĩa là các phương thức không cần khai báo những ngoại lệ mà chúng có thể ném và ta không bắt buộc phải bắt bất kỳ ngoại lệ nào.

Dart cung cấp các loại **Exception** và **Error**, cũng như nhiều loại con được định nghĩa sẵn. Ta cũng có thể định nghĩa ngoại lệ của riêng mình. Dart cho phép ném bất kỳ đối tượng nào không phải là `null` như một ngoại lệ.

- **Ném ngoại lệ**

Dưới đây là ví dụ về cách ném một ngoại lệ:

```
void checkAge(int age) {  
  if (age < 18) {  
    throw Exception('Tuổi phải từ 18 trở lên.');  }  
  print('Tuổi hợp lệ.');}  
  
void main() {  
  try {  
    checkAge(17);  
  } catch (e) {  
    print('Lỗi: $e');  }  
}  
  
// Kết quả: Lỗi: Exception: Tuổi phải từ 18 trở lên.
```

Ta cũng có thể ném các đối tượng tùy ý:

```
void main() {  
  try {  
    throw 'Có lỗi xảy ra!';  
  } catch (e) {  
    print('Lỗi: $e');  
  }  
}  
  
// Kết quả: Lỗi: Có lỗi xảy ra!
```

Lưu ý: Trong môi trường triển khai dự án (**production**), ta nên ném các loại thực thi **Error** hoặc **Exception**.

- **Bắt ngoại lệ**

Bắt ngoại lệ dừng sự lan truyền của ngoại lệ đó (trừ khi ta ném lại ngoại lệ). Điều này cho phép ta xử lý ngoại lệ:

```
void breedMoreLlamas() {  
  throw Exception('Hết lạc đà!');  
}  
  
void buyMoreLlamas() {  
  print('Mua thêm lạc đà.');}  
  
void main() {  
  try {  
    breedMoreLlamas();  
  } on Exception catch (e) {  
    print('Lỗi: $e');  
    buyMoreLlamas();  
  }  
}  
  
// Kết quả:  
// Lỗi: Exception: Hết lạc đà!  
// Mua thêm lạc đà.
```

Để xử lý mã có thể ném nhiều loại ngoại lệ khác nhau, ta có thể chỉ định nhiều khối bắt (**catch**) khác nhau. Khối bắt đầu tiên khớp với loại đối tượng bị ném sẽ xử lý ngoại lệ đó:

```
void main() {  
  try {  
    throw FormatException('Định dạng không hợp lệ!');  
  } on FormatException catch (e) {  
    print('Ngoại lệ định dạng: $e');  
  } on Exception catch (e) {  
    print('Ngoại lệ: $e');  
  } catch (e) {  
    print('Lỗi không xác định: $e');  
  }  
}
```

// Kết quả: Ngoại lệ định dạng: FormatException: Định dạng không hợp lệ!

Ta cũng có thể chỉ định một hoặc hai tham số cho **catch()**. Tham số đầu tiên là ngoại lệ bị ném và tham số thứ hai là dấu vết ngăn xếp (một đối tượng **StackTrace**).

```
void main() {  
  try {  
    throw FormatException('Lỗi định dạng!');  
  } on Exception catch (e, s) {  
    print('Ngoại lệ: $e');  
    print('Dấu vết ngăn xếp: $s');  
  }  
}
```

// Kết quả:
// Ngoại lệ: FormatException: Lỗi định dạng!
// Dấu vết ngăn xếp: (in ra thông tin ngăn xếp)

- **Ném lại ngoại lệ**

Để xử lý một phần ngoại lệ, trong khi vẫn cho phép nó lan truyền, ta sử dụng từ khóa **rethrow**.

```
void misbehave() {  
  try {  
    throw 'Có lỗi!';  
  } catch (e) {
```

```
    print('Đã xử lý một phần: $e');  
    rethrow; // Cho phép người gọi thấy ngoại lệ.  
  }  
}  
  
void main() {  
  try {  
    misbehave();  
  } catch (e) {  
    print('Xử lý xong: $e');  
  }  
}  
  
// Kết quả:  
// Đã xử lý một phần: Có lỗi!  
// Xử lý xong: Có lỗi!
```

- **Khối finally**

Để đảm bảo rằng một số mã luôn chạy dù có xảy ra ngoại lệ hay không, ta sử dụng khối **finally**. Nếu không có khối bắt nào khớp với ngoại lệ, ngoại lệ sẽ lan truyền sau khi khối **finally** chạy xong:

```
void main() {  
  try {  
    throw Exception('Lỗi xảy ra!');  
  } catch (e) {  
    print('Đã bắt lỗi: $e');  
  } finally {  
    print('Luôn chạy khối này.');  }  
}  
  
// Kết quả:  
// Đã bắt lỗi: Exception: Lỗi xảy ra!  
// Luôn chạy khối này.
```

- **Kiểm tra lỗi bằng assert**

Trong quá trình phát triển (**development**), sử dụng câu lệnh **assert(<condition>, <optionalMessage>)**; để kiểm tra một điều kiện nhất định. Nếu điều kiện đó không

đúng, chương trình sẽ dừng lại và báo lỗi. Đây là cách hữu ích để bắt lỗi trong quá trình phát triển.

```
void main() {  
    var number = 50;  
  
    // Đảm bảo giá trị nhỏ hơn 100.  
    assert(number < 100, 'Giá trị phải nhỏ hơn 100');  
  
    // Đảm bảo đây là URL https.  
    var urlString = 'https://example.com';  
    assert(urlString.startsWith('https'), 'URL phải bắt đầu bằng "https"');  
  
    print('Mọi thứ đều đúng.');
```

}

```
// Kết quả (nếu không có lỗi):  
// Mọi thứ đều đúng.
```

Câu lệnh `assert` sẽ bị bỏ qua trong quá trình triển khai dự án (**production**).

2.10 XỬ LÝ ĐỒNG THỜI

2.10.1 Giới thiệu

Trong lập trình, có hai khái niệm quan trọng là đồng bộ (**synchronous**) và bất đồng bộ (**asynchronous**).

- Lập trình Đồng bộ: Các tác vụ được thực hiện tuần tự, tác vụ tiếp theo chỉ được thực hiện khi tác vụ hiện tại hoàn thành. Điều này có thể gây ra tình trạng "chờ đợi" khi một tác vụ mất nhiều thời gian để hoàn thành. Ví dụ:

```
void main() {  
    print('Bắt đầu đọc dữ liệu từ tệp...');  
    String data = readFileSync('data.txt'); // Giả sử hàm này đọc tệp đồng bộ  
    print('Dữ liệu đọc được: $data');  
    print('Hoàn thành');
```

}

- Lập trình Bất đồng bộ: Cho phép thực hiện nhiều tác vụ cùng một lúc mà không phải chờ đợi tác vụ trước hoàn thành. Điều này làm tăng hiệu suất và khả năng phản hồi của chương trình. Ví dụ:

```
Future<String> readFileAsync(String path) async {  
  // Giả sử hàm này đọc tệp không đồng bộ  
  await Future.delayed(Duration(seconds: 2)); // Giả lập thời gian chờ  
  return 'Nội dung tệp';  
}  
  
void main() async {  
  print('Bắt đầu đọc dữ liệu từ tệp...');  
  String data = await readFileAsync('data.txt');  
  print('Dữ liệu đọc được: $data');  
  print('Hoàn thành');  
}
```

2.10.2 Future và Stream

Future đại diện cho một giá trị hoặc lỗi sẽ có sẵn trong tương lai. Nó thường được sử dụng cho các hoạt động không đồng bộ như gọi API, đọc/ghi tệp hoặc bất kỳ hoạt động nào cần thời gian để hoàn thành. Ví dụ:

```
Future<String> fetchUserOrder() {  
  return Future.delayed(Duration(seconds: 2), () => 'Cà phê');  
}  
  
void main() {  
  print('Đang chờ đơn hàng...');  
  fetchUserOrder().then((order) {  
    print('Đơn hàng của bạn: $order');  
  });  
}  
  
// Kết quả:  
// Đang chờ đơn hàng...  
// (sau 2 giây)  
// Đơn hàng của bạn: Cà phê
```

Giải thích:

- **fetchUserOrder** là một hàm trả về **Future<String>**, mô phỏng việc gọi API để lấy thông tin đơn hàng.
- **Future.delayed** giả lập một khoảng thời gian chờ (2 giây) trước khi trả về giá trị 'Cà phê'.
- then được sử dụng để xử lý giá trị trả về của **Future** khi nó hoàn thành.

Stream đại diện cho một chuỗi các giá trị không đồng bộ. Nó hữu ích khi ta cần xử lý một luồng dữ liệu liên tục như dữ liệu từ một kết nối mạng, các sự kiện người dùng hoặc dữ liệu cảm biến. Ví dụ:

```
Stream<int> countStream(int max) async* {
  for (int i = 1; i <= max; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i;
  }
}

void main() {
  print('Bắt đầu đếm:');
  countStream(5).listen((count) {
    print(count);
  });
}

// Kết quả:
// Bắt đầu đếm:
// 1
// (sau 1 giây)
// 2
// (sau 1 giây)
// 3
// (sau 1 giây)
// 4
// (sau 1 giây)
// 5
```

Giải thích:

- **countStream** là một hàm trả về **Stream<int>** sử dụng **async*** và **yield** để tạo ra các giá trị không đồng bộ.

- `await Future.delayed(Duration(seconds: 1))` mô phỏng việc chờ đợi 1 giây trước khi phát ra một giá trị mới.
- `listen` được sử dụng để lắng nghe và xử lý các giá trị từ `Stream`.

2.10.3 Async và Await

`async` và `await` giúp ta viết mã không đồng bộ trông giống như mã đồng bộ, làm cho nó dễ đọc và dễ bảo trì hơn. Ta sử dụng từ khóa `async` để khai báo một hàm không đồng bộ và từ khóa `await` để chờ đợi một `Future` hoàn thành.

```
Future<String> fetchUserOrder() async {
  return Future.delayed(Duration(seconds: 2), () => 'Cà phê');
}

void main() async {
  print('Đang chờ đơn hàng...');
  String order = await fetchUserOrder();
  print('Đơn hàng của bạn: $order');
}

// Kết quả:
// Đang chờ đơn hàng...
// (sau 2 giây)
// Đơn hàng của bạn: Cà phê
```

Giải thích:

- `fetchUserOrder` được khai báo với từ khóa `async` để trở thành một hàm không đồng bộ.
- `await` được sử dụng trong hàm `main` để chờ `Future` từ `fetchUserOrder` hoàn thành và trả về giá trị.

2.10.4 Một số thao tác thường gặp

- **Xử lý lỗi**

Ta có thể sử dụng `try-catch` để bắt lỗi trong các hàm bất đồng bộ:

```
Future<void> fetchUserOrder() async {
```

```
try {
  String order = await Future.delayed(Duration(seconds: 2), () => throw
Exception('Lỗi'));
  print('Đơn hàng của bạn: $order');
} catch (e) {
  print('Đã xảy ra lỗi: $e');
}
}

void main() {
  fetchUserOrder();
}

// Kết quả:
// Đã xảy ra lỗi: Exception: Lỗi
```

- **Quản lý nhiều Future**

Để chờ nhiều Future hoàn thành, ta có thể sử dụng `Future.wait`:

```
Future<void> fetchData() async {
  var futures = <Future>[
    Future.delayed(Duration(seconds: 1), () => print('Hoàn thành tác vụ 1')),
    Future.delayed(Duration(seconds: 2), () => print('Hoàn thành tác vụ 2')),
    Future.delayed(Duration(seconds: 3), () => print('Hoàn thành tác vụ 3')),
  ];

  await Future.wait(futures);
  print('Tất cả các tác vụ đã hoàn thành');
}

void main() {
  fetchData();
}

// Kết quả:
// Hoàn thành tác vụ 1
// (sau 1 giây)
// Hoàn thành tác vụ 2
// (sau 1 giây)
// Hoàn thành tác vụ 3
// Tất cả các tác vụ đã hoàn thành
```

- **Timeout**

Để thêm giới hạn thời gian cho một hoạt động bất đồng bộ, ta có thể sử dụng `timeout`:

```
Future<void> fetchDataWithTimeout() async {
  try {
    await Future.delayed(Duration(seconds: 5)).timeout(Duration(seconds: 2));
    print('Hoàn thành');
  } catch (e) {
    print('Đã quá thời gian chờ: $e');
  }
}

void main() {
  fetchDataWithTimeout();
}

// Kết quả:
// Đã quá thời gian chờ: TimeoutException after 0:00:02.000000: Future not
completed
```

TÓM TẮT

Trong bài học này, chúng ta sẽ bắt đầu với cơ bản về Dart, khám phá cú pháp và các kiểu dữ liệu cơ bản mà ngôn ngữ này cung cấp. Bài học sẽ tiếp tục với các cấu trúc điều khiển, từ đơn giản đến phức tạp và cách chúng được sử dụng để xây dựng logic trong chương trình. Chúng ta cũng sẽ xem xét cách Dart hỗ trợ lập trình hướng đối tượng, một yếu tố thiết yếu trong việc xây dựng các ứng dụng có cấu trúc và dễ bảo trì. Ngoài ra, bài học sẽ đề cập đến xử lý ngoại lệ và lập trình bất đồng bộ, hai khái niệm quan trọng giúp xử lý các tình huống phức tạp trong thực tế phát triển phần mềm.

BÀI TẬP

Câu 1: Viết một đoạn mã Dart để khai báo các kiểu dữ liệu khác nhau như số nguyên, số thực, chuỗi ký tự và Boolean. Sau đó, in giá trị của các biến này ra console để xem kết quả.

Câu 2: Viết một hàm trong Dart tên là `calculateSum` nhận vào hai tham số số nguyên và trả về tổng của chúng. Gọi hàm này trong chương trình chính và in kết quả ra màn hình.

Câu 3: Tạo một lớp `Car` với các thuộc tính là `make`, `model` và `year`. Thêm một phương thức tên là `displayInfo` để in thông tin của xe ra console. Trong chương trình chính, tạo một đối tượng của lớp `Car` và gọi phương thức `displayInfo`.

Câu 4: Viết một đoạn mã Dart để thực hiện phép chia hai số được nhập từ người dùng. Sử dụng khối `try-catch` để xử lý trường hợp phép chia cho số không và in thông báo lỗi phù hợp.

Câu 5: Viết một hàm bất đồng bộ trong Dart sử dụng `async` và `await` để truy xuất dữ liệu từ một API giả lập (ví dụ: `JSON placeholder`). Hàm này phải xử lý dữ liệu trả về và in ra màn hình.

Câu 6: Thảo luận về tầm quan trọng của Generics trong Dart và viết một ví dụ đơn giản sử dụng Generics để xử lý dữ liệu kiểu `List`.

BÀI 3: FLUTTER VÀ CÁC WIDGET CƠ BẢN

Trong bài học này, người học sẽ được trang bị kiến thức cơ bản về Flutter, một framework mạnh mẽ của Google cho phát triển ứng dụng di động đa nền tảng. Chúng ta sẽ khám phá các Widget cơ bản, cách quản lý trạng thái trong Flutter và các công cụ phát triển cần thiết để thiết kế giao diện người dùng linh hoạt và đáp ứng. Sau khi hoàn thành bài học này, người học cần nắm được:

- Cách thiết lập và sử dụng môi trường phát triển Flutter trên các hệ điều hành khác nhau.
- Hiểu biết về các Widget cơ bản và layout trong Flutter.
- Kỹ năng sử dụng *Stateful* và *Stateless Widgets* để quản lý trạng thái.
- Cách tích hợp và sử dụng các *Assets* và *Media* vào dự án Flutter.
- Khả năng tạo và điều hướng giữa các màn hình khác nhau trong một ứng dụng.

3.1 TỔNG QUAN VỀ FLUTTER

3.1.1 Giới thiệu

Flutter là một bộ công cụ giao diện người dùng (UI toolkit) mã nguồn mở do Google phát triển. Nó giúp ta xây dựng các ứng dụng cho nhiều nền tảng như iOS, Android, web và desktop từ cùng một mã nguồn duy nhất. Flutter sử dụng ngôn ngữ lập trình Dart và cung cấp một thư viện phong phú để tạo ra các ứng dụng có giao diện đẹp và hiệu suất cao.

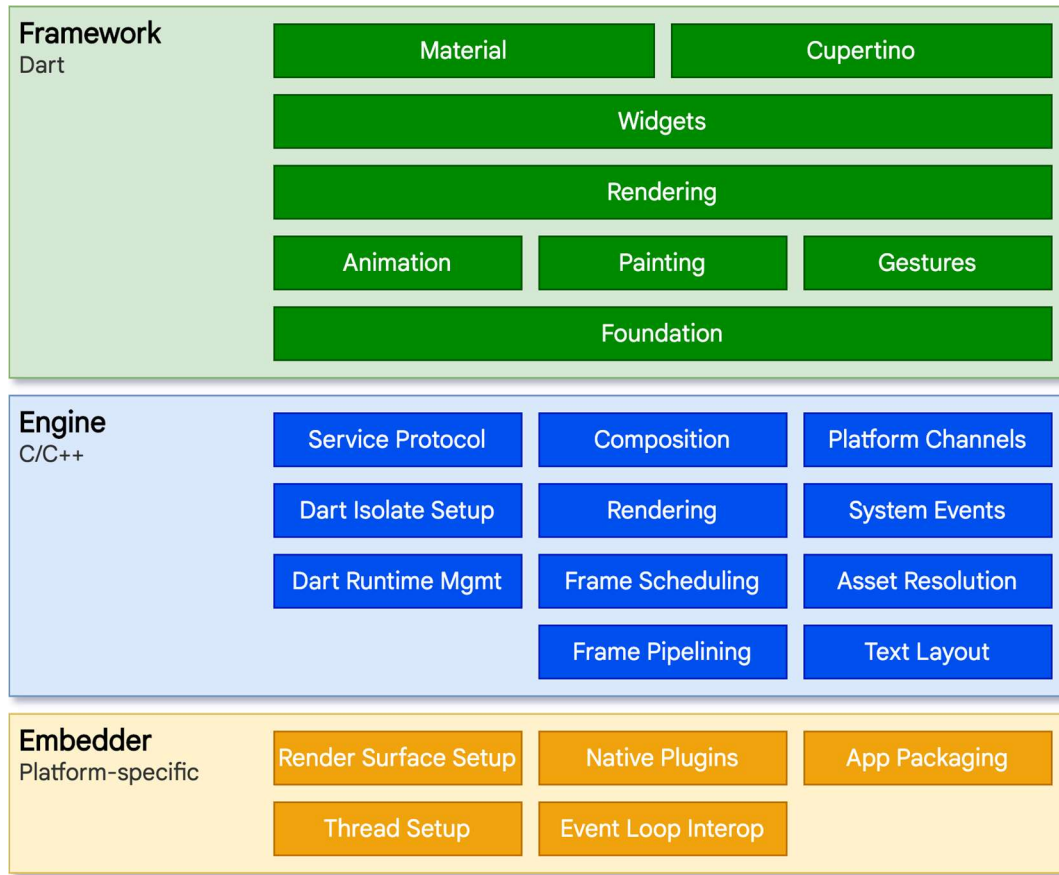
Điểm mạnh của Flutter là khả năng hiển thị giao diện người dùng theo thời gian thực (hot reload), cho phép chúng ta xem ngay lập tức các thay đổi trong mã nguồn mà không cần biên dịch lại toàn bộ ứng dụng. Khi phát hành, ứng dụng Flutter được

biên dịch trực tiếp thành mã máy, tối ưu hóa cho cả x64 và ARM hoặc JavaScript khi nhắm mục tiêu phát triển web.

3.1.2 Kiến trúc ứng dụng

Kiến trúc của Flutter được xây dựng theo mô hình phân lớp, gồm các thành phần chính:

- **Engine:** Phần cốt lõi viết bằng C++, chịu trách nhiệm raster hóa (rasterizing) các cảnh ghép lại và cung cấp các API cơ bản như đồ họa, bố cục văn bản, I/O file và mạng, hỗ trợ truy cập và runtime Dart.
- **Framework:** Cung cấp các thư viện cấp cao để xây dựng giao diện người dùng theo mô hình lập trình phản ứng (reactive programming). Framework này bao gồm các lớp cơ bản như:
 - Lớp nền tảng (Foundation Layer): Cung cấp các dịch vụ cơ bản như hoạt hình, vẽ và xử lý cử chỉ.
 - Lớp kết xuất (Rendering Layer): Quản lý layout và xây dựng cây các đối tượng có thể render.
 - Lớp widget (Widgets Layer): Đơn vị cơ bản của giao diện người dùng, giúp tạo ra các thành phần giao diện phức tạp từ các widget đơn giản.
 - Thư viện Material và Cupertino: Cung cấp các bộ điều khiển giao diện theo thiết kế của Google và Apple.
- **Embedder:** Phần mã gốc trên mỗi nền tảng (như Java và C++ cho Android, Objective-C/Objective-C++ cho iOS và macOS và C++ cho Windows và Linux) cho phép ứng dụng Flutter giao tiếp với hệ điều hành và quản lý vòng đời sự kiện.



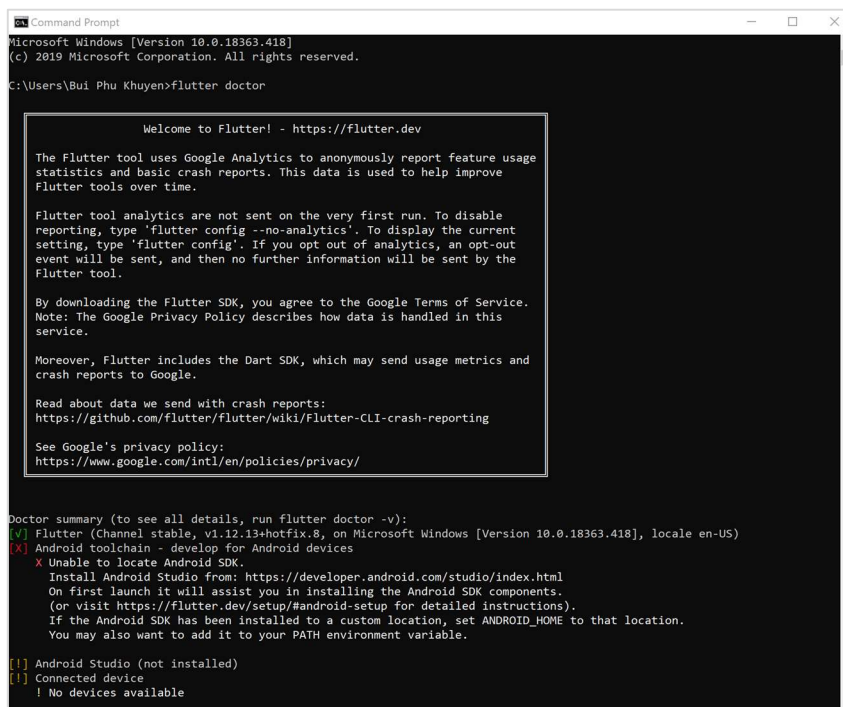
Kiến trúc ứng dụng Flutter

3.1.3 Thiết lập môi trường phát triển

3.1.3.1 Chuẩn bị môi trường

Để thiết lập môi trường phát triển cho Flutter, chúng ta cần thực hiện các bước cơ bản sau:

- **Cài đặt Flutter SDK**
 - Tải Flutter SDK từ trang chủ Flutter tại (<https://docs.flutter.dev/get-started/install>) .
 - Giải nén file tải về và đặt đường dẫn Flutter vào biến môi trường PATH.
- **Cài đặt công cụ dòng lệnh:** Chạy lệnh `flutter doctor` trong terminal để kiểm tra và cài đặt các công cụ cần thiết.



```
Microsoft Windows [Version 10.0.18363.418]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\Bui Phu Khuyen>flutter doctor

Welcome to Flutter! - https://flutter.dev

The Flutter tool uses Google Analytics to anonymously report feature usage
statistics and basic crash reports. This data is used to help improve
Flutter tools over time.

Flutter tool analytics are not sent on the very first run. To disable
reporting, type 'flutter config --no-analytics'. To display the current
setting, type 'flutter config'. If you opt out of analytics, an opt-out
event will be sent, and then no further information will be sent by the
Flutter tool.

By downloading the Flutter SDK, you agree to the Google Terms of Service.
Note: The Google Privacy Policy describes how data is handled in this
service.

Moreover, Flutter includes the Dart SDK, which may send usage metrics and
crash reports to Google.

Read about data we send with crash reports:
https://github.com/flutter/flutter/wiki/Flutter-CLI-crash-reporting

See Google's privacy policy:
https://www.google.com/intl/en/policies/privacy/

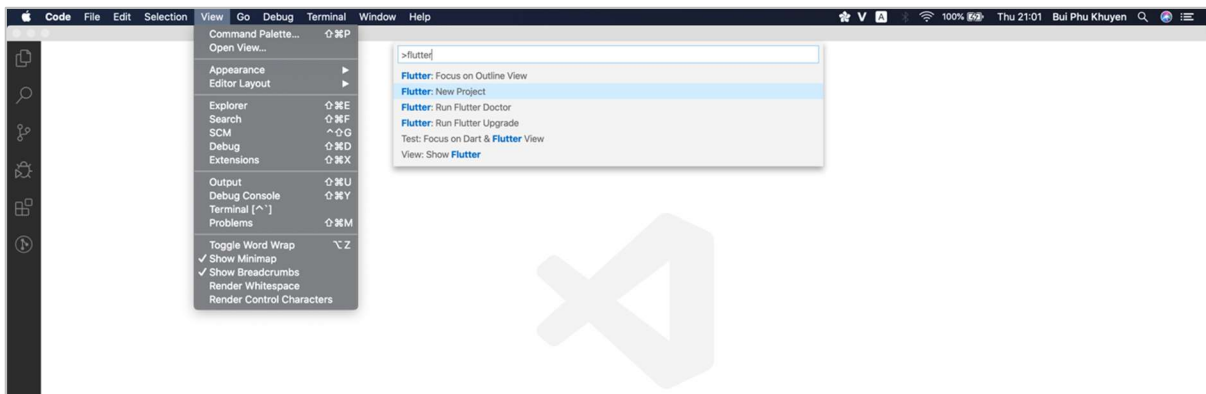
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, v1.12.13+hotfix.8, on Microsoft Windows [Version 10.0.18363.418], locale en-US)
[X] Android toolchain - develop for Android devices
    X Unable to locate Android SDK.
       Install Android Studio from: https://developer.android.com/studio/index.html
       On first launch it will assist you in installing the Android SDK components.
       (or visit https://flutter.dev/setup/#android-setup for detailed instructions).
       If the Android SDK has been installed to a custom location, set ANDROID_HOME to that location.
       You may also want to add it to your PATH environment variable.
[!] Android Studio (not installed)
[!] Connected device
    ! No devices available
```

- Trong trường hợp môi trường hệ thống yêu cầu bổ sung các ứng dụng, câu lệnh cần thiết (dấu ×). Ta phải hoàn thành theo yêu cầu và đưa nó về trạng thái hoàn thành (dấu ✓).
- Những trường hợp cảnh báo xấu Warning (dấu !). Có thể không ảnh hưởng. Tuy nhiên để đảm bảo môi trường phát triển tốt, ta nên hoàn thành luôn những cảnh báo này.
- **Cài đặt IDE**
 - Sử dụng Visual Studio Code, Android Studio hoặc IntelliJ IDEA.
 - Cài đặt plugin Flutter và Dart trong IDE đã lựa chọn ở trên.
- **Cài đặt thiết bị ảo**
 - Cài đặt Android Emulator hoặc sử dụng thiết bị iOS Simulator nếu phát triển cho iOS.
 - Cấu hình thiết bị ảo từ Android Studio hoặc Xcode.

3.1.3.2 Chạy ứng dụng đầu tiên sử dụng VSCode

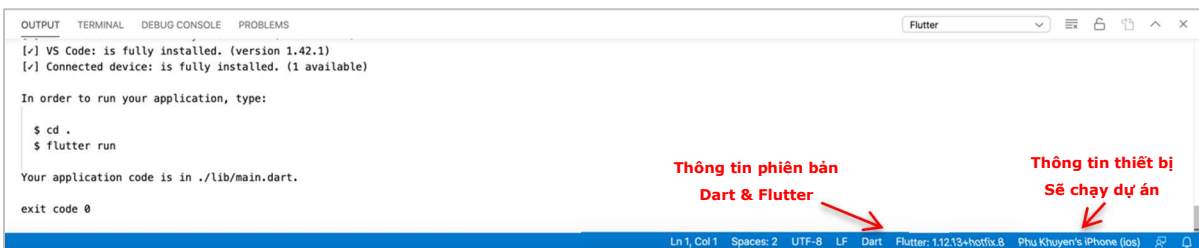
- **Tạo mới dự án**

- Chọn mục **View > Command Palette**. Nhập "flutter" và chọn **Flutter: New Project**.
- Chọn mẫu nền tảng phát triển (**Application**)
- Dẫn đến ổ đĩa trên máy để lưu trữ dự án.
- Đặt tên của dự án (ví dụ **myflutterapp**).
- Đợi IDE tạo mới dự án và File **main.dart** sẽ xuất hiện.



• Chạy dự án vừa tạo:

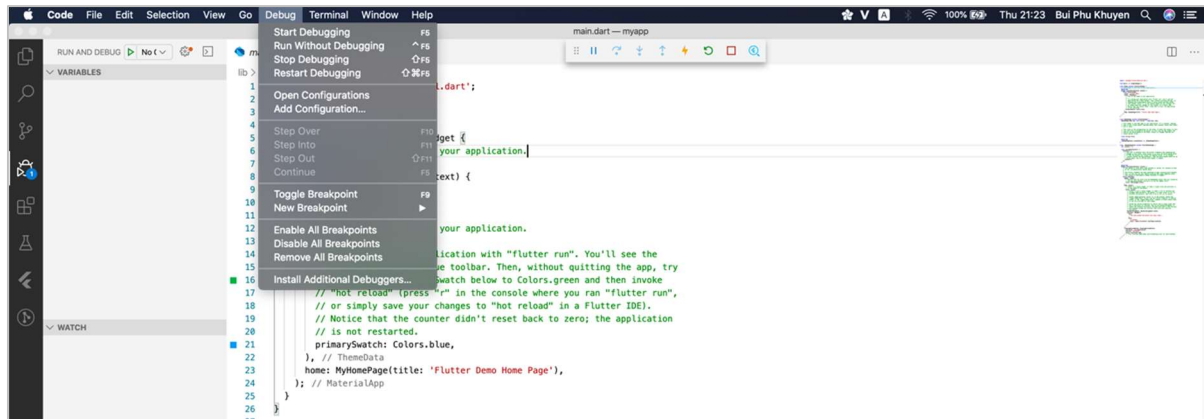
Để Run được dự án, ta phải kiểm tra thanh trạng thái (màu xanh) trên VSCode, bao gồm những thông tin sau:



Lưu ý:

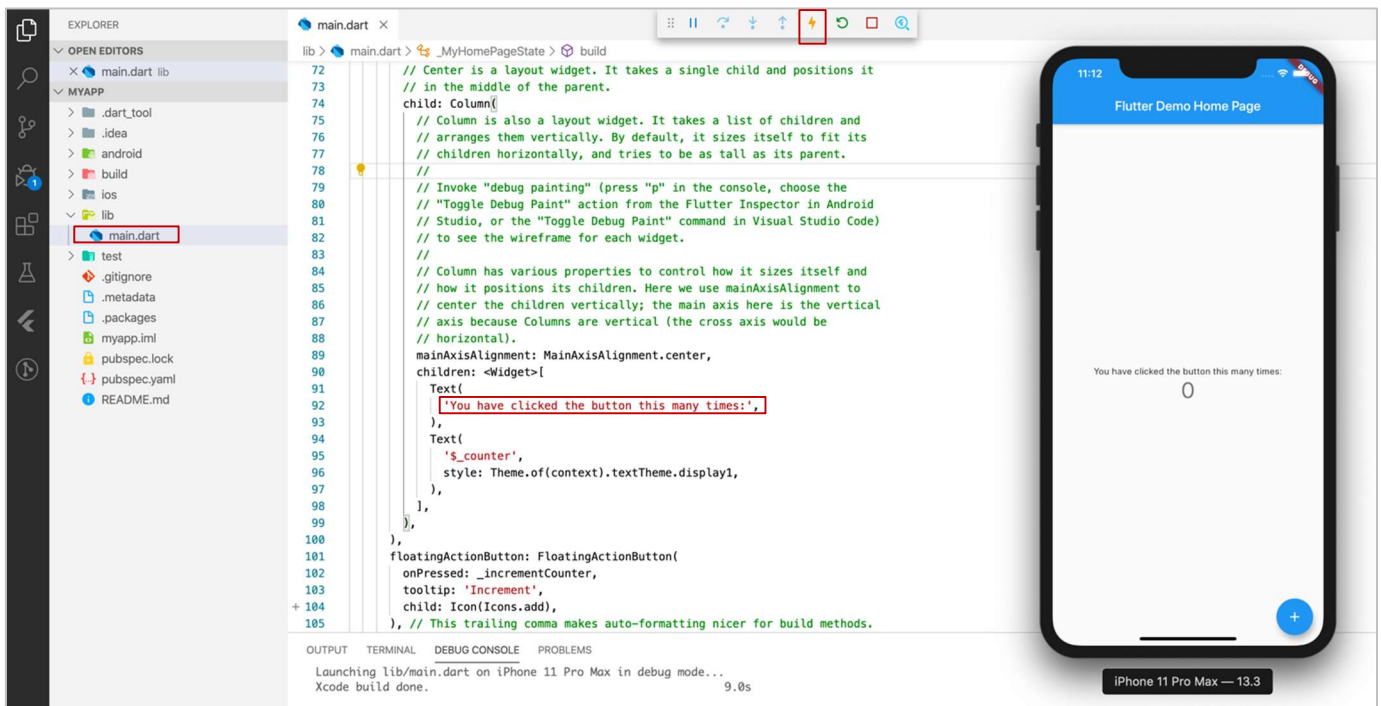
- Ta có thể chuyển đổi Phiên bản, Kênh Flutter (nếu tải nhiều kênh) và Thiết bị kết nối (nếu kết nối nhiều thiết bị).
- Trong trường hợp VSCode không phát hiện thiết bị sẽ hiển thị là **"No Devices"**. Ta có thể ấn vào mục này và tiến hành thiết lập để chạy thiết bị thật hoặc máy ảo.

- Chọn **Debug > Start Debugging** (hoặc nhấn phím **F5**) để tiến hành chạy.



- Quá trình build hoàn tất, ta có kết quả là 1 ứng dụng Flutter Demo đơn giản.

• Tính năng Hot Reload



Flutter được hỗ trợ tính năng Hot Reload. Điều đó có nghĩa là nếu ta thực hiện bất kỳ thay đổi nào trong mã thì ta có thể thấy chúng ngay lập tức mà không cần chờ đợi hoặc khởi động lại ứng dụng. Thông thường, phải mất vài giây để kiểm tra các thay đổi. Điều này giúp các nhà thiết kế thêm các tính năng mới, sửa lỗi và thử nghiệm ứng dụng hiệu quả hơn.

3.1.4 Cấu trúc dự án Flutter

Khi tạo một dự án Flutter mới, ta sẽ thấy một cấu trúc thư mục và file như sau:

```

> .dart_tool
> .idea
> android
> build
> ios
> lib
> linux
> macos
> test
> web
> windows
.gitignore
.metadata
analysis_options.yaml
flutter_application_demo.iml
pubspec.lock
pubspec.yaml
README.md

```

Bên dưới giải thích chức năng của cấu trúc tập thư mục và file thường dùng.

Thư mục/File	Mô tả
pubspec.yaml	File cấu hình chính, chứa thông tin về dự án, dependencies và asset.
pubspec.lock	File tự động tạo khi chạy flutter pub get, chứa phiên bản dependencies.
lib/	Thư mục chứa mã nguồn chính của ứng dụng, gồm file main.dart.
android/ ios/ web/ windows/ linux/ macos/	Mã nguồn và cấu hình cho ứng dụng trên nền tảng Android, iOS, Web, Windows, Linux và MacOS
build/	Thư mục tạo tự động khi build, chứa file build trung gian và kết quả.
.dart_tool/	Thư mục ẩn, chứa các file và thông tin nội bộ mà Dart sử dụng để quản lý các công cụ và dependencies.
test/	Chứa các file kiểm thử (test) cho ứng dụng Flutter.

3.2 WIDGET VÀ LAYOUT

3.2.1 Giới thiệu Widget

Widgets là các thành phần cơ bản để xây dựng giao diện người dùng trong Flutter. Mọi thứ trong Flutter đều là một widget, từ các nút bấm, văn bản đến các bố cục phức tạp hơn.

Widgets có thể lồng nhau để tạo ra giao diện người dùng phong phú và đa dạng.

3.2.2 Phân loại Widgets

Trong Flutter, tất cả các widget được phân loại dựa trên chức năng thành bốn nhóm chính:

- **Các widget giao diện đặc thù theo từng nền tảng (Platform-specific widgets)**
 - Material Widgets: Thiết kế theo Material Design của Google, dành cho Android. Ví dụ: Scaffold, AppBar, TextField, Checkbox, Radio, Switch, Slider, AlertDialog, BottomNavigationBar, FloatingActionButton.
 - Cupertino Widgets: Thiết kế theo Human Interface Guidelines của Apple, dành cho iOS. Ví dụ: CupertinoButton, CupertinoNavigationBar, CupertinoTabBar, CupertinoTextField, CupertinoDialog, CupertinoSlider.

- **Các widget hỗ trợ bố trí giao diện (Layout widgets)**

Dùng để sắp xếp và định vị các widget con bên trong chúng, tạo ra các bố cục giao diện phong phú và linh hoạt. Ví dụ: Container, Center, Row, Column, Stack.

- **Các widget quản lý trạng thái (State maintenance widgets)**

Trong Flutter, tất cả các widget đều kế thừa từ StatelessWidget hoặc StatefulWidget.

- StatelessWidget: Không có trạng thái, phù hợp cho các thành phần giao diện tĩnh.

- StatefulWidget: Có thể thay đổi trạng thái, phù hợp cho các thành phần giao diện động.
- **Các widget cơ bản độc lập với nền tảng (Platform-independent/basic widgets)**

Các widget cơ bản để tạo giao diện người dùng từ đơn giản đến phức tạp, độc lập với nền tảng hệ điều hành. Ví dụ: Text, Icon, Image, ElevatedButton, ListView.

3.2.3 Layout Widgets

Widgets là các lớp dùng để xây dựng giao diện người dùng (UI) trong Flutter. Chúng không chỉ được sử dụng để tạo các thành phần giao diện mà còn để bố trí các thành phần này. Bằng cách kết hợp các widget đơn giản, ta có thể xây dựng nên các widget phức tạp hơn.

Ta có thể tạo bố cục layout bằng cách kết hợp các widget để xây dựng những widget phức tạp hơn. Ví dụ, hình ảnh dưới đây cho thấy ba biểu tượng với một nhãn dưới mỗi biểu tượng:



Hình ảnh tiếp theo hiển thị bố cục trực quan và chi tiết hơn, cho thấy 1 hàng (row) gồm 3 cột (columns) mà mỗi cột chứa một biểu tượng và một nhãn.



3.3 STATELESS VÀ STATEFULL WIDGETS

3.3.1 Stateless Widgets

StatelessWidget là các widget tĩnh không thể thay đổi nội dung hiển thị sau khi được render. Đây là các widget đơn giản với cấu trúc rõ ràng, dễ sử dụng.

Một StatelessWidget yêu cầu một hàm `build(BuildContext context)` để render dữ liệu lên màn hình.

```
class MyStatelessWidget extends StatelessWidget {  
  final String title;  
  
  MyStatelessWidget({required this.title});  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(title);  
  }  
}
```

Trong ví dụ trên, hàm build chỉ được gọi một lần khi widget được tạo và không thay đổi trong suốt quá trình ứng dụng chạy. Dữ liệu hiển thị có thể được truyền qua constructor và sẽ không thay đổi.

Khi nào nên sử dụng StatelessWidget?

- Hiển thị dữ liệu cố định, chẳng hạn như tiêu đề, nút hoặc biểu tượng.
- Khi ta muốn tạo các thành phần giao diện không cần thay đổi sau khi đã được render.
- Khi sử dụng trong StatefulWidget để render lại các item con khi trạng thái của StatefulWidget thay đổi.

3.3.2 Statefull Widgets

StatefulWidget là các widget động có thể thay đổi nội dung hiển thị bằng cách thay đổi trạng thái của chính nó. StatefulWidget cần một hàm createState() để cung cấp trạng thái (State) cho nó.

Ví dụ về StatefulWidget:

```
class MyStatefulWidget extends StatefulWidget {  
  @override  
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();  
}  
  
class _MyStatefulWidgetState extends State<MyStatefulWidget> {  
  String title = 'Hello, Flutter!';  
}
```



```
void _updateTitle() {
  setState(() {
    title = 'Title Updated!';
  });
}

@override
Widget build(BuildContext context) {
  return Column(
    children: <Widget>[
      Text(title),
      ElevatedButton(
        onPressed: _updateTitle,
        child: Text('Update Title'),
      ),
    ],
  );
}
```

Trong ví dụ trên, `StatefulWidget` quản lý trạng thái thông qua lớp `_MyStatefulWidgetState`. Hàm `setState()` được sử dụng để thay đổi trạng thái và cập nhật lại giao diện khi cần.

Khi nào nên sử dụng `StatefulWidget`?

- Khi cần hiển thị dữ liệu có thể thay đổi, chẳng hạn như trạng thái của một form, danh sách động hoặc nội dung thay đổi theo hành vi của người dùng.
- Khi cần quản lý các trạng thái tạm thời, chẳng hạn như bộ đếm, thanh tiến trình hoặc các tương tác người dùng khác.
- Khi cần hiển thị nội dung có thể thay đổi dựa trên sự kiện, như nhập liệu, chọn mục từ danh sách hoặc nhận dữ liệu từ một API.

Lưu ý: Mặc dù `StatefulWidget` linh hoạt hơn trong việc quản lý trạng thái, việc sử dụng chúng cũng có thể tốn tài nguyên hơn. `StatelessWidget` nhẹ hơn và nhanh hơn trong việc render UI. Do đó, ta nên chọn loại widget phù hợp với yêu cầu cụ thể để tối ưu hóa hiệu năng của ứng dụng.

3.3.3 Quản lý trạng thái

Quản lý trạng thái là một khía cạnh quan trọng trong việc phát triển ứng dụng Flutter, giúp duy trì và cập nhật giao diện người dùng theo sự thay đổi của dữ liệu. Có hai loại quản lý trạng thái chính: quản lý trạng thái cục bộ (local state management) và quản lý trạng thái toàn cục (global state management).

3.3.3.1 Local state management

Quản lý trạng thái cục bộ liên quan đến việc quản lý trạng thái của các widget cụ thể và chỉ ảnh hưởng đến một phần nhỏ của ứng dụng. Đây là cách tiếp cận phù hợp khi trạng thái chỉ được sử dụng trong phạm vi một widget hoặc một nhóm nhỏ các widget liên kết chặt chẽ.

Giả sử ta có một widget hiển thị một bộ đếm số lần nhấp vào nút bấm. Trạng thái của bộ đếm này chỉ cần được quản lý trong phạm vi của widget đó.

```
import 'package:flutter/material.dart';

class CounterWidget extends StatefulWidget {
  @override
  _CounterWidgetState createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Local State Management'),
      ),
      body: Center(
```

```
child: Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: <Widget>[  
    Text(  
      'You have pressed the button this many times:',  
    ),  
    Text(  
      '$_counter',  
      style: Theme.of(context).textTheme.headline4,  
    ),  
  ],  
,  
,  
floatingActionButton: FloatingActionButton(  
  onPressed: _incrementCounter,  
  tooltip: 'Increment',  
  child: Icon(Icons.add),  
,  
,  
);  
}  
}  
  
void main() => runApp(MaterialApp(  
  home: CounterWidget(),  
));
```

Trong ví dụ trên, `_counter` là trạng thái cục bộ của `CounterWidget`, được quản lý bởi State của widget này. Mỗi khi nút bấm được nhấn, hàm `setState()` sẽ được gọi để cập nhật trạng thái và render lại giao diện.

3.3.3.2 Global state management

Quản lý trạng thái toàn cục liên quan đến việc quản lý trạng thái được chia sẻ giữa nhiều widget hoặc toàn bộ ứng dụng. Điều này thường cần thiết khi nhiều phần của ứng dụng cần truy cập và cập nhật cùng một trạng thái.

Ví dụ, ta sử dụng thư viện `Provider` để quản lý trạng thái toàn cục. Giả sử ta có một ứng dụng đơn giản với hai màn hình, cả hai đều cần truy cập và cập nhật cùng một trạng thái.

```
import 'package:flutter/material.dart';
```

```
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => Counter(),
      child: MyApp(),
    ),
  );
}

class Counter with ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();
  }
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: FirstScreen(),
    );
  }
}

class FirstScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('First Screen'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text('Count: ${Provider.of<Counter>(context).count}'),
            ElevatedButton(
              onPressed: () {
                Provider.of<Counter>(context, listen: false).increment();
              },
              child: Text('Increment'),
            ),
          ],
        ),
      ),
    );
  }
}
```

```

ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => SecondScreen()),
    );
  },
  child: Text('Go to Second Screen'),
),
],
),
);
}
}

```

```

class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Second Screen'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text('Count: ${Provider.of<Counter>(context).count}'),
            ElevatedButton(
              onPressed: () {
                Provider.of<Counter>(context, listen: false).increment();
              },
              child: Text('Increment'),
            ),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context);
              },
              child: Text('Back to First Screen'),
            ),
          ],
        ),
      ),
    );
  }
}

```

Trong ví dụ trên, Counter là một lớp ChangeNotifier chứa trạng thái toàn cục. Chúng ta sử dụng ChangeNotifierProvider để cung cấp trạng thái này cho toàn bộ ứng

dụng. Bất kỳ widget nào trong cây widget có thể truy cập và cập nhật trạng thái thông qua `Provider.of<Counter>(context)`.

3.4 CÁC WIDGET CƠ BẢN

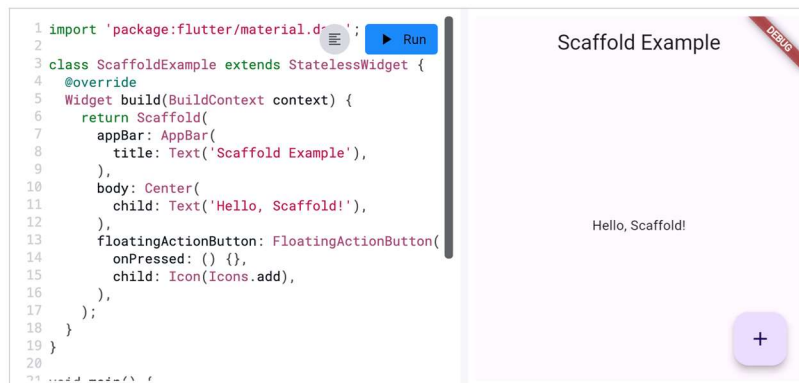
3.4.1 Scaffold

Scaffold là widget cơ bản trong Material Design cung cấp cấu trúc giao diện cho các trang màn hình ứng dụng.

```
Scaffold({  
  Key? key,  
  this.appBar,  
  this.body,  
  this.floatingActionButton,  
  // Các thuộc tính khác  
})
```

Một số thuộc tính hay dùng:

- **appBar**: Thanh tiêu đề của ứng dụng.
- **body**: Nội dung chính của trang.
- **floatingActionButton**: Nút bấm nổi.



3.4.2 Text

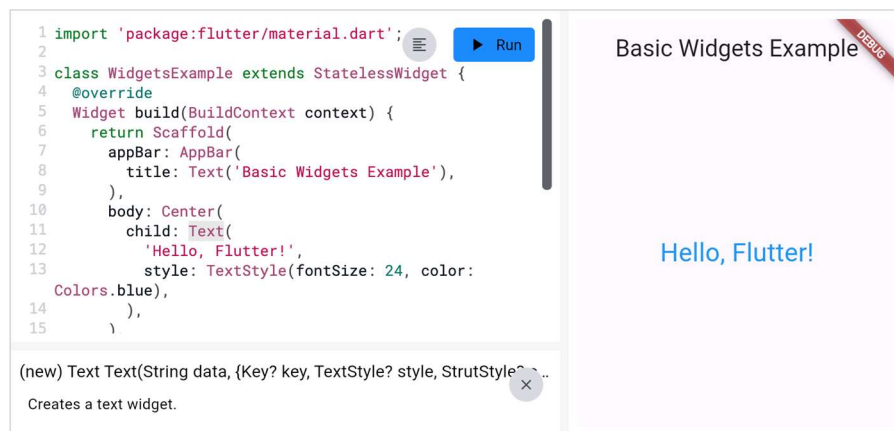
Text hiển thị một đoạn văn bản đơn giản.

```
Text(  
  String data, {
```

```
Key? key,  
this.style,  
this.textAlign,  
this.overflow,  
// Các thuộc tính khác  
})
```

Một số thuộc tính hay dùng:

- **style**: Định dạng văn bản (màu sắc, kích thước, font chữ).
- **textAlign**: Căn chỉnh văn bản (trái, phải, giữa).
- **overflow**: Kiểm soát cách xử lý văn bản khi quá dài (cắt bớt,...).



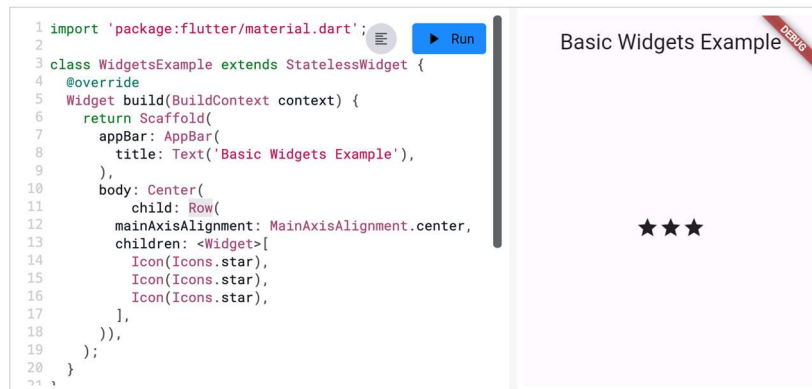
3.4.3 Row

Row sắp xếp các widget con theo chiều ngang.

```
Row({  
  Key? key,  
  this.mainAxisAlignment = MainAxisAlignment.start,  
  this.crossAxisAlignment = CrossAxisAlignment.center,  
  this.children = const <Widget>[],  
  // Các thuộc tính khác  
})
```

Một số thuộc tính hay dùng:

- **mainAxisAlignment**: Căn chỉnh các widget con theo chiều ngang.
- **crossAxisAlignment**: Căn chỉnh các widget con theo chiều dọc.



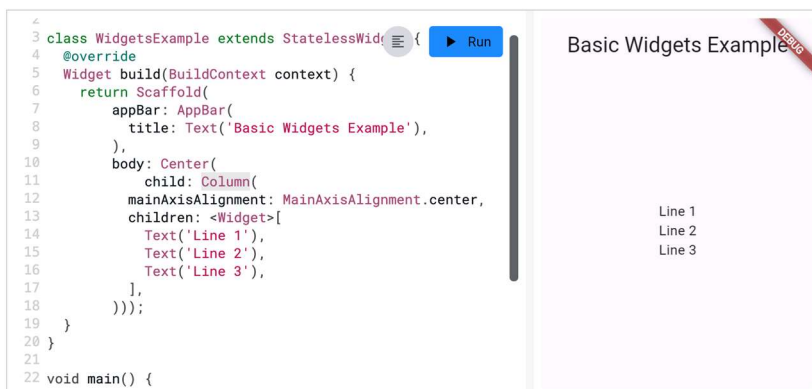
3.4.4 Column

Column sắp xếp các widget con theo chiều dọc.

```
Column({
  Key? key,
  this.mainAxisAlignment = MainAxisAlignment.start,
  this.crossAxisAlignment = CrossAxisAlignment.center,
  this.children = const <Widget>[],
  // Các thuộc tính khác
})
```

Một số thuộc tính hay dùng:

- **mainAxisAlignment**: Căn chỉnh các widget con theo chiều dọc.
- **crossAxisAlignment**: Căn chỉnh các widget con theo chiều ngang.



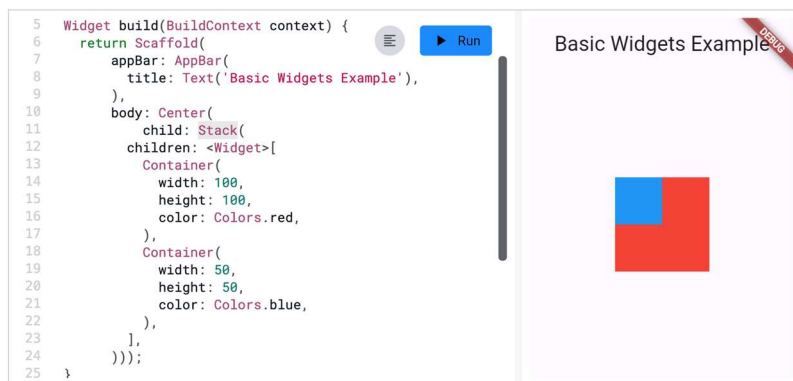
3.4.5 Stack

Stack xếp chồng các widget con lên nhau.


```
Stack({
  Key? key,
  this.alignment = AlignmentDirectional.topStart,
  this.fit = StackFit.loose,
  this.children = const <Widget>[],
  // Các thuộc tính khác
})
```

Một số thuộc tính hay dùng:

- **alignment**: Căn chỉnh các widget con trong stack.
- **fit**: Xác định kích thước của stack (loose hoặc expand).



3.4.6 Container

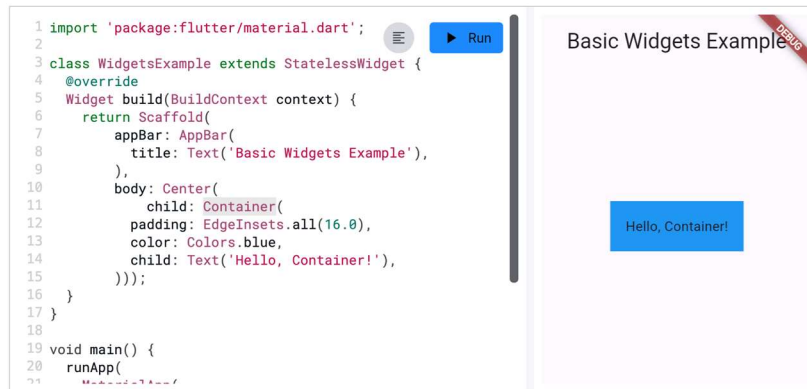
Container là một widget linh hoạt để chứa các widget con với khả năng áp dụng các thuộc tính như padding, margin, border và màu nền.

```
Container({
  Key? key,
  this.alignment,
  this.padding,
  this.color,
  this.decoration,
  this.child,
  // Các thuộc tính khác
})
```

Một số thuộc tính hay dùng:

- **alignment**: Căn chỉnh nội dung bên trong container.
- **padding**: Khoảng cách bên trong container.

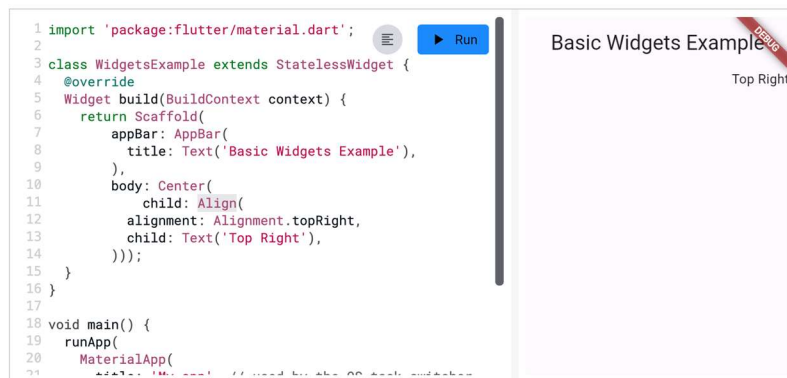
- **margin**: Khoảng cách bên ngoài container.
- **color**: Màu nền của container.
- **decoration**: Trang trí container (màu sắc, viền, bóng đổ).



3.4.7 Align

Align căn chỉnh widget con theo một vị trí cụ thể.

```
Align({
  Key? key,
  this.alignment = Alignment.center,
  this.child,
  // Các thuộc tính khác
})
```



3.4.8 Padding

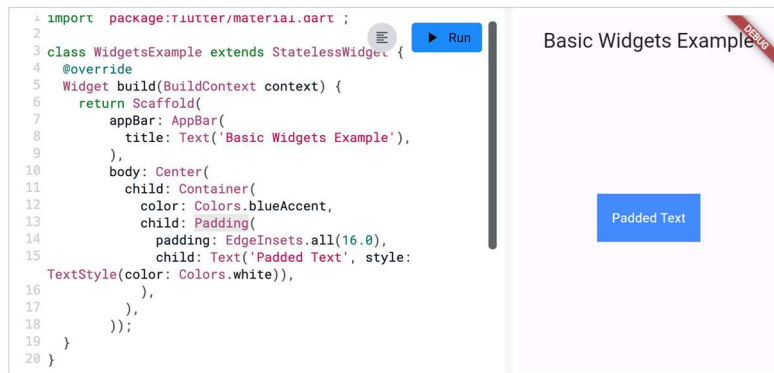
Padding thêm khoảng cách xung quanh widget con.

```
Padding({
```

```

Key? key,
this.padding,
this.child,
})

```



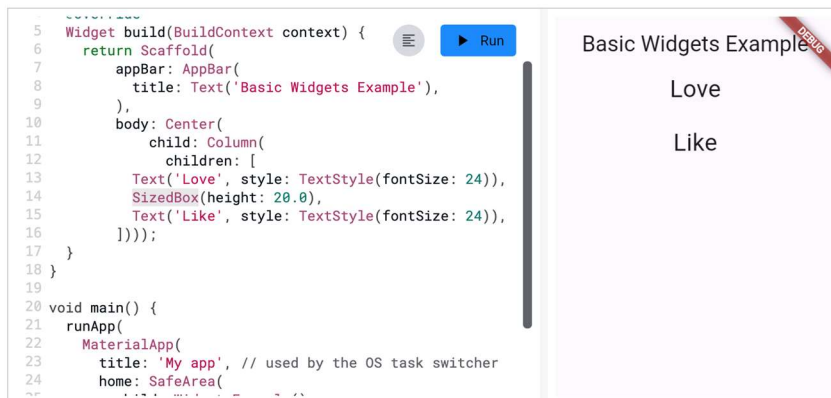
3.4.9 SizedBox

SizedBox đặt kích thước cụ thể cho widget con hoặc tạo khoảng trống giữa các widget.

```

SizedBox({
  Key? key,
  this.width,
  this.height,
  this.child,
})

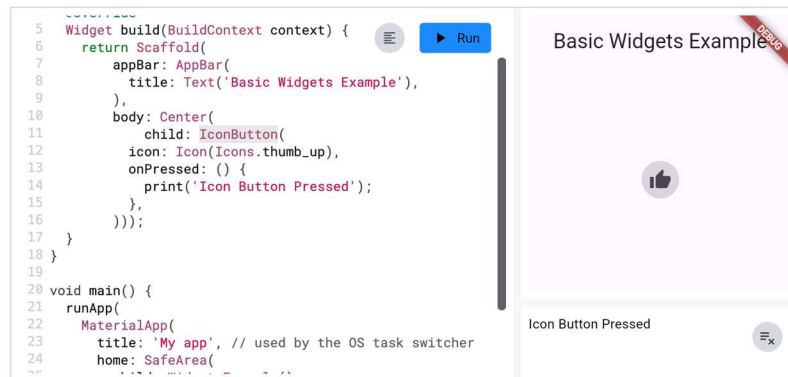
```



3.4.10 IconButton

IconButton là nút bấm hiển thị biểu tượng, có thể bấm được.

```
IconButton({  
  Key? key,  
  this.icon,  
  this.onPressed,  
  // Các thuộc tính khác  
})
```



3.5 ASSET VÀ MEDIA

3.5.1 Xử lý Assets và Images

Trong Flutter, **assets** là các tập tin được đóng gói và triển khai cùng ứng dụng, có thể truy cập tại runtime. Các loại **assets** phổ biến bao gồm dữ liệu tĩnh (ví dụ: tệp **JSON**), tệp cấu hình, biểu tượng và hình ảnh (**JPEG**, **PNG**, **GIF**, **WebP**, **BMP**).

Đầu tiên ta cần sử dụng tệp **pubspec.yaml** để khai báo các **assets** cần thiết.

```
flutter:  
  assets:  
    - assets/my_icon.png  
    - assets/background.jpeg  
    - assets/data/config.json
```

Để tải và hiển thị **assets** trong dự án Flutter, ta có thể áp dụng các cách sau:

- Sử dụng **AssetImage** để tải hình ảnh trong phương thức **build()** của widget.

```
return Image(image: AssetImage('assets/background.png'));
```

- Sử dụng **rootBundle** từ **package:flutter/services.dart** để tải các tệp văn bản hoặc **JSON**.

```
import 'package:flutter/services.dart' show rootBundle;

Future<String> loadAsset() async {
  return await rootBundle.loadString('assets/data/config.json');
}
```

Ngoài ra, ta có thể tải hình ảnh từ Internet bằng cách sử dụng **Image.network**:

```
Image.network('https://picsum.photos/250?image=9')
```

3.5.2 Xử lý Video

Để phát video trong Flutter, ta có thể sử dụng plugin **video_player**. Plugin này hỗ trợ phát video từ hệ thống tệp, dưới dạng asset hoặc từ internet.

Nguyên lý hoạt động:

- Thêm phụ thuộc **video_player**: Thêm thư viện **video_player** vào dự án của mình thông qua tệp **pubspec.yaml**.
- Cấp quyền truy cập: Cập nhật quyền truy cập internet trong tệp cấu hình của Android (**AndroidManifest.xml**) và iOS (**Info.plist**).
- Khởi tạo **VideoPlayerController**: Tạo và khởi tạo **VideoPlayerController** để quản lý video.
- Hiển thị video: Sử dụng widget **VideoPlayer** để hiển thị video trong ứng dụng.
- Điều khiển phát video: Sử dụng các phương thức như **play()**, **pause()** của **VideoPlayerController** để điều khiển việc phát video.

Lưu ý: Chi tiết cài đặt và hướng dẫn sử dụng có thể tham khảo tại trang chủ của thư viện **video_player** tại https://pub.dev/packages/video_player.

3.6 NAVIGATING VÀ ROUTING

Trong Flutter, điều hướng (**navigating**) và định tuyến (**routing**) là những khái niệm cơ bản nhưng rất quan trọng để quản lý và chuyển đổi giữa các màn hình trong ứng dụng của chúng ta. Hệ thống điều hướng của Flutter cho phép chúng ta xây dựng các

ứng dụng mượt mà và trực quan, từ các ứng dụng nhỏ với điều hướng đơn giản đến các ứng dụng phức tạp với liên kết sâu (**deep links**).

3.6.1 Sử dụng Navigator

Navigator là một widget quản lý các màn hình như một ngăn xếp. Mỗi khi chúng ta chuyển sang một màn hình mới, nó sẽ được đẩy vào ngăn xếp và khi quay lại, màn hình sẽ được bật ra khỏi ngăn xếp. Để điều hướng đến một màn hình mới, chúng ta sử dụng phương thức **push()** và để quay lại, chúng ta sử dụng **pop()**.

```
onPressed: () {  
  Navigator.of(context).push(  
    MaterialPageRoute(  
      builder: (context) => const SongScreen(song: song),  
    ),  
  );  
},  
child: Text(song.name),
```

Ở ví dụ này, khi người dùng nhấn nút, ta sử dụng **Navigator.of(context).push()** để chuyển đến màn hình **SongScreen**. Trong đó:

- **Navigator** giữ một ngăn xếp các đối tượng **Route**.
- Phương thức **push()** thêm một **Route** mới vào ngăn xếp.
- **MaterialPageRoute** là một loại **Route** đặc biệt cung cấp các hoạt ảnh chuyển tiếp theo phong cách **Material Design**.

3.6.2 Sử dụng Named Routes

Trong các ứng dụng đơn giản, chúng ta có thể sử dụng **named routes** để định nghĩa các đường dẫn cố định giữa các màn hình. **Named routes** cho phép chúng ta định nghĩa tất cả các tuyến đường trong ứng dụng tại một chỗ duy nhất, giúp mã nguồn trở nên gọn gàng và dễ quản lý hơn. Ví dụ:

```
@override  
Widget build(BuildContext context) {  
  return MaterialApp(  
    routes: {  
      '/': (context) => HomeScreen(),
```

```
'/details': (context) => DetailScreen(),  
},  
);  
}
```

Ở ví dụ này, chúng ta định nghĩa hai tuyến đường: / cho màn hình chính **HomeScreen** và **/details** cho màn hình chi tiết **DetailScreen**.

Tuy nhiên sẽ có một vài hạn chế khi sử dụng **Named Routes**:

- Không thể tùy chỉnh hành vi của liên kết sâu.
- Không hỗ trợ nút **forward** trên trình duyệt.
- Do đó, đối với các ứng dụng phức tạp, chúng ta nên xem xét sử dụng các công cụ khác như **Router**.

3.6.3 Sử dụng Router

Đối với các ứng dụng yêu cầu điều hướng phức tạp và liên kết sâu chi tiết, chúng ta nên sử dụng **Router**. **Router** cung cấp khả năng quản lý các đường dẫn và trạng thái của ứng dụng một cách linh hoạt và mạnh mẽ hơn. Ví dụ với **GoRouter**:

```
MaterialApp.router(  
  routerConfig: GoRouter(  
    // Cấu hình Router  
  )  
);
```

Sử dụng các gói định tuyến như **go_router** cho phép chúng ta xử lý liên kết sâu một cách chính xác và đồng bộ hóa với thanh địa chỉ khi chạy trên web.

Lưu ý:

- **Router** và **Navigator** có thể được sử dụng cùng nhau.
- Khi sử dụng **Router**, các route được tạo từ **Page** sẽ luôn có thể liên kết sâu, trong khi các **route** không có trang thì không.

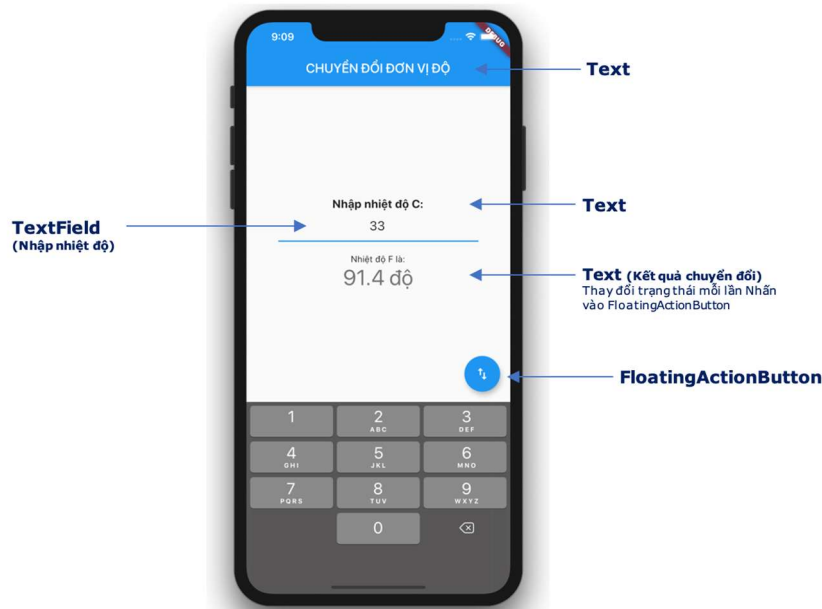
TÓM TẮT

Bài học này cung cấp một cái nhìn tổng quan về nền tảng Flutter và các công cụ liên quan để xây dựng ứng dụng di động hiệu quả. Người học sẽ tìm hiểu về kiến trúc ứng dụng và cách thiết lập môi trường phát triển trên các hệ điều hành khác nhau. Tiếp theo, chúng ta sẽ đi sâu vào việc sử dụng các Widget cơ bản như Text, Row, Column, Stack và Scaffold để xây dựng giao diện người dùng. Bên cạnh đó là cách tích hợp Assets và Media vào dự án Flutter. Bài học cũng sẽ bao gồm các phương pháp quản lý trạng thái trong ứng dụng và cách thực hiện navigation và routing để tạo các ứng dụng có nhiều màn hình.

BÀI TẬP

Câu 1: Hãy cài đặt Flutter trên hệ điều hành của bản thân và viết báo cáo ngắn mô tả quá trình cài đặt và thiết lập. Sau đó, tạo một ứng dụng Flutter đơn giản hiển thị "Hello, Flutter" trên màn hình.

Câu 2: Dựa trên dự án Demo mẫu của Flutter, hãy xử lý lại main.dart với chức năng chuyển đổi đơn vị độ (từ độ C sang độ F) theo yêu cầu như hình.



BÀI 4: THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG NÂNG CAO

Bài học này sẽ trang bị cho người học các kỹ thuật nâng cao trong việc thiết kế và phát triển giao diện người dùng sử dụng Flutter. Người học sẽ khám phá các widget phức tạp hơn, cách thiết kế tương tác và áp dụng các hiệu ứng hình ảnh trong ứng dụng. Sau khi hoàn thành bài học này, người học cần nắm được:

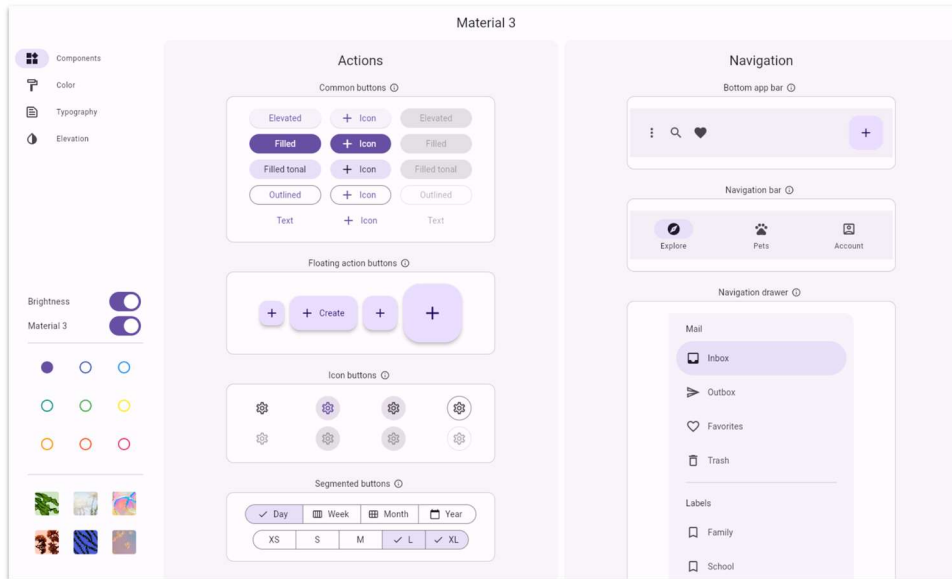
- *Cách phân biệt Material Design và Cupertino Design.*
- *Các kỹ thuật điều khiển Layout và Navigation thông qua BottomNavigationBar, ListView, GridView và SingleChildScrollView.*
- *Kỹ năng sử dụng các widget tương tác như Buttons, Gestures và Forms.*
- *Khả năng sử dụng các phản hồi trong ứng dụng thông qua Visual Feedback.*
- *Biết các chức năng liên quan đến trợ năng và hỗ trợ đa ngôn ngữ.*
- *Hiểu biết sâu về cách tạo và kiểm soát hiệu ứng animations và transitions*

4.1 DESIGN & THEMING

4.1.1 Material Design

Material Design là một hệ thống thiết kế mã nguồn mở được xây dựng và hỗ trợ bởi các nhà thiết kế và nhà phát triển của Google.

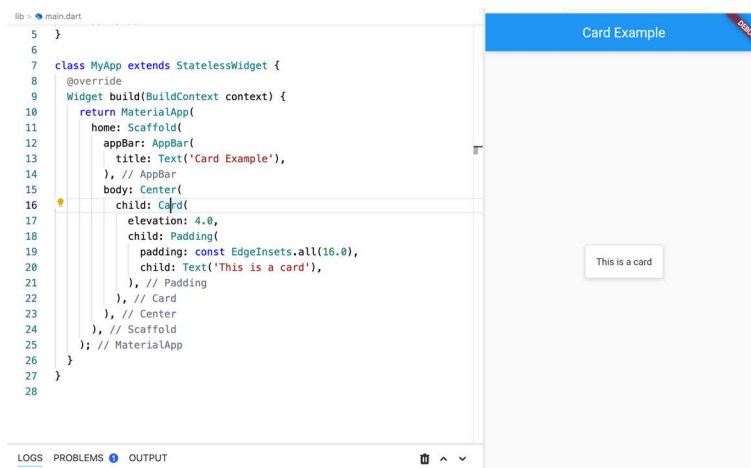
Phiên bản mới nhất (Material 3) cho phép trải nghiệm cá nhân hóa được tối ưu nhất. Có thể tìm hiểu thêm tại: <https://m3.material.io/develop/flutter>



Material 3

4.1.1.1 Card

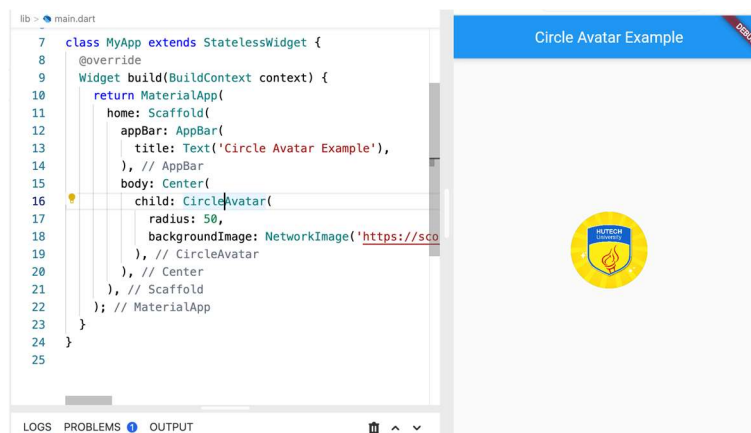
Card là một widget Material Design dùng để hiển thị nội dung và hành động trong một khung có thể tùy chỉnh.



Trong đó **elevation** là độ bóng của thẻ Card.

4.1.1.2 CircleAvatar

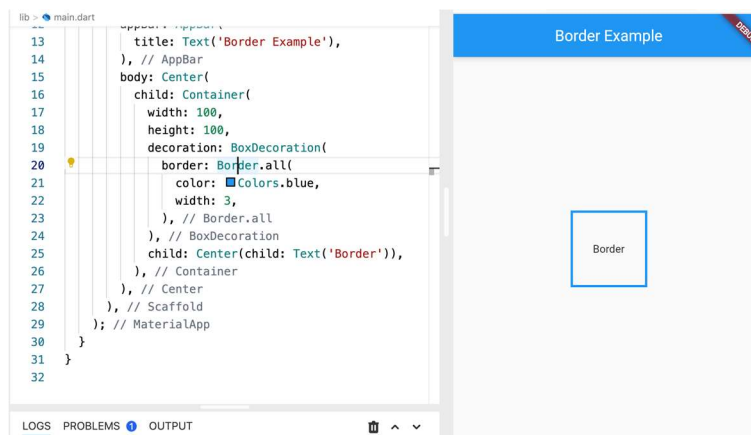
CircleAvatar là một widget để hiển thị hình ảnh hoặc chữ cái trong một vòng tròn, thường được sử dụng cho ảnh đại diện.



Trong đó có thuộc tính **radius** thể hiện bán kính của vòng tròn và **backgroundImage** là nơi chứa ảnh nền của **avatar**.

4.1.1.3 Border

Border là một thuộc tính dùng để tạo viền cho các widget.

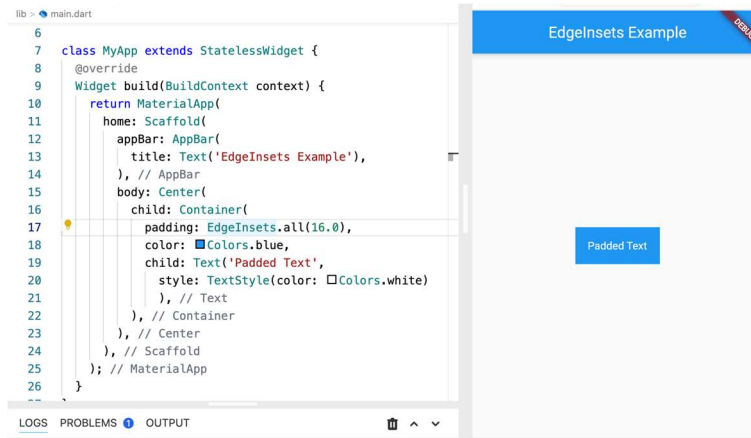


Ở ví dụ trên:

- **Container**: Widget chứa nội dung và trang trí.
- **decoration**: Thuộc tính **BoxDecoration** để trang trí **container**.
- **border**: Tạo viền cho **container** với màu sắc viền (**color**) và độ dày viền (**width**).

4.1.1.4 EdgeInsets

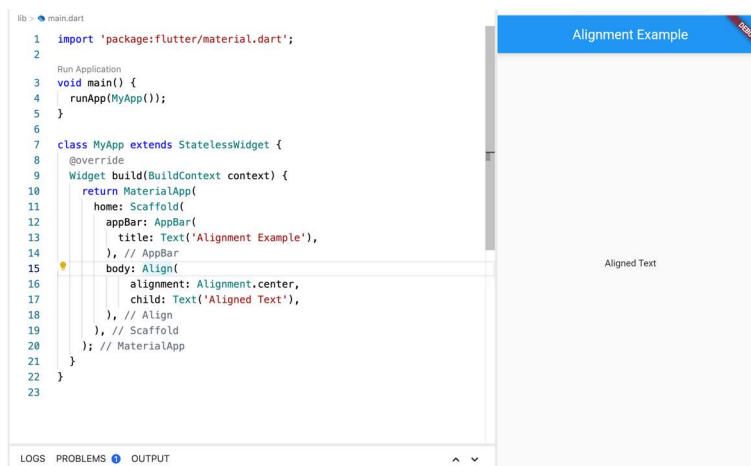
EdgeInsets là một lớp dùng để xác định khoảng cách đệm (**padding**) và khoảng cách viền (**margin**) trong Flutter.



Trong đó: Thuộc tính **padding** để khoảng cách đệm bên trong widget **Container** và **EdgeInsets.all()** để tạo khoảng cách đệm đều nhau ở tất cả các cạnh.

4.1.1.5 Alignment

Alignment là một thuộc tính để căn chỉnh các widget con bên trong một widget cha.

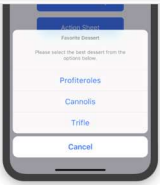
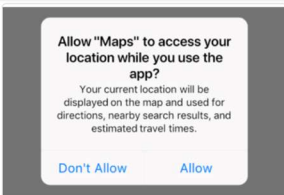
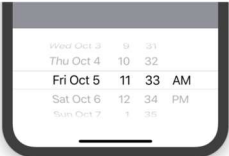


Trong đó **alignment** là thuộc tính của **Alignment** để xác định vị trí căn chỉnh. Bên cạnh đó **Alignment.center** dùng để căn chỉnh widget con `Text` ở giữa.

4.1.2 Cupertino Design

Cupertino Design là hệ thống thiết kế do Apple phát triển, được sử dụng trên các nền tảng iOS và macOS. Flutter cung cấp một bộ widget phong phú để tạo giao diện người dùng theo phong cách Cupertino, giúp ứng dụng của ta có vẻ ngoài và cảm giác tự nhiên trên các thiết bị của Apple.

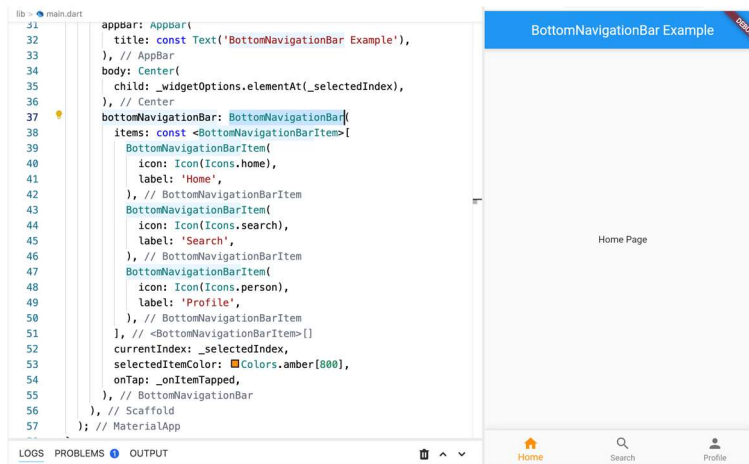
Cupertino Design tập trung vào việc tạo ra các trải nghiệm người dùng mượt mà, trực quan và dễ sử dụng, với các thành phần thiết kế quen thuộc như thanh điều hướng, nút bấm, các thành phần nhập liệu và nhiều hơn nữa.

 CupertinoActionSheet An iOS-style modal bottom action sheet to choose an option among many.	 CupertinoActivityIndicator An iOS-style activity indicator. Displays a circular 'spinner'.	 CupertinoAlertDialog An iOS-style alert dialog.
 CupertinoButton An iOS-style button.	 CupertinoContextMenu An iOS-style full-screen modal route that opens when the child is long-pressed. Used to display relevant actions for your content.	 CupertinoDatePicker An iOS-style date or date and time picker.
 CupertinoSwitch An iOS-style switch. Used to toggle the on/off state of a single setting.	 CupertinoTabBar An iOS-style bottom tab bar. Typically used with CupertinoTabScaffold.	 CupertinoTabScaffold Tabbed iOS app structure. Positions a tab bar on top of tabs of content.
 CupertinoTabView Root content of a tab that supports parallel navigation between tabs. Typically used with CupertinoTabScaffold.	 CupertinoTextField An iOS-style text field.	 CupertinoTimerPicker An iOS-style countdown timer picker.

4.2 NAVIGATION & LAYOUT CONTROL

4.2.1 BottomNavigationBar

BottomNavigationBar là một widget phổ biến trong các ứng dụng di động, giúp người dùng điều hướng giữa các màn hình khác nhau bằng cách sử dụng một thanh điều hướng ở cuối màn hình. Nó đặc biệt hữu ích trong các ứng dụng có cấu trúc phẳng (**flat structure**) với một số lượng màn hình giới hạn.

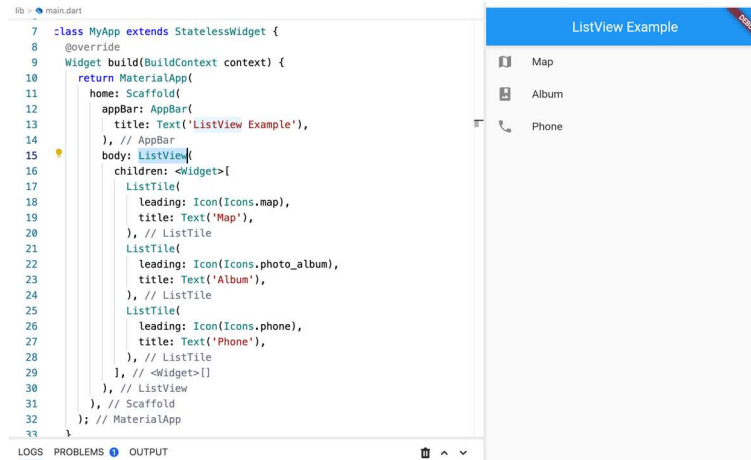


Trong ví dụ này, **BottomNavigationBar** chứa ba mục: **Home**, **Search** và **Profile**. Khi người dùng nhấn vào một mục, trạng thái của ứng dụng sẽ thay đổi và hiển thị màn hình tương ứng. Cụ thể hơn:

- **bottomNavigationBar** là một thuộc tính của **Scaffold** để xác định thanh điều hướng dưới cùng.
- **items** là danh sách các mục trong **BottomNavigationBar**, mỗi mục là một **BottomNavigationBarItem**.
- **icon**: Biểu tượng của mục.
- **label**: Nhãn của mục.
- **currentIndex**: Chỉ mục của mục đang được chọn.
- **selectedItemColor**: Màu của mục được chọn.
- **onTap**: Hàm gọi lại khi một mục được nhấn.

4.2.2 ListView

ListView là một widget mạnh mẽ giúp hiển thị danh sách các mục theo chiều dọc. Nó tự động cuộn khi danh sách dài hơn chiều cao của màn hình.

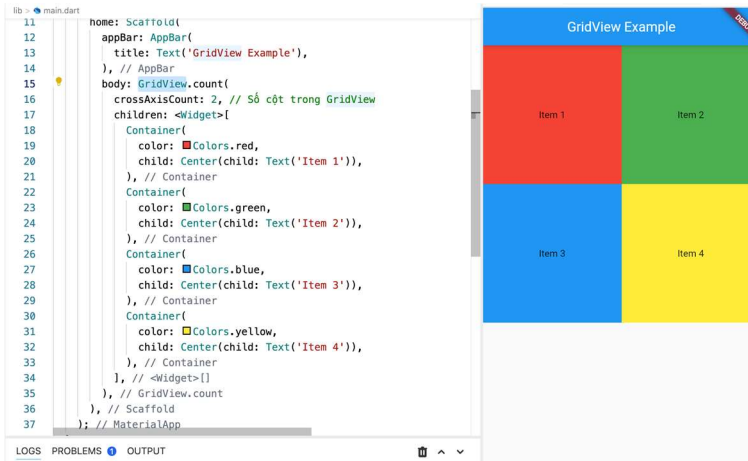


Trong ví dụ trên:

- **ListView** chứa các mục con, mỗi mục có thể là bất kỳ widget nào (như **ListTile**).
- **ListTile** có thể có **leading** (biểu tượng ở bên trái), **title** (văn bản chính) và **subtitle** (văn bản phụ).

4.2.3 GridView

GridView tương tự như **ListView** nhưng hiển thị các mục theo dạng lưới (**grid**). Đây là một cách tuyệt vời để hiển thị các mục với bố cục lưới, đặc biệt là trong các ứng dụng hiển thị hình ảnh hoặc sản phẩm.

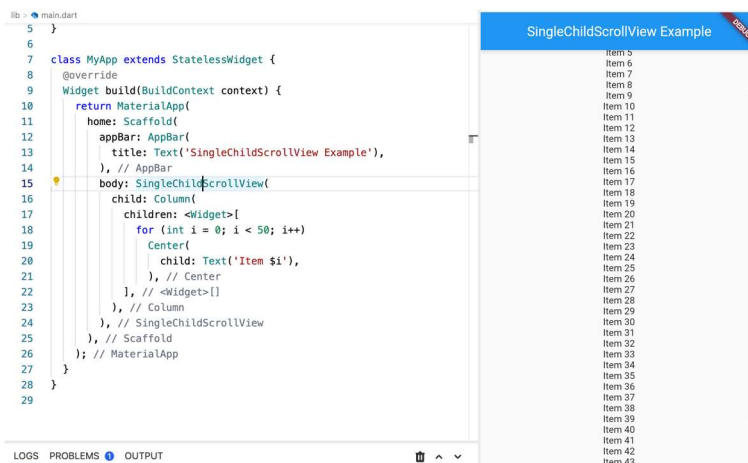


Trong đó:

- **GridView.count** tạo một lưới với số cột xác định bởi **crossAxisCount**.
- **children** chứa các **widget** con sẽ được hiển thị trong lưới.
- **crossAxisCount**: Số cột trong lưới.

4.2.4 SingleChildScrollView

SingleChildScrollView là một widget hỗ trợ cuộn nội dung khi nội dung vượt quá kích thước của màn hình. Nó rất hữu ích khi ta có một danh sách dài các mục mà không thể hiển thị hết trên một màn hình duy nhất.



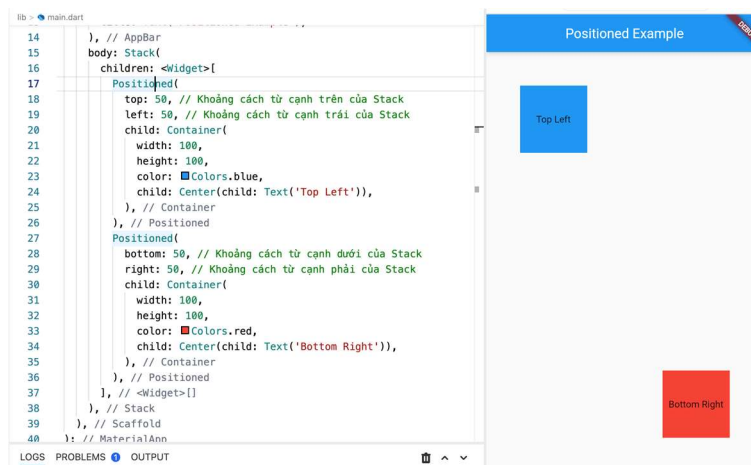
Ở ví dụ trên:

- **SingleChildScrollView** cho phép cuộn nội dung bên trong nó.

- **Column** chứa các mục con và được bao bọc bởi **SingleChildScrollView** để có thể cuộn.
- **Center** và **Text** hiển thị các nội dung chữ nằm ở giữa.

4.2.5 Positioned

Positioned là một widget được sử dụng bên trong **Stack** để định vị các widget con của nó tương đối so với các cạnh của stack. Đây là một widget hữu ích khi ta cần định vị chính xác các widget con.

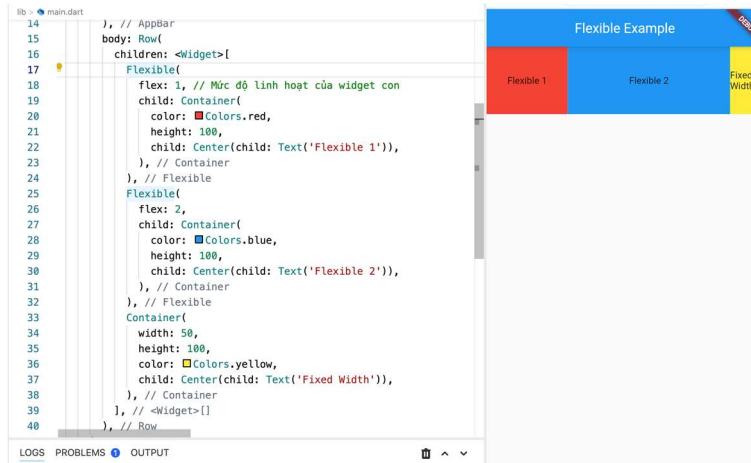


Giải thích ví dụ trên:

- **Positioned** sử dụng bên trong **Stack** để định vị widget con.
- Thuộc tính **top**, **left**, **right**, **bottom** xác định vị trí của widget con tương đối so với các cạnh của **Stack**.
- **child**: Widget con của **Positioned**.

4.2.6 Flexible

Flexible là một widget tương tự như **Expanded** nhưng cho phép ta kiểm soát mức độ linh hoạt của widget con. Đây là một widget hữu ích khi ta cần một widget con chiếm không gian nhưng không nhất thiết phải chiếm tất cả không gian còn lại.

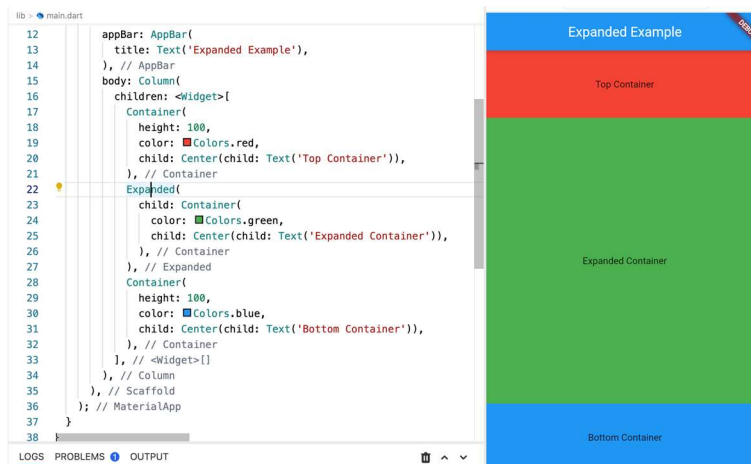


Giải thích:

- **Flexible** cho phép widget con chiếm không gian nhưng không bắt buộc phải chiếm tất cả không gian còn lại.
- **flex**: Mức độ linh hoạt của widget con.
- **child**: Widget con của Flexible.

4.2.7 Expanded

Expanded là một widget giúp mở rộng một widget con trong một **Row**, **Column** hoặc **Flex** để chiếm không gian còn lại. Điều này hữu ích khi ta muốn một widget con chiếm không gian nhiều nhất có thể trong một hướng cụ thể.



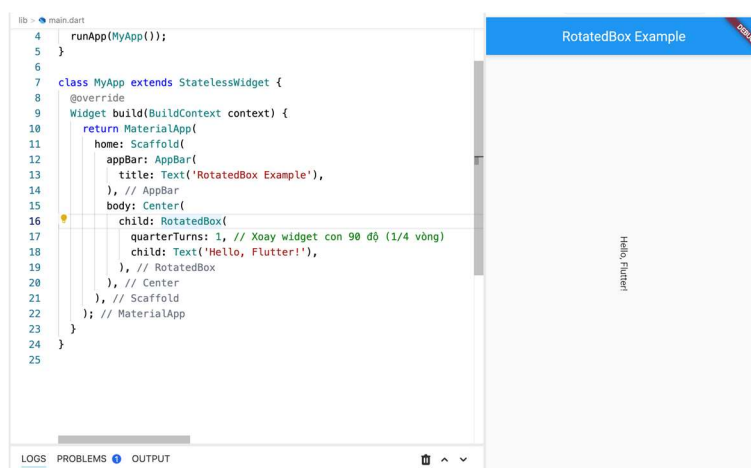
Ở ví dụ trên:

- **Expanded** mở rộng widget con để chiếm không gian còn lại trong **Column**.

- **Column** chứa 3 **Container**:
 - o **Container** đầu tiên và cuối cùng có chiều cao cố định.
 - o **Expanded** giúp **Container** ở giữa chiếm không gian còn lại.

4.2.8 Rotated Box

RotatedBox là một widget xoay một widget con theo bội số của 90 độ. Đây là một widget hữu ích khi ta cần xoay một widget con một cách dễ dàng.



Trong đó thuộc tính **quarterTurns** thể hiện số lần xoay 90 độ.

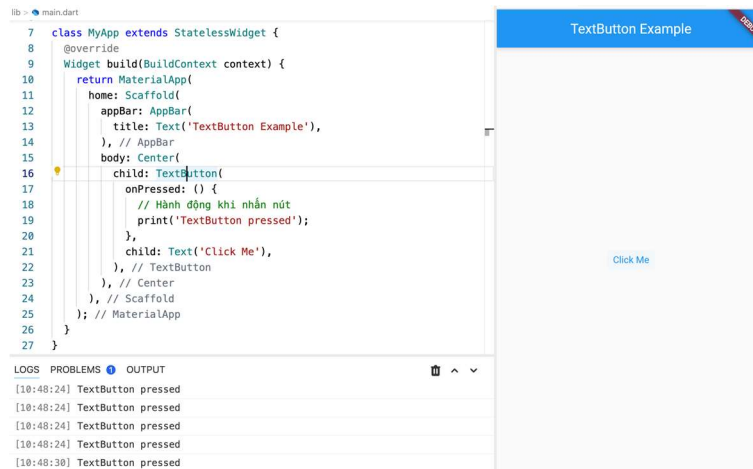
4.3 INTERACTIVITY

Trong Flutter, tính tương tác (interactivity) là một phần quan trọng để xây dựng các ứng dụng có khả năng phản hồi và tương tác với người dùng.

4.3.1 Buttons

4.3.1.1 Text Button

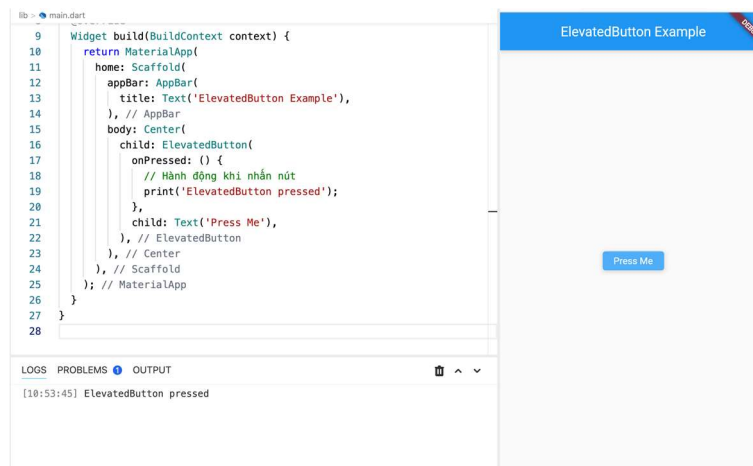
TextButton là một loại nút bấm đơn giản, thường được sử dụng cho các hành động không quá quan trọng.



Trong ví dụ trên `TextButton` có thuộc tính `onPressed` là hàm xử lý các lệnh khi nút được nhấn; thuộc tính `child` là widget con (thường là `Text` thể hiện nội dung của nút)

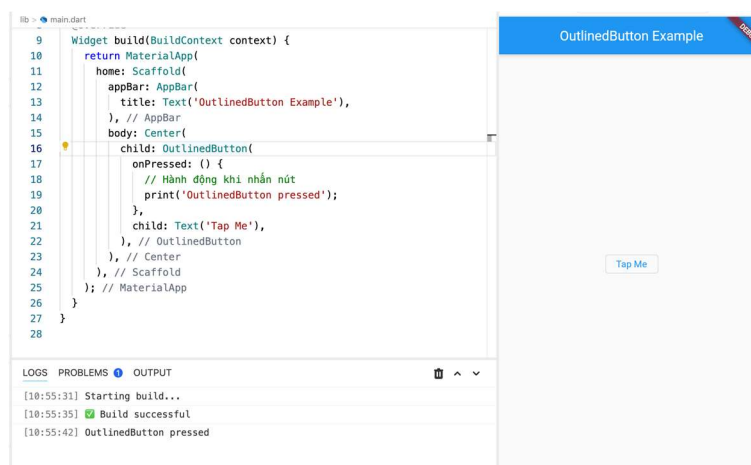
4.3.1.2 Elevated Button

`ElevatedButton` là một nút bấm nâng cao với hiệu ứng nảy lên đẹp mắt khi được nhấn, thường được sử dụng cho các hành động chính trong ứng dụng.



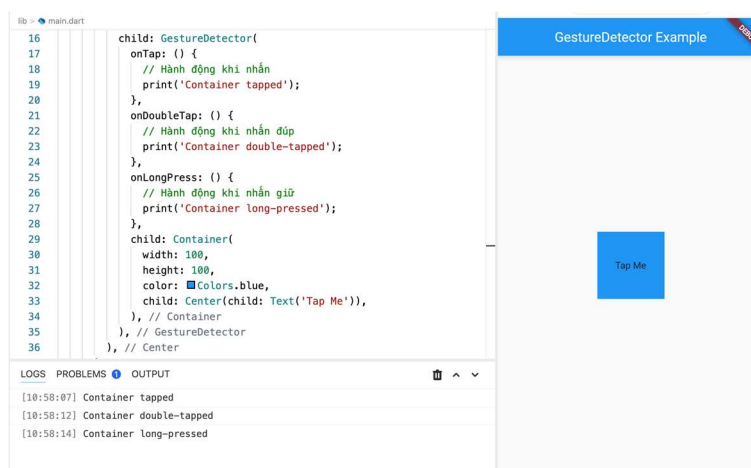
4.3.1.3 Outline Button

`OutlinedButton` là một nút bấm với viền, thường được sử dụng cho các hành động phụ.



4.3.2 Gestures

Gestures là các hành động của người dùng như chạm, kéo, vuốt mà ứng dụng có thể phản hồi. Flutter cung cấp các widget và lớp để xử lý các cử chỉ này một cách dễ dàng. Bên dưới là ví dụ minh họa về widget `GestureDetector`:



Cú pháp và giải thích:

- **GestureDetector**: Widget để phát hiện và xử lý các cử chỉ.
- **onTap**: Hàm gọi lại khi người dùng nhấn vào widget.
- **onDoubleTap**: Hàm gọi lại khi người dùng nhấn đúp vào widget.
- **onLongPress**: Hàm gọi lại khi người dùng nhấn giữ vào widget.

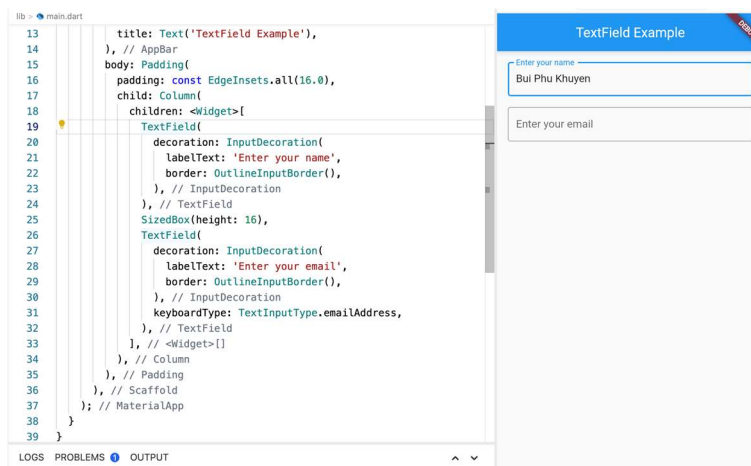
- **child**: Widget con, thường là widget hiển thị mà chúng ta muốn áp dụng các cử chỉ lên.

4.3.3 Input & Form

Trong Flutter, các input và form là các thành phần quan trọng để thu thập thông tin từ người dùng.

4.3.3.1 TextField

TextField là một widget cho phép người dùng nhập và chỉnh sửa văn bản. Nó thường được sử dụng để thu thập dữ liệu như tên, email, mật khẩu, v.v.

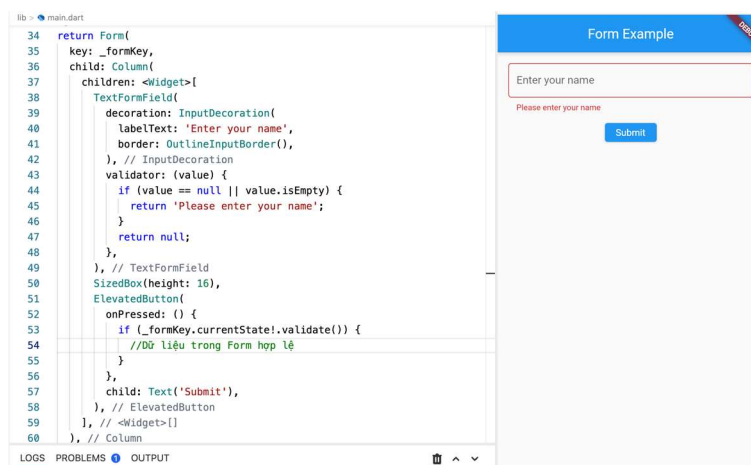


Giải thích về widget **TextField** và các thuộc tính của nó trên ví dụ:

- **decoration**: Thuộc tính **InputDecoration** để thêm nhãn và viền cho **TextField**.
- **labelText**: Văn bản nhãn hiển thị bên trên **TextField**.
- **border**: Kiểu viền của **TextField**, thường là **OutlineInputBorder**.
- **keyboardType**: Kiểu bàn phím, ví dụ **TextInputType.emailAddress** để nhập email.

4.3.3.2 Form

Form là một widget chứa các input field và cung cấp cơ chế xác thực và lưu trữ dữ liệu. Nó giúp chúng ta quản lý các input field và kiểm tra tính hợp lệ của dữ liệu người dùng.



Ví dụ này tạo một form đơn giản với một trường nhập liệu cho tên và một nút gửi. Khi nút được nhấn, form sẽ kiểm tra tính hợp lệ của dữ liệu và hiển thị thông báo nếu dữ liệu hợp lệ. Cụ thể ở ví dụ trên:

- **Form**: Widget chứa các **input field**.
- **key**: Thuộc tính **GlobalKey** để xác định form và quản lý trạng thái của nó.
- **TextFormField**: Một phiên bản của **TextField** với cơ chế xác thực.
- **validator**: Hàm xác thực giá trị của **TextFormField**.
- **ElevatedButton**: Nút để gửi form.

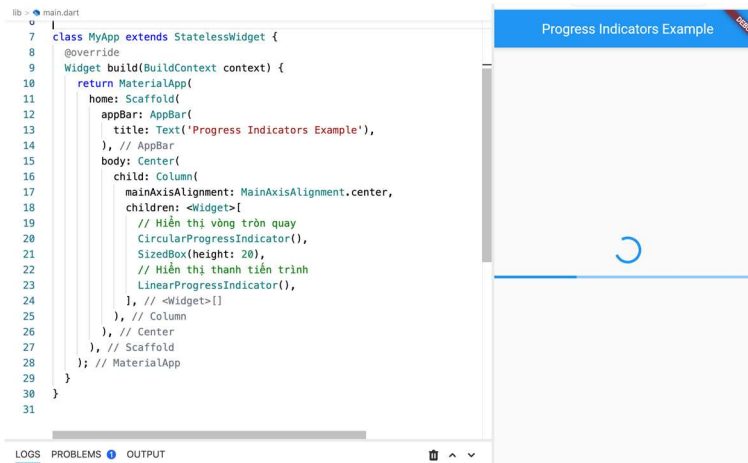
4.3.4 Visual Feedback

Trong Flutter, Visual Feedback là cách hiển thị thông tin hoặc trạng thái của ứng dụng cho người dùng. Các widget phản hồi trực quan giúp người dùng hiểu rõ hơn về những gì đang diễn ra trong ứng dụng, chẳng hạn như quá trình tải dữ liệu, thông báo ngăn gọn hoặc cảnh báo.

4.3.4.1 Progress Indicator

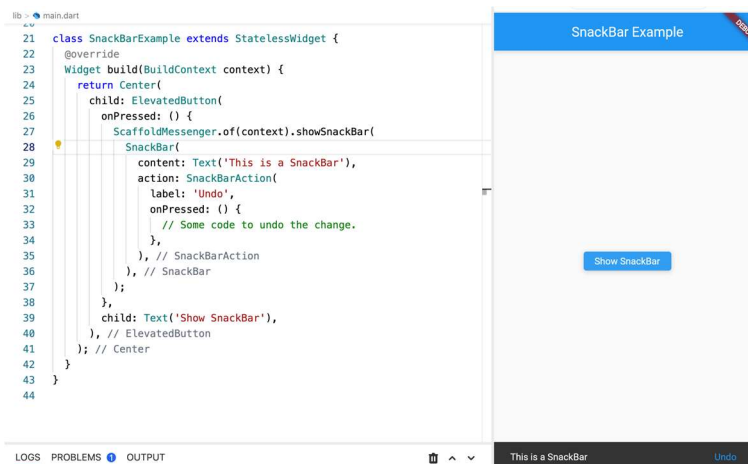
Progress Indicators là các widget dùng để hiển thị trạng thái đang xử lý của một tác vụ.

Flutter cung cấp 2 loại **Progress Indicators** phổ biến là **CircularProgressIndicator** và **LinearProgressIndicator**.



4.3.4.2 Snackbar

`SnackBar` là một widget hiển thị thông báo ngắn gọn ở dưới cùng của màn hình. Thường được sử dụng để cung cấp thông tin phản hồi cho người dùng sau khi họ thực hiện một hành động nào đó.



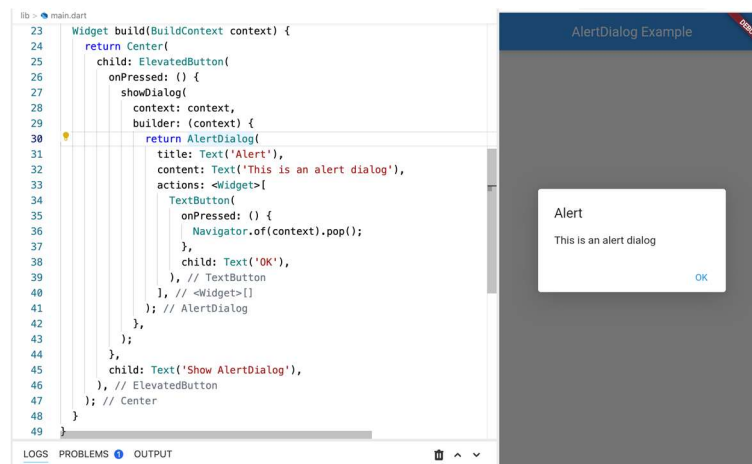
Cú pháp và giải thích:

- `ScaffoldMessenger.of(context).showSnackBar()`: Phương thức để hiển thị `SnackBar`.
- `SnackBar`: Widget hiển thị thông báo ngắn gọn ở dưới cùng của màn hình.
- `content`: Nội dung của `SnackBar`, thường là một widget `Text`.
- `action`: Hành động bổ sung cho `SnackBar`, thường là một `SnackBarAction`.

Ở ví dụ trên khi nhấn vào nút "Show SnackBar", một SnackBar sẽ xuất hiện ở dưới cùng của màn hình với thông báo "This is a SnackBar" và một hành động "Undo".

4.3.4.3 Alert Dialog

AlertDialog là một widget hiển thị hộp thoại cảnh báo, thường được sử dụng để yêu cầu người dùng xác nhận hoặc thực hiện một hành động nào đó.



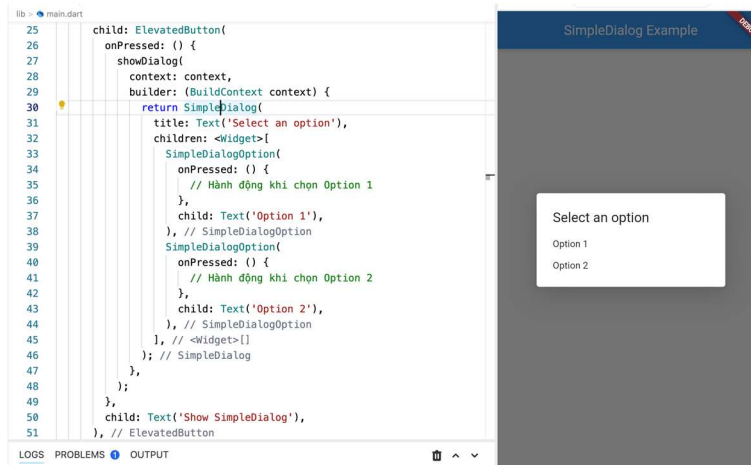
Cú pháp và giải thích:

- **showDialog**: Hàm để hiển thị hộp thoại cảnh báo.
- **AlertDialog**: Widget hiển thị hộp thoại cảnh báo.
- **title**: Tiêu đề của **AlertDialog**, thường là một widget **Text**.
- **content**: Nội dung của **AlertDialog**, thường là một widget **Text**.
- **actions**: Danh sách các hành động trong **AlertDialog**, thường là các nút bấm.
- **TextButton**: Widget cho nút, trong trường hợp này được sử dụng cho nút "OK".

Với ví dụ này, khi ta nhấn vào nút "Show AlertDialog", một **AlertDialog** sẽ xuất hiện với tiêu đề "Alert", nội dung "This is an alert dialog" và một nút "OK".

4.3.4.4 Simple Dialog

SimpleDialog là một widget hiển thị hộp thoại đơn giản, thường được sử dụng để cung cấp một danh sách các lựa chọn cho người dùng.



Cú pháp và giải thích:

- **showDialog**: Hàm để hiển thị hộp thoại đơn giản.
- **SimpleDialog**: Widget hiển thị hộp thoại đơn giản.
- **title**: Tiêu đề của **SimpleDialog**, thường là một widget **Text**.
- **children**: Danh sách các tùy chọn trong **SimpleDialog**, thường là các **SimpleDialogOption**.
- **SimpleDialogOption**: Widget cho các tùy chọn trong **SimpleDialog**.

Với ví dụ này, khi ta nhấn vào nút "Show SimpleDialog", một SimpleDialog sẽ xuất hiện với tiêu đề "Select an option" và hai tùy chọn "Option 1" và "Option 2". Ta có thể viết thêm các hàm xử lý khi người dùng chọn vào tại thuộc tính **onPressed**

4.4 ACCESSIBILITY & INTERNATIONALIZATION

4.4.1 Acessibility

Accessibility là khả năng truy cập của ứng dụng đối với người dùng có nhu cầu đặc biệt. Điều này bao gồm:

- Large fonts (Phông chữ lớn): Flutter hỗ trợ việc hiển thị văn bản với kích thước phông chữ do người dùng chỉ định.
- Screen readers (Đọc màn hình): Cung cấp phản hồi bằng giọng nói về nội dung của giao diện người dùng.

- Sufficient contrast (Độ tương phản đủ): Đảm bảo các widget được hiển thị với màu sắc có độ tương phản đủ để dễ đọc.

Kiểm tra hỗ trợ accessibility

- Android: Sử dụng Accessibility Scanner để quét và kiểm tra ứng dụng.
- iOS: Sử dụng Accessibility Inspector trong Xcode.

4.4.2 Internationalization

Internationalization (i18n) là việc xây dựng ứng dụng hỗ trợ đa ngôn ngữ, giúp ứng dụng của ta có thể sử dụng được bởi người dùng ở nhiều quốc gia và khu vực khác nhau.

Các bước cơ bản để thiết lập internationalization trong Flutter:

- Thêm package `flutter_localizations` và `intl` vào `pubspec.yaml`
- Cấu hình `MaterialApp` hoặc `CupertinoApp` để hỗ trợ đa ngôn ngữ:
- Tạo các tệp `.arb` cho từng ngôn ngữ
- Sử dụng `AppLocalizations` trong ứng dụng để hiển thị văn bản đã dịch:

Chú ý: Để biết thêm chi tiết về cách sử dụng và hướng dẫn từng bước, ta có thể truy cập tại trang chủ `flutter.dev`.

4.5 ANIMATIONS & TRANSITIONS

Hiệu ứng và Chuyển tiếp (Animations & Transitions) trong Flutter giúp tạo ra các ứng dụng sinh động và hấp dẫn hơn. Flutter cung cấp nhiều công cụ và widget để tạo và quản lý hiệu ứng, giúp cải thiện trải nghiệm người dùng.

• Implicit Animations

Implicit Animations (Hiệu ứng ngầm) là các hiệu ứng mà Flutter tự động xử lý khi trạng thái của widget thay đổi. Ta chỉ cần thay đổi giá trị và Flutter sẽ tự động xử lý hiệu ứng. Ví dụ: `AnimatedContainer`, `AnimatedOpacity`, `AnimatedAlign`.

- **Explicit Animations**

Explicit Animations (Hiệu ứng tường minh) yêu cầu ta tự quản lý trạng thái và tiến trình của hiệu ứng. Flutter cung cấp các lớp **Animation** và **AnimationController** để giúp ta làm điều này. Ví dụ: **AnimationController**, **CurvedAnimation**, **Tween**.

- **Hero Animations**

Hero Animations (Hiệu ứng Hero) là một kiểu hiệu ứng trong đó một widget (được gọi là hero) "bay" từ màn hình này sang màn hình khác. Hero Animations thường được sử dụng để tạo hiệu ứng chuyển tiếp mượt mà giữa hai màn hình có chứa các thành phần chung.

Cách thức hoạt động:

- Sử dụng hai widget Hero trong các **route** khác nhau nhưng có cùng **tag** để thực hiện hiệu ứng.
- Khi đẩy hoặc loại bỏ một **route** khỏi **stack** của **Navigator**, hiệu ứng sẽ được kích hoạt.
- Flutter tính toán một **RectTween** để xác định biên của **hero** khi bay từ **route** nguồn đến **route** đích.

- **Staggered Animations**

Staggered Animations (Hiệu ứng giai đoạn) là các hiệu ứng thực hiện theo từng giai đoạn hoặc chồng chéo lên nhau. Điều này giúp tạo ra các hiệu ứng phức tạp và đẹp mắt hơn. Hiệu ứng có thể diễn ra tuần tự hoặc có thể có các khoảng trống giữa các giai đoạn.

Cách thức hoạt động:

- Sử dụng nhiều đối tượng **Animation**.
- Một **AnimationController** điều khiển tất cả các hiệu ứng.
- Mỗi đối tượng **Animation** chỉ định hiệu ứng trong một **Interval**.
- Tạo **Tween** cho mỗi thuộc tính cần hiệu ứng.

Ví dụ:

- **basic_staggered_animation**: Hiển thị một loạt các hiệu ứng tuần tự và chồng chéo của một widget duy nhất.
- **staggered_pic_selection**: Hiển thị việc xóa hình ảnh từ danh sách hình ảnh với các hiệu ứng lựa chọn và xóa hình ảnh.

TÓM TẮT

Bài học này cung cấp cái nhìn tổng quan về các kỹ thuật thiết kế giao diện người dùng nâng cao sử dụng Flutter. Người học sẽ khám phá các chiến lược điều khiển Layout và Navigation, áp dụng Material Design và phát triển các thành phần tương tác như Buttons và Forms. Đồng thời, bài học đề cập đến việc thiết kế giao diện thân thiện với người dùng và hỗ trợ đa ngôn ngữ, cùng với việc sử dụng Animations và Transitions để tạo hiệu ứng giao diện động và đẹp mắt.

BÀI TẬP

Câu 1: Xây dựng giao diện đăng nhập. Yêu cầu:

- Tạo một giao diện Login với hai trường nhập liệu (email và mật khẩu) và một nút đăng nhập (Login Button).
- Hiển thị thông báo lỗi nếu người dùng không nhập đủ thông tin.
- Nếu nhập đầy đủ thông tin, hãy chuyển sang màn hình Dashboard, sử dụng BottomNavigationBar có 2 màn hình: Trang chủ (HomePage) và Hồ sơ (Profile). Đồng thời, hiển thị Snackbar thông báo đăng nhập thành công.

Câu 2: Thiết kế giao diện cho màn hình Trang chủ. Yêu cầu:

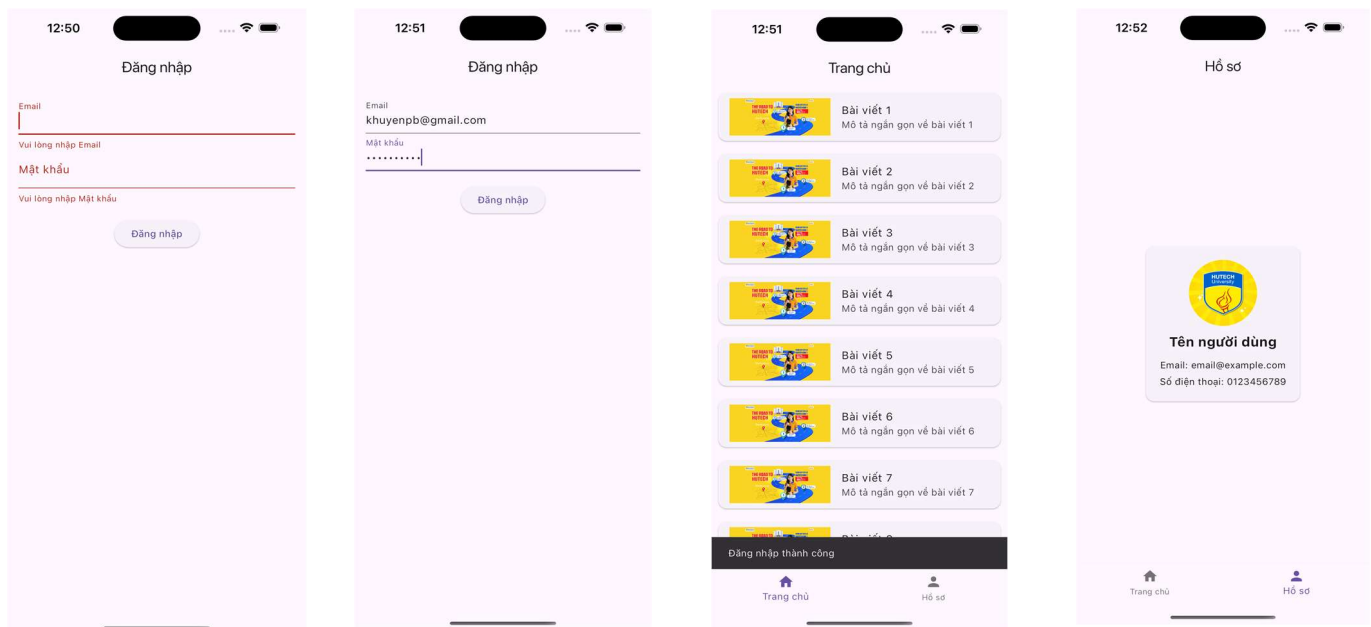
- Tạo một màn hình Trang chủ hiển thị danh sách các mục tin tức hoặc bài viết.
- Sử dụng ListView để hiển thị danh sách.

- Mỗi mục trong danh sách bao gồm một hình ảnh, tiêu đề và mô tả ngắn gọn, được bao quanh bởi một Card.

Câu 3: Thiết kế giao diện cho màn hình Hồ sơ. Yêu cầu

- Tạo một màn hình Hồ sơ hiển thị thông tin cá nhân của người dùng.
- Sử dụng CircleAvatar để hiển thị hình đại diện của người dùng.
- Hiển thị thông tin cá nhân như tên, email, số điện thoại trong một Card.

Tham khảo tổng quan giao diện của 3 bài tập trên



BÀI 5: THƯ VIỆN FLUTTER

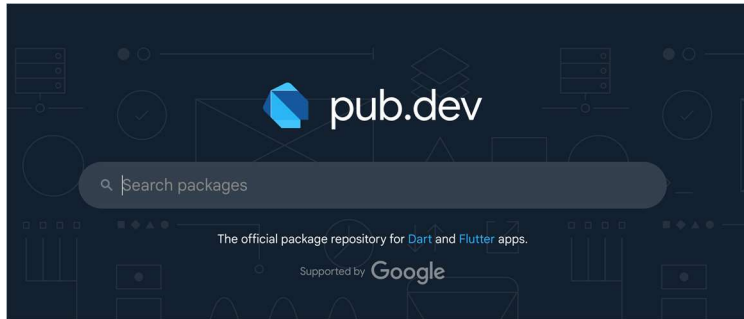
Bài học này sẽ trang bị cho người học các kiến thức về việc sử dụng và tích hợp các thư viện phổ biến từ pub.dev vào dự án Flutter. Người học sẽ tìm hiểu cách tìm kiếm, cài đặt và sử dụng các thư viện, cũng như các nhóm thư viện quan trọng như quản lý trạng thái, giao tiếp mạng, lưu trữ dữ liệu và tương tác với các chức năng hệ thống. Sau khi hoàn thành bài học này, người học cần nắm được:

- *Cách sử dụng nền tảng pub.dev để tìm kiếm và tích hợp các thư viện vào dự án Flutter.*
- *Hiểu và sử dụng các thư viện quản lý trạng thái như Provider, Flutter Bloc và Get.*
- *Cách giao tiếp với các API và dịch vụ web sử dụng các thư viện HTTP và Dio.*
- *Quản lý và lưu trữ dữ liệu cục bộ và từ xa bằng các thư viện như Shared Preferences và Firebase, SQLite.*
- *Hiển thị và tương tác với hình ảnh trong ứng dụng bằng Cached Network Image và Image Picker.*
- *Tương tác với các chức năng hệ thống như mở URL, kiểm tra kết nối mạng và hiển thị trang web bằng URL Launcher, WebView Flutter và Connectivity Plus.*

5.1 PUB.DEV

5.1.1 Giới thiệu

PUB.DEV là nền tảng cung cấp và quản lý các gói thư viện cho Dart và Flutter. Nó giúp các nhà phát triển dễ dàng tìm kiếm, cài đặt và quản lý các thư viện bên thứ ba để tích hợp vào ứng dụng của mình. Các gói trên pub.dev được đóng gói và chia sẻ bởi cộng đồng, mang lại giải pháp cho nhiều vấn đề thường gặp trong quá trình phát triển ứng dụng.



Các lợi ích của PUB.DEV:

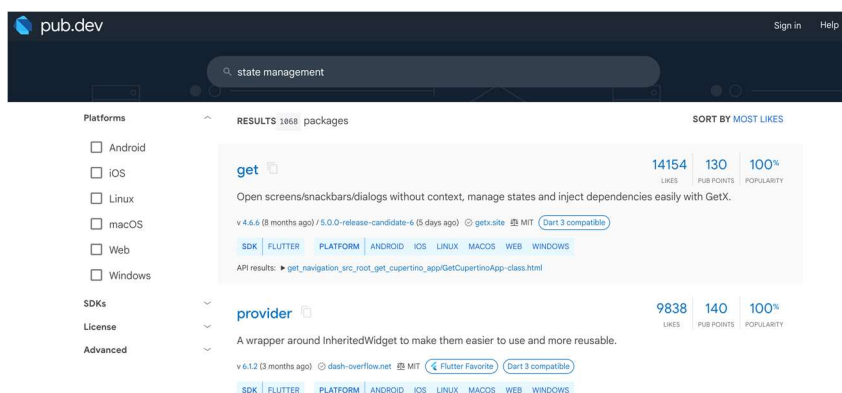
- Dễ dàng tìm kiếm: Giao diện thân thiện với bộ lọc tìm kiếm mạnh mẽ, giúp tìm kiếm thư viện phù hợp với nhu cầu.
- Cộng đồng mạnh mẽ: Hàng nghìn gói thư viện được chia sẻ bởi cộng đồng, cập nhật thường xuyên và hỗ trợ tốt.
- Quản lý dễ dàng: Các công cụ tích hợp sẵn giúp quản lý các gói thư viện một cách dễ dàng và hiệu quả.
- Tích hợp liền mạch: Tích hợp thư viện vào ứng dụng Flutter nhanh chóng chỉ với vài bước đơn giản.

5.1.2 Tích hợp thư viện vào Flutter

Việc tích hợp thư viện từ pub.dev vào dự án Flutter rất đơn giản và chỉ cần thực hiện một vài bước cơ bản. Dưới đây là hướng dẫn cụ thể từng bước:

Bước 1: Tìm kiếm thư viện trên pub.dev

- Truy cập trang web <https://pub.dev>.
- Sử dụng thanh tìm kiếm để tìm thư viện phù hợp với nhu cầu. Ví dụ: Để tìm thư viện quản lý trạng thái, ta có thể tìm kiếm "state management".



Bước 2: Thêm thư viện vào dự án Flutter

Có 2 cách để thêm thư viện vào dự án Flutter: chỉnh sửa trực tiếp file `pubspec.yaml` hoặc sử dụng lệnh `flutter pub add`.

- **Cách 1: Chỉnh sửa trực tiếp file `pubspec.yaml`**

- Trong dự án Flutter, mở file `pubspec.yaml`.
- Thêm tên và phiên bản của thư viện vào phần **dependencies**. Ví dụ, để thêm thư viện `provider`, ta thêm đoạn mã sau:

```
dependencies:  
  flutter:  
    sdk: flutter  
  provider: ^6.1.2
```

- Lưu file `pubspec.yaml`.
- Mở terminal trong thư mục dự án Flutter và chạy lệnh sau để cài đặt các thư viện đã thêm: `flutter pub get`

- **Cách 2: Sử dụng lệnh `flutter pub add`**

- Mở terminal trong thư mục dự án Flutter.
- Chạy lệnh sau để thêm thư viện (giả sử ta cần thêm thư viện `provider`):

`flutter pub add provider`

- Lệnh này sẽ thêm một dòng như sau vào file `pubspec.yaml` và tự động chạy `flutter pub get`:

```
dependencies:  
  provider: ^6.1.2
```

Bước 3: Sử dụng thư viện trong dự án

- Mở file Dart nơi ta muốn sử dụng thư viện.
- **Import** thư viện vào file Dart. Ví dụ, để sử dụng thư viện **provider**, ta chỉ thêm đoạn mã sau:

```
import 'package:provider/provider.dart';
```

- Sử dụng các tính năng của thư viện theo hướng dẫn trong tài liệu trên **pub.dev**.

5.2 THƯ VIỆN PHỔ BIẾN

5.2.1 Quản lý trạng thái (State Management)

Quản lý trạng thái là một phần quan trọng trong phát triển ứng dụng di động. Các thư viện quản lý trạng thái giúp duy trì và cập nhật trạng thái ứng dụng một cách hiệu quả, đồng thời giúp mã nguồn dễ đọc và bảo trì hơn.

5.2.1.1 Provider

Provider là một thư viện quản lý trạng thái đơn giản và hiệu quả cho Flutter, dựa trên `InheritedWidget`. Provider giúp làm giảm boilerplate và làm cho việc tái sử dụng và quản lý tài nguyên dễ dàng hơn. Một số đặc điểm của Provider:

- Đơn giản hóa việc cấp phát và hủy bỏ tài nguyên.
- Hỗ trợ **lazy-loading**.
- Tích hợp tốt với **devtool**, cho phép theo dõi trạng thái ứng dụng trong Flutter **devtool**.
- Cung cấp cách thức tiêu chuẩn để sử dụng các **InheritedWidget** (**Provider.of**, **Consumer**, **Selector**).
- Tăng khả năng mở rộng cho các lớp có cơ chế lắng nghe phức tạp.

Chi tiết thư viện: <https://pub.dev/packages/provider>

5.2.1.2 Flutter_Bloc

Flutter Bloc giúp tách biệt logic nghiệp vụ khỏi phần trình bày, làm cho mã nguồn nhanh chóng, dễ kiểm thử và tái sử dụng. Một số chức năng chính của thư viện này bao gồm:

- Quản lý trạng thái dựa trên các sự kiện và trạng thái.
- Hỗ trợ kiểm thử dễ dàng từng trường hợp để đảm bảo ứng dụng phản hồi đúng cách.
- Ghi lại mọi tương tác của người dùng để ra quyết định dựa trên dữ liệu.
- Giúp phát triển các ứng dụng nhanh và phản hồi nhanh.
- Cho phép nhiều nhà phát triển làm việc trong một mã nguồn duy nhất theo các mẫu và quy ước chung.

Chi tiết thư viện: https://pub.dev/packages/flutter_bloc

5.2.1.3 Get

Get là một giải pháp nhẹ và mạnh mẽ cho Flutter, kết hợp quản lý trạng thái hiệu suất cao, có cơ chế **DI** thông minh và quản lý route một cách nhanh chóng và thực tế. Chức năng chính của Get kể đến:

- Quản lý trạng thái không cần context.
- Quản lý cơ chế **DI** (tiêm phụ thuộc) và điều hướng không cần phụ thuộc vào cây widget.
- Hỗ trợ **lazy-loading** và loại bỏ tự động các **controller** không còn được sử dụng.
- Tách biệt hoàn toàn **logic** nghiệp vụ ra khỏi lớp giao diện.
- Cung cấp hệ sinh thái lớn với nhiều tính năng (đa ngôn ngữ, **snackbar**, **route**, **responsive...**), dễ sử dụng cho người mới bắt đầu.

Chi tiết thư viện: <https://pub.dev/packages/get>

5.2.2 Giao tiếp mạng (Networking)

Giao tiếp mạng là một phần không thể thiếu trong phát triển ứng dụng di động hiện đại. Các thư viện networking giúp tương tác với các API và dịch vụ web một cách hiệu quả.

5.2.2.1 http

Thư viện HTTP cung cấp các công cụ đơn giản và dễ sử dụng để gửi yêu cầu HTTP và xử lý phản hồi. Một số chức năng chính của HTTP:

- Gửi các yêu cầu HTTP GET, POST, PUT, DELETE.
- Xử lý phản hồi và dữ liệu JSON.
- Hỗ trợ cho các headers và body.

Chi tiết thư viện: <https://pub.dev/packages/http>

5.2.2.2 Dio

Dio là một thư viện HTTP mạnh mẽ hơn, cung cấp nhiều tính năng nâng cao. Một số chức năng chính của Dio:

- Hỗ trợ cho các yêu cầu HTTP phức tạp.
- Quản lý cookies và headers dễ dàng.
- Hỗ trợ interceptor để xử lý các yêu cầu và phản hồi.
- Hỗ trợ tải lên và tải xuống tệp tin.

Chi tiết thư viện: <https://pub.dev/packages/dio>

5.2.3 Lưu trữ dữ liệu

Lưu trữ dữ liệu là phần quan trọng để lưu trữ thông tin người dùng và trạng thái ứng dụng. Các thư viện lưu trữ giúp ta dễ dàng quản lý dữ liệu.

5.2.3.1 Shared Preferences

Shared Preferences cung cấp một cách đơn giản để lưu trữ dữ liệu dạng key-value. Một số chức năng chính của Shared Preferences:

- Lưu trữ dữ liệu dạng key-value.
- Dễ dàng truy xuất và cập nhật dữ liệu.
- Lưu trữ dữ liệu cục bộ trên thiết bị.

Chi tiết thư viện: https://pub.dev/packages/shared_preferences

5.2.3.2 Firebase

Firebase là một nền tảng phát triển ứng dụng mạnh mẽ của Google. Firebase có rất nhiều dịch vụ đi kèm, một số dịch vụ nổi tiếng không thể không nhắc đến như:

- Xác thực người dùng (Firebase Authentication).
- Lưu trữ và đồng bộ dữ liệu (Cloud Firestore, Realtime Database).
- Lưu trữ tệp tin (Firebase Storage)...

Các thư viện của Firebase: <https://pub.dev/publishers/firebase.google.com/packages>

5.2.3.3 SQFLite

SQFLite là một thư viện SQLite dành cho Flutter. Một số chức năng chính bao gồm:

- Thực hiện các truy vấn SQL.
- Lưu trữ dữ liệu dưới dạng bảng.
- Hỗ trợ các thao tác CRUD (Create, Read, Update, Delete).

Chi tiết thư viện: <https://pub.dev/packages/sqflite>

5.2.4 Hiện thị và tương tác hình ảnh (Image Interaction)

Hình ảnh là một phần không thể thiếu trong các ứng dụng di động. Các thư viện hiện thị và tương tác với hình ảnh giúp ta tải, hiện thị và tương tác với hình ảnh một cách dễ dàng và hiệu quả.

5.2.4.1 Cached Network Image

Cached Network Image là một thư viện mạnh mẽ giúp tải và lưu trữ cache hình ảnh từ mạng. Khác với việc sử dụng NetworkImage của Flutter mặc định, Cached Network Image cung cấp các tính năng như cache hình ảnh và hỗ trợ xử lý lỗi khi tải hình ảnh. Một số điểm mạnh bao gồm:

- Tải hình ảnh từ URL và lưu trữ cache để cải thiện hiệu suất và tiết kiệm băng thông.
- Hỗ trợ placeholder trong khi tải hình ảnh và widget lỗi khi không thể tải được hình ảnh.
- Quản lý cache dễ dàng với các tùy chọn xóa và làm mới cache.

Chi tiết thư viện: https://pub.dev/packages/cached_network_image

5.2.4.2 Image Picker

Image Picker là thư viện phổ biến cho phép người dùng chọn hình ảnh và video từ thư viện hoặc camera của thiết bị. Thư viện này rất dễ sử dụng và tích hợp mạnh mẽ với các chức năng hệ thống của thiết bị. Một số đặc điểm của thư viện này bao gồm:

- Chọn hình ảnh từ thư viện với giao diện người dùng gốc của thiết bị.
- Chụp ảnh trực tiếp từ camera và trả về hình ảnh đã chụp.
- Quay video từ camera và trả về video đã quay.

Chi tiết thư viện: https://pub.dev/packages/image_picker

5.2.5 Thông báo và quản lý quyền (Notifications and Permissions)

Các ứng dụng di động thường cần hiển thị thông báo và yêu cầu quyền truy cập từ người dùng. Các thư viện dưới đây giúp ta thực hiện các chức năng này một cách dễ dàng.

5.2.5.1 Flutter Local Notifications

Flutter Local Notifications giúp tạo và quản lý thông báo cục bộ trong ứng dụng Flutter. Một số chức năng chính bao gồm:

- Hiển thị thông báo với tiêu đề, nội dung và biểu tượng tùy chỉnh.
- Lên lịch thông báo vào thời gian cụ thể.
- Quản lý các hành động khi người dùng tương tác với thông báo.

Chi tiết thư viện: https://pub.dev/packages/flutter_local_notifications

5.2.5.2 Permission Handler

Permission Handler là thư viện giúp quản lý quyền truy cập trong ứng dụng Flutter. Thư viện này giúp kiểm tra và yêu cầu quyền truy cập một cách dễ dàng. Một số chức năng chính bao gồm:

- Kiểm tra trạng thái quyền truy cập.
- Yêu cầu quyền truy cập từ người dùng.
- Xử lý các trường hợp khi người dùng từ chối quyền truy cập.

Chi tiết thư viện: https://pub.dev/packages/permission_handler

5.2.6 Định vị và bản đồ (Location and Maps)

Các ứng dụng cần định vị và hiển thị bản đồ có thể sử dụng các thư viện dưới đây để tương tác với các dịch vụ định vị và bản đồ một cách dễ dàng.

5.2.6.1 Geolocator

Geolocator là thư viện giúp lấy thông tin vị trí của thiết bị. Thư viện này rất hữu ích cho các ứng dụng cần xác định vị trí hiện tại của người dùng. Một số chức năng chính bao gồm:

- Lấy tọa độ hiện tại của thiết bị.
- Theo dõi thay đổi vị trí của thiết bị.
- Tính toán khoảng cách giữa các điểm vị trí.

Chi tiết thư viện: <https://pub.dev/packages/geolocator>

5.2.6.2 Google Maps Flutter

Google Maps Flutter là thư viện giúp tích hợp Google Maps vào ứng dụng Flutter. Thư viện này cung cấp các công cụ để hiển thị bản đồ và tương tác với các dịch vụ của Google Maps. Một số chức năng chính bao gồm:

- Hiển thị bản đồ Google Maps trong ứng dụng.
- Thêm marker, polyline, polygon lên bản đồ.
- Tương tác với bản đồ thông qua các sự kiện như zoom, pan và click.

Chi tiết thư viện: https://pub.dev/packages/google_maps_flutter

5.2.7 Tương tác với các chức năng hệ thống (System Interaction)

Các ứng dụng di động thường cần tương tác với các chức năng hệ thống như mở URL, gửi email, kiểm tra kết nối mạng và hiển thị trang web.... Các thư viện dưới đây giúp ta thực hiện các chức năng này một cách dễ dàng.

5.2.7.1 URL Launcher

URL Launcher là một thư viện giúp mở URL trong trình duyệt, gửi email, gọi điện thoại và thực hiện các tác vụ liên quan đến URL một cách dễ dàng. Các chức năng chính bao gồm:

- Mở URL trong trình duyệt web của thiết bị.
- Gửi email với địa chỉ và nội dung được xác định trước.
- Gọi điện thoại và gửi tin nhắn SMS trực tiếp từ ứng dụng.

Chi tiết thư viện: https://pub.dev/packages/url_launcher

5.2.7.2 WebView Flutter

WebView Flutter là thư viện giúp hiển thị các trang web bên trong ứng dụng Flutter. Điều này cho phép tích hợp nội dung web mà không cần phải rời khỏi ứng dụng. Bên dưới là các chức năng chính của thư viện:

- Hiển thị trang web bên trong ứng dụng bằng WebView.
- Tương tác với nội dung trang web như một trình duyệt thông thường.
- Quản lý các sự kiện web như load trang, lỗi khi tải trang.

Chi tiết thư viện: https://pub.dev/packages/webview_flutter

5.2.7.3 Connectivity Plus

Connectivity Plus là thư viện giúp kiểm tra trạng thái kết nối mạng của thiết bị. Thư viện này rất hữu ích khi ta cần xác định tình trạng kết nối mạng trước khi thực hiện các tác vụ liên quan đến Internet. Một số đặc điểm của thư viện này bao gồm:

- Kiểm tra trạng thái kết nối mạng như WiFi, Mobile hoặc Offline.
- Theo dõi thay đổi trạng thái kết nối mạng và cập nhật trong thời gian thực.

Chi tiết thư viện: https://pub.dev/packages/connectivity_plus

5.2.7.4 Package Info Plus

Package Info Plus là thư viện giúp lấy thông tin về gói ứng dụng như tên, phiên bản và mã phiên bản. Thư viện này rất hữu ích khi cần hiển thị thông tin ứng dụng hoặc kiểm tra phiên bản hiện tại. Một số chức năng chính bao gồm:

- Lấy tên gói ứng dụng.

- Lấy tên phiên bản và mã phiên bản của ứng dụng.
- Hỗ trợ cả trên Android và iOS.

Chi tiết thư viện: https://pub.dev/packages/package_info_plus

TÓM TẮT

Trong bài học này cung cấp cái nhìn tổng quan về cách tích hợp và sử dụng các thư viện phổ biến trong Flutter từ nền tảng pub.dev. Người học sẽ hiểu rõ về các nhóm thư viện quan trọng như quản lý trạng thái, giao tiếp mạng, lưu trữ dữ liệu và tương tác với các chức năng hệ thống. Qua đó, sẽ có khả năng tìm kiếm, cài đặt và sử dụng các thư viện này để xây dựng các ứng dụng Flutter hiệu quả và mạnh mẽ.

BÀI TẬP

Câu 1: So sánh hai cách thêm thư viện vào dự án Flutter: Chỉnh sửa trực tiếp file `pubspec.yaml` và sử dụng lệnh `flutter pub add`.

Câu 2: Tại sao phải quản lý trạng thái trên các ứng dụng di động viết bằng Flutter ?

Câu 3: Lợi ích của việc sử dụng thư viện như `HTTP` hoặc `Dio` so với việc sử dụng các phương pháp `HTTP` gốc của Dart là gì?

Câu 4: Các phương pháp lưu trữ dữ liệu phổ biến trong Flutter là gì và khi nào nên sử dụng mỗi phương pháp?

Câu 5: Xây dựng ứng dụng quản lý công việc cá nhân (**To-Do List**). Yêu cầu :

- Hiển thị danh sách công việc bằng `ListView`. Mỗi công việc hiển thị tên công việc, mô tả ngắn, ảnh đính kèm (nếu có) và nút để đánh dấu đã hoàn thành.
- Khi nhấn nút thêm công việc, mở `Dialog` để nhập tên, mô tả công việc mới và đính kèm hình ảnh (nếu có).

- Khi nhấn nút đánh dấu hoàn thành, cập nhật trạng thái công việc và di chuyển công việc đó xuống dưới danh sách.

Định hướng các thư viện có thể sử dụng để giải quyết bài tập trên :

- Lưu trữ dữ liệu các công việc: **Shared Preferences**, **Sqflite**, **Firestore**, **Hive**,...
- Đính kèm hình ảnh từ thư viện/chụp ảnh trực tiếp : **Image Picker**

BÀI 6: LẬP TRÌNH MẠNG VÀ CƠ SỞ DỮ LIỆU

Bài học này cung cấp một cái nhìn toàn diện về việc làm việc với dữ liệu trong ứng dụng Flutter, bao gồm cách xử lý JSON, sử dụng REST APIs, lưu trữ dữ liệu đơn giản bằng Shared Preferences và quản lý cơ sở dữ liệu SQL bằng SQLite. Ngoài ra, bài học còn giới thiệu cách tích hợp Firebase vào ứng dụng Flutter để sử dụng các dịch vụ như Realtime Database, Firestore và FlutterFire UI. Qua bài học, người học sẽ:

- *Hiểu và xử lý dữ liệu JSON trong ứng dụng Flutter.*
- *Sử dụng REST APIs để lấy và gửi dữ liệu giữa ứng dụng và máy chủ.*
- *Lưu trữ và quản lý dữ liệu đơn giản bằng Shared Preferences.*
- *Làm việc với cơ sở dữ liệu SQL trên thiết bị di động bằng SQLite.*
- *Tích hợp và sử dụng các dịch vụ Firebase như Realtime Database, Firestore và FlutterFire UI để quản lý và hiển thị dữ liệu.*

6.1 XỬ LÝ JSON

JSON (JavaScript Object Notation) là một định dạng dữ liệu nhẹ và dễ đọc, được sử dụng rộng rãi để trao đổi dữ liệu giữa máy khách (**client**) và máy chủ (**server**). Trong Flutter, việc xử lý **JSON** rất quan trọng để truyền và nhận dữ liệu từ các dịch vụ web.

6.1.1 Tổng quan về JSON

JSON là một kiểu định dạng dữ liệu tuân theo quy luật nhất định mà hầu hết các ngôn ngữ lập trình hiện nay đều có thể đọc được. JSON sử dụng các cặp **key - value** để lưu trữ dữ liệu và hỗ trợ các cấu trúc dữ liệu như đối tượng (**object**) và mảng (**array**).

Xét ví dụ sau:

```
{  
  "name": "Bùi Phú Khuyên",  
  "job": "Giảng viên",  
  "faculty": "Công nghệ thông tin"  
}
```

Ta thấy rằng JSON có các cặp **key-value**. Mỗi **key** phải nằm trong dấu nháy kép ("") và **value** có thể là một chuỗi (**string**), số (**number**), rỗng (**null**), mảng (**array**) hoặc đối tượng (**object**).

Cấu trúc JSON thường dùng:

- **Object**: Được biểu diễn bằng dấu ngoặc nhọn {}. Các cặp **key-value** nằm bên trong dấu ngoặc nhọn. Ví dụ:

```
{  
  "firstName": "Khuyên",  
  "lastName": "Bùi",  
  "age": 26  
}
```

- **Array**: Được biểu diễn bằng dấu ngoặc vuông []. Một mảng chứa nhiều giá trị hoặc đối tượng. Ví dụ:

```
[  
  {  
    "name": "Khuyên",  
    "age": 26,  
    "gender": "Nam"  
  },  
  {  
    "name": "Cường",  
    "age": 30,  
    "gender": "Nam"  
  },  
  {  
    "name": "Thảo",  
    "age": 25,  
    "gender": "Nữ"  
  }  
]
```

- **Nested Object:** Là đối tượng lồng bên trong đối tượng khác. Ví dụ:

```
{
  "khuyen": {
    "name": "Khuyên",
    "age": 26,
    "gender": "Nam"
  },
  "cuong": {
    "name": "Cường",
    "age": 30,
    "gender": "Nam"
  },
  "thao": {
    "name": "Thảo",
    "age": 25,
    "gender": "Nữ"
  }
}
```

6.1.2 Seriblize

Serialize là quá trình chuyển đổi một đối tượng Dart thành một chuỗi **JSON**. Quá trình này rất hữu ích khi ta cần gửi dữ liệu từ ứng dụng Flutter đến máy chủ.

Giả sử ta có một class **User** và ta muốn chuyển đổi đối tượng của **class** này thành **JSON** cần trải qua 2 bước:

Bước 1: Tạo Model User

```
class User {
  String name;
  int age;

  User({required this.name, required this.age});

  // Phương thức để chuyển đổi từ đối tượng Dart sang JSON
  Map<String, dynamic> toJson() => {
    'name': name,
    'age': age,
  };
}
```

Bước 2: Chuyển đổi tượng sang JSON

```
import 'dart:convert';

void main() {
  User user = User(name: 'Khuyên', age: 26);
  String jsonString = jsonEncode(user.toJson());
  print(jsonString); // Output: {"name":"Khuyên","age":26}
}
```

Trong ví dụ này, chúng ta đã tạo một class **User** với hai thuộc tính là **name** và **age**. Phương thức **toJson** giúp chuyển đổi đối tượng **User** thành một **Map**, sau đó sử dụng **jsonEncode** để chuyển **Map** này thành một chuỗi **JSON**.

6.1.3 Deserialize

Deserialize là quá trình chuyển đổi một chuỗi **JSON** thành một đối tượng Dart. Quá trình này rất quan trọng khi ta nhận dữ liệu từ máy chủ và cần sử dụng nó trong ứng dụng Flutter.

Giả sử ta nhận được một chuỗi **JSON** và muốn chuyển nó thành đối tượng **User** cũng cần trải qua 2 bước:

Bước 1: Tạo Model User

```
class User {
  String name;
  int age;

  User({required this.name, required this.age});

  // Phương thức để chuyển đổi từ JSON sang đối tượng Dart
  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      name: json['name'],
      age: json['age'],
    );
  }
}
```

Bước 2: Chuyển đổi JSON sang đối tượng

```
import 'dart:convert';

void main() {
  String jsonString = '{"name":"Khuyên","age":26}';
  Map<String, dynamic> userMap = jsonDecode(jsonString);
  User user = User.fromJson(userMap);
  print('Name: ${user.name}, Age: ${user.age}');
  //Output: Name: Khuyên, Age: 26
}
```

Trong ví dụ này, phương thức `fromJson` giúp chuyển đổi một **Map** (được tạo từ chuỗi JSON) thành một đối tượng **User**.

6.2 LÀM VIỆC VỚI REST APIS

6.2.1 Giới thiệu về REST APIs

REST APIs là một loại API sử dụng các phương thức HTTP (như GET, POST, PUT, DELETE) để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên tài nguyên. Các REST APIs thường trả về dữ liệu dưới dạng JSON, giúp dễ dàng tương tác và xử lý trong ứng dụng Flutter.

6.2.2 Xử lý dữ liệu

Dự án ví dụ sau minh họa cách làm việc với REST APIs trong Flutter. Mục tiêu của dự án là lấy dữ liệu từ một API (lấy thông tin chi tiết của 1 bài viết), chuyển đổi dữ liệu JSON thành đối tượng Dart và hiển thị dữ liệu đó trên giao diện người dùng. Các bước chính bao gồm:

Bước 1: Xây dựng lớp **Post** chứa các hàm **serialize** và **deserialize JSON**

```
class Post {
  final int id;
  final String title;
  final String body;

  Post({required this.id, required this.title, required this.body});

  // Hàm tạo đối tượng Post từ JSON
  factory Post.fromJson(Map<String, dynamic> json) {
```



```
return Post(  
  id: json['id'],  
  title: json['title'],  
  body: json['body'],  
);  
}  
  
// Hàm chuyển đổi đối tượng Post thành JSON  
Map<String, dynamic> toJson() => {  
  'id': id,  
  'title': title,  
  'body': body,  
};  
}
```

Giải thích:

- **Post**: Lớp biểu diễn đối tượng dữ liệu với các thuộc tính **id**, **title** và **body**.
- **fromJson**: Hàm tạo đối tượng **Post** từ dữ liệu **JSON**.
- **toJson**: Hàm chuyển đổi đối tượng **Post** thành **JSON** để gửi đi.

Bước 2: Xây dựng lớp ApiService để gửi yêu cầu và xử lý phản hồi HTTP

- Sử dụng thư viện **http** hoặc **dio** để gửi yêu cầu đến **server**. (Xem hướng dẫn cài đặt và sử dụng ở bài 5)
- Xây dựng lớp **ApiService** như ví dụ minh họa sử dụng **http**:

```
import 'package:http/http.dart' as http;  
import 'dart:convert';  
import 'post.dart';  
  
class ApiService {  
  //Url lấy thông tin bài viết (có id là 1)  
  final String url = 'https://my-json-server.typicode.com/buiphukhuyen/api/posts/1';  
  
  // Hàm gửi yêu cầu HTTP và nhận phản hồi  
  Future<Post> fetchPost() async {  
    final response = await http.get(Uri.parse(url));  
  
    if (response.statusCode == 200) {
```

```
// Nếu yêu cầu thành công, chuyển đổi JSON thành đối tượng Dart
Map<String, dynamic> data = jsonDecode(response.body);
return Post.fromJson(data);
} else {
// Nếu yêu cầu thất bại, ném ra ngoại lệ
throw Exception('Failed to load post');
}
}
}
```

Giải thích:

- **http.get**: Gửi yêu cầu HTTP GET đến URL.
- **response.statusCode**: Kiểm tra mã trạng thái của phản hồi.
- **jsonDecode**: Chuyển đổi chuỗi JSON thành tập dữ liệu **Map** Dart.
- **Post.fromJson**: Chuyển đổi dữ liệu JSON thành đối tượng **Post**.

Bước 3: Xử lý dữ liệu hiển thị giao diện Flutter

Cuối cùng, chúng ta sử dụng Flutter để hiển thị dữ liệu trên giao diện người dùng. Chúng ta sẽ sử dụng **FutureBuilder** để xử lý dữ liệu bất đồng bộ từ API và hiển thị nó trên màn hình.

```
import 'package:flutter/material.dart';
import 'ApiService.dart';
import 'post.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter REST API Example',
      theme: ThemeData(
        primarySwatch: Colors.blue,
```

```
    ),  
    home: PostScreen(),  
  );  
}  
}  
  
class PostScreen extends StatefulWidget {  
  @override  
  _PostScreenState createState() => _PostScreenState();  
}  
  
class _PostScreenState extends State<PostScreen> {  
  late Future<Post> futurePost;  
  
  @override  
  void initState() {  
    super.initState();  
    // Gọi hàm fetchPost khi khởi tạo  
    futurePost = ApiService().fetchPost();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Flutter REST API Example'),  
      ),  
      body: Center(  
        // Sử dụng FutureBuilder để hiển thị dữ liệu từ API  
        child: FutureBuilder<Post>(  
          future: futurePost,  
          builder: (context, snapshot) {  
            if (snapshot.connectionState == ConnectionState.waiting) {
```

```
        // Hiển thị vòng tròn loading khi đang chờ dữ liệu
        return CircularProgressIndicator();
    } else if (snapshot.hasError) {
        // Hiển thị lỗi nếu có
        return Text('Error: ${snapshot.error}');
    } else if (snapshot.hasData) {
        // Hiển thị dữ liệu khi thành công
        return Text('Title: ${snapshot.data!.title}\nBody:
${snapshot.data!.body}');
    } else {
        // Hiển thị khi không có dữ liệu
        return Text('No data');
    }
  },
),
);
}
```

Giải thích:

- **FutureBuilder**: Widget để xây dựng giao diện dựa trên giá trị của **Future**.
- **snapshot.connectionState**: Kiểm tra trạng thái kết nối của **Future**.
- **snapshot.hasData**: Kiểm tra xem dữ liệu có sẵn hay không và hiển thị nó.

6.3 SHARED PREFERENCES

6.3.1 Giới thiệu

Shared Preferences là một plugin trong Flutter dùng để lưu trữ dữ liệu đơn giản dưới dạng **key-value** trên các nền tảng khác nhau.

Đây là phương pháp lưu trữ dữ liệu không đồng bộ, giúp lưu trữ các dữ liệu nhỏ như cấu hình ứng dụng, trạng thái đăng nhập của người dùng.

Shared Preferences không được khuyến khích sử dụng để lưu trữ các dữ liệu quan trọng do cơ chế ghi dữ liệu vào bộ nhớ máy. Nếu gỡ ứng dụng sẽ mất toàn bộ dữ liệu.

Các kiểu dữ liệu được hỗ trợ bao gồm: `int`, `double`, `bool`, `String` và `List<String>`.

6.3.2 Cách sử dụng

Để sử dụng Shared Preferences trong Flutter, ta cần lưu ý các vấn đề sau:

- **Cài đặt và thêm thư viện `shared_preferences` vào dự án Flutter** (có thể xem lại bài 5)
- **Lưu trữ dữ liệu:** Sử dụng các phương thức `set`

```
void saveData() async {  
  //Khởi tạo Shared Preferences  
  final prefs = await SharedPreferences.getInstance();  
  
  //Lưu trữ giá trị int 10 vào key 'counter'  
  await prefs.setInt('counter', 10);  
  
  //Lưu trữ giá trị bool true vào key 'repeat'  
  await prefs.setBool('repeat', true);  
  
  //Lưu trữ giá trị double 1.5 vào key 'decimal'  
  await prefs.setDouble('decimal', 1.5);  
  
  //Lưu trữ giá trị String Start vào key 'action'  
  await prefs.setString('action', 'Start');  
  
  //Lưu trữ danh sách String vào key 'items'  
  await prefs.setStringList('items', <String>['Earth', 'Moon', 'Sun']);  
}
```

- **Lấy dữ liệu:** Sử dụng các phương thức `get`.

```
void loadData() async {  
  final prefs = await SharedPreferences.getInstance();  
  //Shared Preferences sẽ tìm kiếm và lấy data với key truyền vào. Nếu không tồn  
  tại giá trị chứa key đó sẽ trả về null
```

```
int? counter = prefs.getInt('counter');
bool? repeat = prefs.getBool('repeat');
double? decimal = prefs.getDouble('decimal');
String? action = prefs.getString('action');
List<String>? items = prefs.getStringList('items');
}
```

- **Xoá dữ liệu:** Sử dụng các phương thức `remove`.

```
void removeData() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.remove('counter'); // Xóa dữ liệu với key 'counter'
}
```

6.4 SQLFLITE

6.4.1 Giới thiệu

SQLite là một hệ quản trị cơ sở dữ liệu quan hệ nhúng, nhẹ, dễ sử dụng và không yêu cầu một server riêng biệt. SQLite thường được sử dụng trong ứng dụng di động để lưu trữ dữ liệu cục bộ như thông tin người dùng, cấu hình ứng dụng hoặc các dữ liệu khác mà ta muốn lưu trữ một cách có tổ chức.

Thư viện `sqflite` là một plugin Flutter phổ biến cho phép tương tác với SQLite một cách dễ dàng. `sqflite` cung cấp các phương thức để thực hiện các thao tác cơ bản như thêm, sửa, xóa và truy vấn dữ liệu.

6.4.2 Cách sử dụng

- **Cài đặt và thêm thư viện `sqflite` vào dự án Flutter** (có thể xem lại bài 5)
- **Mở cơ sở dữ liệu**

SQLite database được lưu trữ dưới dạng tệp trong hệ thống tệp, được xác định bởi một đường dẫn. Nếu đường dẫn là tương đối, nó sẽ là tương đối so với đường dẫn mặc định do `getDatabasesPath()` trả về.

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

// Mở cơ sở dữ liệu
```

```
var db = await openDatabase('demo.db');

// Đóng cơ sở dữ liệu
await db.close();
```

- **Thực hiện truy vấn bằng Raw SQL queries**

Dưới đây là ví dụ về cách thực hiện các truy vấn SQL Raw sử dụng sqflite:

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

// Lấy đường dẫn lưu trữ cơ sở dữ liệu
var databasesPath = await getDatabasesPath();
String path = join(databasesPath, 'demo.db');

// Xóa cơ sở dữ liệu
await deleteDatabase(path);

// Mở cơ sở dữ liệu
Database database = await openDatabase(path, version: 1, onCreate: (Database db,
int version) async {
  // Khi tạo cơ sở dữ liệu, tạo bảng
  await db.execute('CREATE TABLE Test (id INTEGER PRIMARY KEY, name TEXT, value
INTEGER, num REAL)');
});

// Chèn một số bản ghi trong một giao dịch
await database.transaction((txn) async {
  int id1 = await txn.rawInsert('INSERT INTO Test(name, value, num) VALUES("some
name", 1234, 456.789)');
  print('inserted1: $id1');
  int id2 = await txn.rawInsert('INSERT INTO Test(name, value, num) VALUES(?, ?,
?)', ['another name', 12345678, 3.1416]);
  print('inserted2: $id2');
});

// Cập nhật một số bản ghi
int count = await database.rawUpdate('UPDATE Test SET name = ?, value = ? WHERE
name = ?', ['updated name', '9876', 'some name']);
```

```
print('updated: $count');

// Truy vấn các bản ghi
List<Map> list = await database.rawQuery('SELECT * FROM Test');
print(list);

// Đếm các bản ghi
count = Sqflite.firstIntValue(await database.rawQuery('SELECT COUNT(*) FROM Test'));
print('count: $count');

// Xóa một bản ghi
count = await database.rawDelete('DELETE FROM Test WHERE name = ?', ['another name']);
print('deleted: $count');

// Đóng cơ sở dữ liệu
await database.close();
```

- **Thực hiện truy vấn bằng SQL Helpers**

Ngoài cách sử dụng truy vấn trực tiếp thông qua câu lệnh SQL. Sqflite còn hỗ trợ truy vấn thông qua SQL helpers. Dưới đây là ví dụ về cách sử dụng các SQL helpers để quản lý cơ sở dữ liệu:

```
final String tableTodo = 'todo';
final String columnId = '_id';
final String columnTitle = 'title';
final String columnDone = 'done';

class Todo {
  int? id;
  String title;
  bool done;

  Todo({this.id, required this.title, required this.done});

  // Chuyển đổi đối tượng Todo thành map
```



```
Map<String, Object?> toMap() {
    var map = <String, Object?>{
        columnTitle: title,
        columnDone: done ? 1 : 0
    };
    if (id != null) {
        map[columnId] = id;
    }
    return map;
}

// Tạo đối tượng Todo từ map
Todo.fromMap(Map<String, Object?> map) {
    id = map[columnId] as int?;
    title = map[columnTitle] as String;
    done = map[columnDone] == 1;
}
}

class TodoProvider {
    late Database db;

    // Mở cơ sở dữ liệu
    Future open(String path) async {
        db = await openDatabase(path, version: 1, onCreate: (Database db, int version)
    async {
        await db.execute('''
create table $tableTodo (
    $columnId integer primary key autoincrement,
    $columnTitle text not null,
    $columnDone integer not null)
''');
    }
}
```

```
});  
}  
  
// Chèn một bản ghi Todo  
Future<Todo> insert(Todo todo) async {  
    todo.id = await db.insert(tableTodo, todo.toMap());  
    return todo;  
}  
  
// Lấy một bản ghi Todo theo ID  
Future<Todo?> getTodo(int id) async {  
    List<Map> maps = await db.query(tableTodo,  
        columns: [columnId, columnDone, columnTitle],  
        where: '$columnId = ?',  
        whereArgs: [id]);  
    if (maps.isNotEmpty) {  
        return Todo.fromMap(maps.first);  
    }  
    return null;  
}  
  
// Xóa một bản ghi Todo theo ID  
Future<int> delete(int id) async {  
    return await db.delete(tableTodo, where: '$columnId = ?', whereArgs: [id]);  
}  
  
// Cập nhật một bản ghi Todo  
Future<int> update(Todo todo) async {  
    return await db.update(tableTodo, todo.toMap(),  
        where: '$columnId = ?', whereArgs: [todo.id]);  
}  
  
// Đóng cơ sở dữ liệu
```

```
Future close() async => db.close();  
}
```

6.5 FIREBASE

6.5.1 Giới thiệu Firebase

Firebase là một nền tảng phát triển ứng dụng của Google, cung cấp nhiều dịch vụ hỗ trợ phát triển ứng dụng di động và web như xác thực người dùng, cơ sở dữ liệu thời gian thực, lưu trữ tệp tin, thông báo đẩy và nhiều dịch vụ khác. Firebase giúp các nhà phát triển dễ dàng xây dựng và mở rộng ứng dụng mà không cần quản lý cơ sở hạ tầng phía sau.

Các dịch vụ chính của Firebase bao gồm:

- Firebase Authentication: Xác thực người dùng qua email, số điện thoại và các nhà cung cấp xác thực bên thứ ba như Google, Facebook, GitHub.
- Firebase Realtime Database: Cơ sở dữ liệu NoSQL lưu trữ và đồng bộ hóa dữ liệu giữa người dùng theo thời gian thực.
- Firebase Firestore: Cơ sở dữ liệu NoSQL hiện đại hơn, hỗ trợ truy vấn mạnh mẽ và mở rộng tốt hơn.
- Firebase Storage: Lưu trữ và chia sẻ tệp tin.
- Firebase Cloud Messaging: Dịch vụ thông báo đẩy.
- Firebase Analytics: Công cụ phân tích ứng dụng.

6.5.2 Firebase Core

Firebase Core là gói cốt lõi để tích hợp Firebase vào ứng dụng Flutter. Nó cung cấp các chức năng cơ bản để khởi tạo và cấu hình Firebase.

- **Cài đặt thư viện:** Bằng cách chạy câu lệnh `flutter pub add firebase_core`
- **Khởi tạo Firebase:** Trong hàm main dự án Flutter

```
import 'package:firebase_core/firebase_core.dart';
```

```
void main() async {  
  // Đảm bảo rằng Flutter đã được khởi tạo.  
  WidgetsFlutterBinding.ensureInitialized();  
  // Khởi tạo Firebase.  
  await Firebase.initializeApp();  
  // Chạy ứng dụng.  
  runApp(MyApp());  
}
```

Bên cạnh đó ta cần đăng ký dự án Flutter của mình trên Firebase Console tại <https://console.firebase.google.com> để sử dụng được các dịch vụ của Firebase trên Flutter (FlutterFire)

6.5.3 Firebase Realtime Database & Firestore

6.5.3.1 Firebase Realtime Database

Firebase Realtime Database là cơ sở dữ liệu NoSQL cho phép lưu trữ và đồng bộ hóa dữ liệu theo thời gian thực giữa các thiết bị, phù hợp cho các ứng dụng yêu cầu cập nhật dữ liệu ngay lập tức như chat, trò chuyện hoặc các ứng dụng cộng tác trực tuyến.

Các khái niệm chính

- **DatabaseReference**: Tham chiếu đến vị trí của dữ liệu trong cơ sở dữ liệu. Mỗi **DatabaseReference** có thể được sử dụng để thực hiện các thao tác đọc và ghi dữ liệu.
- **DatabaseEvent**: Sự kiện được phát ra khi dữ liệu tại vị trí tham chiếu thay đổi.

Cài đặt: Để sử dụng ta cần cài đặt thư viện cho dự án Flutter bằng câu lệnh:

```
flutter pub add firebase_database
```

Các cú pháp thường được sử dụng:

- **Đọc dữ liệu**

```
// Tạo tham chiếu đến vị trí "users/123"  
DatabaseReference ref = FirebaseDatabase.instance.ref("users/123");  
// Đọc dữ liệu một lần từ cơ sở dữ liệu  
ref.once().then((DatabaseEvent event) {
```

```
print('Data: ${event.snapshot.value}');  
});
```

Trong đó:

- **DatabaseReference** là tham chiếu đến vị trí của dữ liệu trong cơ sở dữ liệu.
- **once()** được sử dụng để lấy dữ liệu một lần từ cơ sở dữ liệu và không lắng nghe các thay đổi sau đó.
- **event.snapshot.value** chứa dữ liệu từ cơ sở dữ liệu.

- **Ghi dữ liệu**

```
DatabaseReference ref = FirebaseDatabase.instance.ref("users/123");  
// Ghi dữ liệu vào cơ sở dữ liệu  
await ref.set({  
  "name": "Phú Khuyên",  
  "age": 26  
});
```

Hàm **set()** được sử dụng để lưu trữ dữ liệu tại vị trí chỉ định, thay thế bất kỳ dữ liệu nào đã tồn tại tại vị trí đó.

- **Xoá dữ liệu**

```
DatabaseReference ref = FirebaseDatabase.instance.ref("users/123");  
// Xoá dữ liệu từ cơ sở dữ liệu  
await ref.remove();
```

Hàm **remove()** được sử dụng để xoá dữ liệu tại vị trí chỉ định.

6.5.3.2 Firebase Cloud Firestore

Firestore là cơ sở dữ liệu NoSQL hiện đại hơn, cung cấp khả năng truy vấn mạnh mẽ và mở rộng tốt hơn so với Realtime Database. Nó phù hợp cho các ứng dụng cần quản lý dữ liệu phức tạp và yêu cầu truy vấn linh hoạt, ví dụ như quản lý dữ liệu người dùng, các ứng dụng thương mại điện tử...

Các khái niệm chính

- **Collection**: Một tập hợp các tài liệu (**documents**). Mỗi **Collection** có thể chứa nhiều tài liệu.

- **Document**: Một tài liệu chứa dữ liệu dưới dạng cặp khóa-giá trị (**key-value**). Mỗi **Document** nằm trong một **Collection**.

Cài đặt: Để sử dụng ta cần cài đặt thư viện cho dự án Flutter bằng câu lệnh:

```
flutter pub add cloud_firestore
```

Các cú pháp thường được sử dụng:

- **Đọc dữ liệu**

```
Firestore firestore = FirebaseFirestore.instance;  
// Tạo tham chiếu đến collection "users"  
CollectionReference users = firestore.collection('users');  
// Đọc tất cả các document trong collection  
users.get().then((QuerySnapshot querySnapshot) {  
  querySnapshot.docs.forEach((doc) {  
    print(doc["name"]);  
  });  
});
```

Trong đó:

- **CollectionReference** là tham chiếu đến một tập hợp các **document** trong Firestore.
- **get()** được sử dụng để lấy tất cả các **document** trong **collection** một lần.

- **Ghi dữ liệu**

```
CollectionReference users = FirebaseFirestore.instance.collection('users');  
// Thêm một document vào collection  
users.add({  
  "name": "Phú Khuyên",  
  "age": 26  
});
```

Hàm **add()** được sử dụng để thêm một **document** mới với một ID tự động tạo vào **collection**.

- **Xoá dữ liệu**

```
CollectionReference users = FirebaseFirestore.instance.collection('users');  
// Xoá document với ID chỉ định  
users.doc('documentID').delete();
```

Hàm **delete()** được sử dụng để xoá **document** với ID chỉ định.

6.5.4 FlutterFire UI

FlutterFire UI cung cấp nhiều công cụ mạnh mẽ và tiện lợi để tích hợp giao diện người dùng với Firebase Realtime Database và Firestore. Sử dụng các widget như FirebaseDatabaseListView và FirestoreListView, ta có thể dễ dàng xây dựng các ứng dụng với chức năng cuộn vô hạn và xử lý các trường hợp tải và lỗi một cách hiệu quả.

Cài đặt thư viện bằng lệnh sau:

```
flutter pub add flutterfire_ui
```

- **Ví dụ minh họa với FlutterFire UI Database:**

```
import 'package:flutterfire_ui/database.dart';
FirebaseDatabaseListView(
  // Truy vấn đến Realtime Database với danh sách người dùng, sắp xếp theo tên
  query: FirebaseDatabase.instance.ref('users').orderByChild('name'),
  // Thiết lập kích thước trang (phân trang)
  pageSize: 20,
  // Hàm xây dựng các mục
  itemBuilder: (context, snapshot) {
    Map<String, dynamic> user = snapshot.value as Map<String, dynamic>;
    return ListTile(
      title: Text('User name is ${user['name']}'),
    );
  },
  // Hiển thị chỉ báo tải khi dữ liệu đang được tải
  loadingBuilder: (context) => CircularProgressIndicator(),
  // Hiển thị lỗi nếu có
  errorBuilder: (context, error, stackTrace) => Text('Error: $error'),
);
```

- **Ví dụ minh họa với FlutterFire UI Database:**

```
import 'package:flutterfire_ui/firestore.dart';
FirestoreListView<Map<String, dynamic>>(
  // Truy vấn đến Firestore với danh sách người dùng, sắp xếp theo tên
  query: FirebaseFirestore.instance.collection('users').orderBy('name'),
  // Thiết lập kích thước trang (phân trang)
  pageSize: 20,
  // Hàm xây dựng các mục
```

```
itemBuilder: (context, snapshot) {  
  Map<String, dynamic> user = snapshot.data();  
  return ListTile(  
    title: Text('User name is ${user['name']}'),  
  );  
},  
// Hiển thị chỉ báo tải khi dữ liệu đang được tải  
loadingBuilder: (context) => CircularProgressIndicator(),  
// Hiển thị lỗi nếu có  
errorBuilder: (context, error, stackTrace) => Text('Error: $error'),  
);
```

Trong các ví dụ trên:

- **FirebaseDatabaseListView/FirestoreListView**: Widget hiển thị danh sách dữ liệu từ Realtime Database/Firestore với tính năng cuộn vô hạn.
- **query**: Truy vấn đến Realtime Database/Firestore, ở đây là danh sách người dùng sắp xếp theo tên.
- **pageSize**: Thiết lập số lượng mục được tải mỗi lần (phân trang).
- **itemBuilder**: Hàm xây dựng các mục trong danh sách.
- **loadingBuilder**: Widget hiển thị khi dữ liệu đang được tải.
- **errorBuilder**: Widget hiển thị khi có lỗi xảy ra

TÓM TẮT

Bài học này đã cung cấp các kiến thức cần thiết về việc làm việc với dữ liệu trong Flutter, bao gồm cách xử lý JSON, làm việc với REST APIs, lưu trữ dữ liệu đơn giản bằng Shared Preferences, quản lý cơ sở dữ liệu SQL bằng SQLite và sử dụng các dịch vụ Firebase như Realtime Database và Firestore. Đồng thời, bài học cũng hướng dẫn cách sử dụng FlutterFire UI để hiển thị dữ liệu từ Firebase một cách dễ dàng và hiệu quả.

BÀI TẬP

Câu 1: Tạo một ứng dụng Flutter kết nối với một REST API để lấy danh sách các bài viết từ một trang web và hiển thị danh sách này trên giao diện người dùng. Bên cạnh đó có thể tạo, sửa, xoá bài viết.

Câu 2: Viết một ứng dụng Flutter sử dụng Shared Preferences để lưu trữ và quản lý trạng thái đăng nhập của người dùng. Khi người dùng đăng nhập thành công, lưu thông tin đăng nhập và chuyển hướng người dùng đến màn hình chính. Khi mở lại ứng dụng, nếu người dùng đã đăng nhập, tự động chuyển hướng đến màn hình chính.

Câu 3: Viết một ứng dụng Flutter sử dụng SQLite để quản lý danh sách công việc cần làm (to-do list). Cho phép người dùng thêm, sửa, xóa và đánh dấu công việc là đã hoàn thành.

Câu 4: Tạo một ứng dụng Flutter sử dụng Firebase Realtime Database để lưu trữ và đồng bộ dữ liệu giữa các thiết bị. Ứng dụng sẽ là một trò chuyện đơn giản, cho phép người dùng gửi và nhận tin nhắn theo thời gian thực.

Câu 5: Viết một ứng dụng Flutter sử dụng Firestore để quản lý danh sách các sản phẩm trong một cửa hàng. Cho phép người dùng thêm, sửa, xóa và tìm kiếm sản phẩm. Dữ liệu phải được đồng bộ và cập nhật theo thời gian thực giữa các thiết bị.

BÀI 7: XỬ LÝ TRẠNG THÁI VÀ LOGIC ỨNG DỤNG

Trong lập trình ứng dụng hiện đại, đặc biệt là với các ứng dụng di động, việc quản lý trạng thái và logic ứng dụng đóng vai trò vô cùng quan trọng. Khi ứng dụng phát triển và trở nên phức tạp, việc quản lý trạng thái hiệu quả sẽ giúp chúng ta duy trì tính nhất quán và khả năng bảo trì của mã nguồn. Học xong bài này, người học cần nắm được các nội dung sau:

- Kiến thức cơ bản về quản lý trạng thái trong Flutter.
- Hiểu được nguyên lý hoạt động của các thư viện quản lý trạng thái phổ biến: *Provider*, *BLoC* và *Get*.
- Biết cách sử dụng các thư viện này để quản lý trạng thái trong ứng dụng Flutter.
- Áp dụng kiến thức vào các ví dụ minh họa cụ thể.

7.1 GIỚI THIỆU TỔNG QUAN

Trong bất kỳ ứng dụng di động nào, việc quản lý trạng thái (state) và logic của ứng dụng là một phần không thể thiếu.

Trạng thái của ứng dụng bao gồm tất cả các dữ liệu mà ứng dụng cần để hoạt động. Ví dụ như thông tin người dùng, dữ liệu hiển thị trên màn hình hay trạng thái của các nút bấm...

Logic ứng dụng là cách mà dữ liệu này được xử lý và phản ứng lại các sự kiện như người dùng nhấn nút hoặc dữ liệu từ máy chủ được cập nhật...

- **Vì sao cần xử lý trạng thái và logic ứng dụng?**

- Duy trì tính nhất quán của dữ liệu: Giúp đảm bảo dữ liệu hiển thị trên giao diện luôn đồng bộ với trạng thái thật của ứng dụng.

- Quản lý luồng dữ liệu: Cho phép xử lý dữ liệu từ nhiều nguồn khác nhau, chẳng hạn như từ API, cơ sở dữ liệu hoặc các sự kiện người dùng.
- Tái sử dụng và tổ chức code: Giúp code dễ dàng bảo trì, mở rộng và tái sử dụng trong nhiều phần khác nhau của ứng dụng.

- **Khuyết điểm của `setState` trong các dự án lớn**

Trong các ứng dụng nhỏ, việc sử dụng `setState` để cập nhật giao diện có thể là đủ. Tuy nhiên, trong các dự án lớn, việc quản lý trạng thái bằng `setState` có thể gặp một số khuyết điểm sau:

- Phức tạp và khó bảo trì: Khi ứng dụng trở nên phức tạp, việc sử dụng `setState` có thể dẫn đến việc khó khăn trong việc theo dõi và quản lý trạng thái của ứng dụng.
- Khả năng mở rộng kém: `setState` không phù hợp cho việc quản lý trạng thái phức tạp và nhiều thành phần của ứng dụng. Khi số lượng các biến trạng thái và các thành phần giao diện tăng lên, việc sử dụng `setState` sẽ trở nên khó kiểm soát và dễ gây ra lỗi.
- Hiệu năng kém: Việc sử dụng `setState` có thể dẫn đến việc cập nhật lại toàn bộ giao diện, ngay cả khi chỉ có một phần nhỏ của giao diện thay đổi. Điều này làm giảm hiệu năng của ứng dụng, đặc biệt là khi ứng dụng có nhiều thành phần hoặc hoạt động phức tạp.
- Khó khăn trong việc chia sẻ trạng thái: `setState` chỉ có thể cập nhật trạng thái trong phạm vi của một widget. Điều này làm cho việc chia sẻ và quản lý trạng thái giữa các thành phần khác nhau của ứng dụng trở nên khó khăn.

Giả sử ta có một ứng dụng đơn giản với một bộ đếm và một danh sách các mục. Khi nhấn nút tăng bộ đếm, giá trị của bộ đếm sẽ tăng lên và giao diện sẽ được cập nhật lại.

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CounterScreen(),
    );
  }
}

class CounterScreen extends StatefulWidget {
  @override
  _CounterScreenState createState() => _CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  int _counter = 0; // Biến trạng thái đếm
  List<String> _items = List.generate(100, (index) => 'Item $index'); // Danh sách các mục

  // Hàm tăng bộ đếm
  void _incrementCounter() {
    setState(() {
      _counter++; // Tăng giá trị bộ đếm
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('SetState Example')),
      body: Column(
```

```
children: [
  // Hiển thị giá trị bộ đếm
  Text('Counter: $_counter', style: TextStyle(fontSize: 24)),
  Expanded(
    // Danh sách các mục
    child: ListView.builder(
      itemCount: _items.length,
      itemBuilder: (context, index) {
        return ListTile(
          title: Text(_items[index]),
        );
      },
    ),
  ),
],
),
floatingActionButton: FloatingActionButton(
  onPressed: _incrementCounter, // Gọi hàm tăng bộ đếm khi nhấn nút
  child: Icon(Icons.add),
),
);
}
```

Xét ví dụ trên ta thấy:

- Mỗi khi `_counter` thay đổi, toàn bộ widget `CounterScreen` được xây dựng lại. Điều này bao gồm cả danh sách `_items`, mặc dù danh sách này không thay đổi. Điều này làm lãng phí tài nguyên và giảm hiệu năng của ứng dụng, đặc biệt khi danh sách rất dài.
- Bên cạnh đó, khi ứng dụng trở nên phức tạp hơn với nhiều biến trạng thái, việc theo dõi và quản lý tất cả các `setState` này sẽ trở nên rất khó khăn đồng thời cũng làm tăng độ phức tạp của mã nguồn.

- Ngoài ra, nếu ta có nhiều màn hình khác nhau cần truy cập và cập nhật giá trị `_counter`, ta sẽ phải triển khai nhiều giải pháp phức tạp để chia sẻ trạng thái này.

- **Các thư viện quản lý trạng thái ra đời**

Để giải quyết các vấn đề trên, nhiều thư viện quản lý trạng thái đã ra đời, cung cấp các cách tiếp cận khác nhau để quản lý trạng thái và logic của ứng dụng một cách hiệu quả và dễ dàng hơn. Ví dụ như Provider, BLoC, Get, Redux...

7.2 PROVIDER

7.2.1 Giới thiệu

Provider là một trong những thư viện quản lý trạng thái phổ biến và dễ sử dụng nhất trong Flutter. Thư viện này giúp tách biệt rõ ràng giữa giao diện và logic ứng dụng, giúp quản lý trạng thái một cách hiệu quả và dễ bảo trì. Provider sử dụng **ChangeNotifier** để thông báo cho các widget khi có sự thay đổi trạng thái. Nguyên lý hoạt động của Provider như sau:

- **Provider** cung cấp các class như **ChangeNotifier** và **ChangeNotifierProvider** để quản lý trạng thái.
- **ChangeNotifier** là một class đơn giản trong **Flutter SDK** cung cấp thông báo thay đổi cho các **listeners**.
- **ChangeNotifierProvider** là widget cung cấp **instance** của **ChangeNotifier** cho các widget con.

7.2.2 Cách sử dụng

Bước 1: Thêm thư viện vào dự án Flutter: bằng `pubspec.yaml` hoặc dùng lệnh

```
flutter pub add provider
```

Bước 2: Tạo một lớp ChangeNotifier

```
import 'package:flutter/material.dart';

class Counter with ChangeNotifier {
  int _count = 0;
```

```
int get count => _count;

void increment() {
  _count++;
  notifyListeners();
}
}
```

Lớp **Counter** mở rộng từ **ChangeNotifier**, cho phép thông báo cho các widget lắng nghe khi có thay đổi.

Bước 3: Sử dụng **ChangeNotifierProvider** trong widget gốc (main)

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'counter.dart'; // Import lớp Counter

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => Counter(),
      child: MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CounterScreen(),
    );
  }
}
```

ChangeNotifierProvider: Cung cấp một instance của **Counter** cho cây widget bên dưới.

Bước 4: Sử dụng **Consumer** để truy cập và hiển thị trạng thái

```
class CounterScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text('Provider Example')),  
      body: Center(  
        child: Consumer<Counter>(  
          builder: (context, counter, child) {  
            return Text('Counter: ${counter.count}', style: TextStyle(fontSize:  
24));  
          },  
        ),  
      ),  
      floatingActionButton: FloatingActionButton(  
        onPressed: () => context.read<Counter>().increment(),  
        child: Icon(Icons.add),  
      ),  
    );  
  }  
}
```

Consumer: Widget này lắng nghe các thay đổi từ **Counter** và cập nhật giao diện khi có thay đổi.

7.3 BLoC

7.3.1 Giới thiệu

BLoC (Business Logic Component) là một mẫu kiến trúc giúp tách biệt rõ ràng giữa giao diện người dùng và logic ứng dụng. Nó sử dụng các **Stream** và **Sink** để quản lý trạng thái và cung cấp các sự kiện từ **UI** đến logic nghiệp vụ. Nguyên lý hoạt động của **BLoC** như sau:

- Sử dụng **Stream** để truyền tải các trạng thái mới.
- Sử dụng **Sink** để gửi các sự kiện từ **UI** đến **BLoC**.
- **BLoC** xử lý các sự kiện và phát ra các trạng thái mới qua **Stream**.

7.3.2 Cách sử dụng

Bước 1: Thêm thư viện vào dự án Flutter: bằng `pubspec.yaml` hoặc dùng lệnh

```
flutter pub add flutter_bloc
```

Bước 2: Tạo một Bloc và các Event liên quan

```
import 'package:bloc/bloc.dart';

// Định nghĩa các sự kiện
abstract class CounterEvent {}

class Increment extends CounterEvent {}

// Tạo Bloc để quản lý trạng thái
class CounterBloc extends Bloc<CounterEvent, int> {
  CounterBloc() : super(0);

  @override
  Stream<int> mapEventToState(CounterEvent event) async* {
    if (event is Increment) {
      yield state + 1;
    }
  }
}
```

Trong đó:

- **CounterEvent** là lớp trừu tượng để định nghĩa các sự kiện cho **Bloc**.
- **Increment** là một sự kiện cụ thể, kế thừa từ **CounterEvent**.
- **CounterBloc** là **Bloc** để quản lý trạng thái bộ đếm, xử lý các sự kiện **CounterEvent** và phát ra các trạng thái kiểu `int`.
- **mapEventToState** là phương thức xử lý sự kiện và phát ra trạng thái mới. Nếu sự kiện là **Increment**, giá trị bộ đếm sẽ được tăng lên 1.

Bước 3: Sử dụng BlocProvider trong widget gốc:

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
```

```
import 'counter_bloc.dart'; // Import CounterBloc

void main() {
  runApp(
    BlocProvider(
      create: (context) => CounterBloc(),
      child: MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CounterScreen(),
    );
  }
}
```

BlocProvider: Cung cấp một instance của **CounterBloc** cho cây widget bên dưới.

Bước 4: Sử dụng **BlocBuilder** để truy cập và hiển thị trạng thái:

```
class CounterScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('BLoC Example')),
      body: Center(
        child: BlocBuilder<CounterBloc, int>(
          builder: (context, count) {
            return Text('Counter: $count', style: TextStyle(fontSize: 24));
          },
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => context.read<CounterBloc>().add(Increment()),
        child: Icon(Icons.add),
      ),
    );
  }
}
```

```
}
```

BlocBuilder: Widget này lắng nghe các thay đổi từ **CounterBloc** và cập nhật giao diện khi có thay đổi.

7.4 GET

7.4.1 Giới thiệu

Get là một thư viện quản lý trạng thái mạnh mẽ và linh hoạt, cung cấp các giải pháp quản lý trạng thái, điều hướng và cơ chế DI (tiêm phụ thuộc) một cách dễ dàng và hiệu quả. Nguyên lý hoạt động của **Get** bao gồm:

- Sử dụng các **Controller** để quản lý trạng thái.
- Các biến trạng thái được khai báo với kiểu **obs** để có thể quan sát và phản ứng khi có thay đổi.
- Sử dụng **Get.put** để khởi tạo và cung cấp các **Controller**.
- Sử dụng **Obx** để lắng nghe và cập nhật giao diện khi có thay đổi.

7.4.2 Cách sử dụng

Bước 1: Thêm thư viện vào dự án Flutter: bằng `pubspec.yaml` hoặc dùng lệnh

```
flutter pub add get
```

Bước 2: Tạo một Controller mở rộng từ `GetxController`:

```
import 'package:get/get.dart';

class CounterController extends GetxController {
  var counter = 0.obs;

  void increment() {
    counter++;
  }
}
```

Trong đó:

- **GetxController**: Lớp **CounterController** mở rộng từ **GetxController**, cho phép quản lý trạng thái.
- **obs**: Biến counter được khai báo với kiểu **obs** để có thể quan sát và phản ứng khi có thay đổi.

Bước 3: Sử dụng Controller trong widget:

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'counter_controller.dart'; // Import CounterController

class CounterScreen extends StatelessWidget {
  final CounterController _controller = Get.put(CounterController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Get Example')),
      body: Center(
        child: Obx(() {
          return Text('Counter: ${_controller.counter}', style:
TextStyle(fontSize: 24));
        }),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _controller.increment,
        child: Icon(Icons.add),
      ),
    );
  }
}
```

Trong đó:

- **Get.put**: Hàm này cung cấp một **instance** của **CounterController** và lưu trữ nó trong bộ nhớ.
- **Obx**: Widget này lắng nghe và cập nhật giao diện khi biến **counter** thay đổi.

TÓM TẮT

Trong bài học này, chúng ta đã tìm hiểu về ba thư viện quản lý trạng thái phổ biến trong Flutter: **Provider**, **BLoC** (Business Logic Component) và **Get**. Mỗi thư viện đều có những đặc điểm và cách tiếp cận riêng để quản lý trạng thái và logic ứng dụng. **Provider** cung cấp một cách đơn giản và dễ hiểu để quản lý trạng thái, thích hợp cho các ứng dụng nhỏ và vừa. **BLoC** mang lại cấu trúc rõ ràng và khả năng kiểm thử tốt, phù hợp cho các ứng dụng lớn và phức tạp. **Get** cung cấp một giải pháp linh hoạt với cú pháp đơn giản, hỗ trợ nhiều tính năng như quản lý trạng thái, điều hướng và DI.

BÀI TẬP

Câu 1: Sử dụng **Provider** để quản lý trạng thái:

- Tạo một ứng dụng đếm số sử dụng **Provider**. Ứng dụng có hai màn hình: màn hình chính hiển thị giá trị bộ đếm và nút tăng giá trị, màn hình thứ hai hiển thị lịch sử các giá trị đã được tăng.
- Các bước thực hiện:
 - + Tạo một class **Counter** với **ChangeNotifier**.
 - + Sử dụng **ChangeNotifierProvider** để cung cấp **Counter** cho cây widget.
 - + Sử dụng **Consumer** để lắng nghe và hiển thị giá trị bộ đếm.

Câu 2: Sử dụng **BLoC** để quản lý trạng thái

- Tạo một ứng dụng quản lý công việc (**to-do list**) sử dụng **BLoC**. Ứng dụng có thể thêm, xóa và đánh dấu công việc đã hoàn thành.
- Các bước thực hiện:
 - + Định nghĩa các sự kiện và trạng thái cho **Bloc**.
 - + Tạo class **TodoBloc** để quản lý trạng thái công việc.

+ Sử dụng **BlocProvider** để cung cấp **TodoBloc** cho cây widget.

+ Sử dụng **BlocBuilder** để hiển thị danh sách công việc và xử lý các sự kiện từ người dùng.

Câu 3: Sử dụng **GetX** để quản lý trạng thái :

- Tạo một ứng dụng quản lý người dùng sử dụng **GetX**. Ứng dụng có thể thêm, xóa và cập nhật thông tin người dùng.
- Các bước thực hiện:
 - + Tạo một class **UserController** với **GetxController** để quản lý trạng thái người dùng.
 - + Sử dụng **GetMaterialApp** để khởi tạo ứng dụng.
 - + Sử dụng **GetBuilder** để hiển thị danh sách người dùng và xử lý các sự kiện từ người dùng.

BÀI 8: XUẤT BẢN VÀ PHÂN PHỐI ỨNG DỤNG

Trong quá trình phát triển ứng dụng di động, việc xuất bản và phân phối ứng dụng là một bước quan trọng để sản phẩm của chúng ta có thể tiếp cận được với người dùng cuối. Bài học này sẽ hướng dẫn chi tiết quy trình xuất bản và phân phối ứng dụng Flutter trên hai nền tảng phổ biến là Android và iOS. Học xong bài này, người học cần nắm được các nội dung sau:

- Kiến thức cơ bản về quy trình xuất bản ứng dụng.
- Cách quản lý phiên bản ứng dụng trong Flutter.
- Các bước chi tiết để build và release ứng dụng trên Google Play Store và App Store.
- Hiểu rõ về công cụ Google Play Console và App Store Connect.
- Biết cách chuẩn bị và tối ưu ứng dụng trước khi xuất bản.

8.1 QUY TRÌNH XUẤT BẢN

Quy trình xuất bản ứng dụng bao gồm nhiều bước để chuẩn bị và đưa ứng dụng của ta lên các cửa hàng ứng dụng như Google Play Store và Apple App Store. Trong phần này, chúng ta sẽ tìm hiểu chi tiết về cách build và release ứng dụng Flutter cho Android và iOS.

8.1.1 Quản lý phiên bản ứng dụng

Quản lý phiên bản ứng dụng là một phần quan trọng trong quy trình phát triển phần mềm. Nó giúp người dùng và nhà phát triển theo dõi các thay đổi, cập nhật và cải tiến trong mỗi lần phát hành ứng dụng. Trong Flutter, việc quản lý phiên bản ứng dụng được thực hiện thông qua file `pubspec.yaml`.

- **Cấu trúc phiên bản trong `pubspec.yaml`**

Trong file pubspec.yaml, phiên bản ứng dụng được xác định bằng một chuỗi phiên bản (ví dụ 1.0.0) và số build (+1).

```
version: 1.0.0+1
```

Cụ thể:

- **Chuỗi phiên bản (Ví dụ: 1.0.0):** Bao gồm ba số được ngăn cách bằng dấu chấm: major.minor.patch.

- + Major version (Số phiên bản chính): Thay đổi khi có các cải tiến lớn, thay đổi cấu trúc hoặc thay đổi có thể không tương thích với phiên bản trước.

- + Minor version (Số phiên bản phụ): Thay đổi khi có các tính năng mới nhưng vẫn giữ tính tương thích với các phiên bản trước.

- + Patch version (Số bản vá): Thay đổi khi có các bản sửa lỗi hoặc cải tiến nhỏ, không ảnh hưởng đến các tính năng chính.

- **Số build (Ví dụ: +1):** Số build là một số nguyên duy nhất đại diện cho mỗi lần build của ứng dụng. Nó giúp theo dõi các build khác nhau và đảm bảo mỗi build có một số nhận dạng duy nhất.

- **Cập nhật phiên bản trong pubspec.yaml**

Để cập nhật phiên bản ứng dụng trong file pubspec.yaml, ta chỉ cần thay đổi giá trị của version theo cú pháp major.minor.patch+build. Ví dụ:

- Từ phiên bản 1.0.0+1:

```
version: 1.0.0+1
```

- Cập nhật phiên bản phụ (minor version) khi thêm tính năng mới:

```
version: 1.0.0+2
```

- Cập nhật phiên bản chính (major version) khi có thay đổi lớn:

```
version: 2.0.0+3
```

```
version: 2.0.0+3
```

- Cập nhật bản vá (patch version) khi sửa lỗi:

```
version: 2.0.1+4
```


8.1.2 Xuất bản Android

Khi đã sẵn sàng chuẩn bị phiên bản phát hành của ứng dụng, ta có thể làm theo các bước dưới đây để **build** và **release** ứng dụng Flutter cho Android. (khuyến khích xem chi tiết tại website của Flutter)

Bước 1: Thêm biểu tượng launcher

– Sử dụng `package flutter_launcher_icons` hoặc làm thủ công theo các bước sau:

+ Đặt các file biểu tượng trong thư mục `[project]/android/app/src/main/res/`: Tạo các thư mục con như `mipmap-mdpi`, `mipmap-hdpi`, `mipmap-xhdpi`, `mipmap-xxhdpi` và `mipmap-xxxhdpi` trong thư mục `res` và đặt các file biểu tượng có kích thước tương ứng vào đó.

+ Cập nhật thuộc tính `android:icon` trong file `AndroidManifest.xml`: Mở file `AndroidManifest.xml` và thay đổi thuộc tính `android:icon` trong thẻ `<application>` để trỏ đến biểu tượng mới.

```
<application
  android:icon="@mipmap/ic_launcher"
  ...>
</application>
```

Chú thích: Thay `@mipmap/ic_launcher` bằng đường dẫn đến biểu tượng ta đã đặt trong thư mục `res`.

Bước 2: Ký ứng dụng

– Tạo **keystore**

+ Trên macOS hoặc Linux:

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA -keysize 2048 -
validity 10000 -alias upload
```

+ Trên Windows:

```
keytool -genkey -v -keystore %userprofile%\upload-keystore.jks -storetype JKS -
keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

Chú thích: Lệnh này tạo **keystore** và lưu nó trong thư mục người dùng. Ta sẽ cần cung cấp mật khẩu và thông tin chi tiết khác trong quá trình này.

- Tham chiếu keystore từ ứng dụng Flutter:

+ Tạo file **key.properties** trong thư mục **[project]/android** và thêm nội dung sau:

```
storePassword=<password-from-previous-step>
keyPassword=<password-from-previous-step>
keyAlias=upload
storeFile=<keystore-file-location>
```

Chú thích: Thay **<password-from-previous-step>** bằng mật khẩu ta đã đặt cho **keystore** và **<keystore-file-location>** bằng đường dẫn đến file **keystore**.

- Cấu hình ký trong **Gradle**: Mở file **android/app/build.gradle** và thêm cấu hình ký:

```
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

android {
    signingConfigs {
        release {
            keyAlias keystoreProperties['keyAlias']
            keyPassword keystoreProperties['keyPassword']
            storeFile keystoreProperties['storeFile'] ?
file(keystoreProperties['storeFile']) : null
            storePassword keystoreProperties['storePassword']
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release
        }
    }
}
```

Chú thích: Đảm bảo ta cần tải các thuộc tính **keystore** từ file **key.properties** và cấu hình ký ứng dụng khi **build** ở chế độ **release**.

Bước 4: Xem lại cấu hình build và manifest

- Kiểm tra file **AndroidManifest.xml** để đảm bảo các quyền và thuộc tính được thiết lập chính xác (như quyền **INTERNET**) được khai báo trong file này.
- Kiểm tra file **build.gradle** để đảm bảo các giá trị như **applicationId**, **minSdkVersion**, **targetSdkVersion**, **versionCode** và **versionName** được thiết lập chính xác.

Bước 5: Xuất bản

Với Android, ta có thể có 2 cách xuất bản dự án ra 2 dạng (**APK** hoặc **App Bundle**) thông qua các câu lệnh sau (chạy **terminal** ngay tại dự án Flutter cần xuất bản)

```
flutter build appbundle  
hoặc  
flutter build apk
```

8.1.3 Xuất bản iOS

Để xây dựng và xuất bản ứng dụng Flutter cho iOS, ta làm theo các bước dưới đây:

Bước 1: Xem lại cấu hình dự án trên Xcode

- Mở **workspace** trên Xcode của dự án Flutter tại thư mục **ios/Runner.xcworkspace**
- Xem lại các cài đặt quan trọng trong mục **Runner** của **Xcode**:
 - + **Display Name**: Tên hiển thị của ứng dụng.
 - + **Bundle Identifier**: ID ứng dụng mà ta đã đăng ký trên **App Store Connect**.
 - + **Automatically manage signing**: Để **Xcode** tự động quản lý việc ký ứng dụng và **provisioning**.
 - + **Team**: Chọn team liên quan đến tài khoản Apple Developer.
 - + **iOS Deployment Target**: Phiên bản iOS tối thiểu mà ứng dụng hỗ trợ. Flutter hỗ trợ iOS 12 trở lên.

Bước 2: Thêm biểu tượng ứng dụng

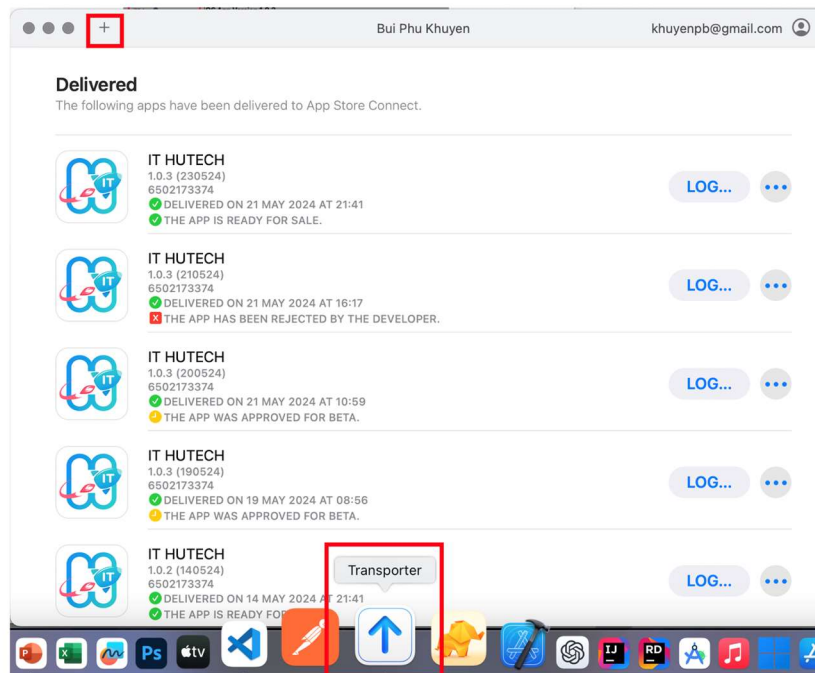
Tương tự như Android, ta có thể sử dụng thư viện **flutter_launcher_icons** hoặc làm thủ công ở **Assets.xcassets** trong Xcode.

Bước 3: Xuất bản

Tại dự án Flutter cần xuất bản, ta sử dụng câu lệnh sau để tiến hành xuất bản cho hệ điều hành iOS.

```
flutter build ipa
```

Sau đó, ta sử dụng phần mềm **Transporter** để tải lên file **IPA** của ứng dụng lên **App Store Connect**.



8.2 QUY TRÌNH PHÂN PHỐI

Xuất bản và phân phối ứng dụng là bước cuối cùng nhưng vô cùng quan trọng trong quá trình phát triển ứng dụng di động. Nó giúp ứng dụng của chúng ta tiếp cận được với người dùng cuối thông qua các cửa hàng ứng dụng như Google Play Store và Apple App Store.

8.2.1 Google Play Console

Google Play Console là một nền tảng quản lý ứng dụng cho phép các nhà phát triển Android quản lý và phân phối ứng dụng của họ trên Google Play Store. Dưới đây là các bước chính để xuất bản ứng dụng trên Google Play Store:

Bước 1: Tạo tài khoản Google Play Developer

- Đăng ký tài khoản nhà phát triển Google Play tại <https://play.google.com/console>
- Thanh toán phí đăng ký một lần (\$25).

Bước 2: Tạo ứng dụng mới trên Google Play Console

- Truy cập vào trang chủ Google Play Console
- Chọn “Tạo ứng dụng” để tiến hành tạo ứng dụng mới

The screenshot shows the Google Play Console interface. On the left is a sidebar menu with options like 'Trang chủ', 'Hộp thư đến', 'Trạng thái tuân thủ chính sách', 'Người dùng và quyền', 'Quản lý đơn đặt hàng', 'Bài đánh giá', 'Số liệu thống kê', 'Tài chính', 'Chi tiết tài khoản', 'Trang dành cho nhà phát triển', 'Tài khoản nhà phát triển liên kết', 'Nhật ký hoạt động', and 'Cài đặt'. The main area displays account details for 'Bui Phu Khuyen' and a table of applications.

Người dùng đã cài đặt ứng dụng	Chuyển đổi người dùng	Điểm xếp hạng trên Google Play	Tổng doanh thu
206 0,0% so với 30 ngày trước đó	190 0,0% so với 30 ngày trước đó	5,000 ★	-

Ứng dụng	Người dùng đã cài đặt ứng dụng	Trạng thái ứng dụng	Cập nhật trạng thái	Cập nhật lần cuối
Dark Dungeons - Roguelikes com.DuckStudio.DarkDungeon	0	Phát hành công khai		5 thg 9, 2023
IT HUTECH com.ithutech	206	Phát hành công khai		21 thg 5, 2024

- Nhập các thông tin chi tiết cho ứng dụng: Tên ứng dụng, Ngôn ngữ mặc định, Loại (ứng dụng hay trò chơi), có phí hay miễn phí...

Tạo ứng dụng

Chi tiết ứng dụng

Tên ứng dụng Đây là tên của ứng dụng khi xuất hiện trên Google Play 9 / 30

Ngôn ngữ mặc định

Ứng dụng hay trò chơi Bạn có thể thay đổi tùy chọn này sau trong mục tùy chọn cài đặt của Cửa hàng

☒ Ứng dụng
☐ Trò chơi

Miễn phí hay có tính phí Bạn có thể chỉnh sửa tùy chọn này sau trên trang Ứng dụng phải trả phí

☒ Miễn phí
☐ Phải trả tiền

[Huỷ](#) [Tạo ứng dụng](#)

Bước 3: Cung cấp thông tin chi tiết về ứng dụng

- Trên Google Play Console, chọn ứng dụng cần tạo bản phát hành.
- Vào mục Phát triển → Sự hiện diện trong Cửa hàng Play
- Nhập thông tin về ứng dụng như tên, mô tả, biểu tượng, ảnh chụp màn hình và video giới thiệu (nếu có).
- Đảm bảo các thông tin này tuân thủ các nguyên tắc của Google Play.

Google Play Console

IT HUTECH

- Phạm vi tiếp cận và thiết bị
- Trình khám phá gói ứng dụng
- Tính toán vận của ứng dụng
- Thiết lập
 - Ký ứng dụng
 - Chia sẻ ứng dụng nội bộ
 - Cài đặt nâng cao
- Phát triển**
 - Sự hiện diện trong Cửa hàng Play**
 - Trang thông tin chính trên Cửa hàng Play
 - Trang thông tin tùy chỉnh trên Cửa hàng Play
 - Thử nghiệm trang thông tin trên Cửa hàng Play

Trang thông tin chính trên Cửa hàng Play

Mặc định – Tiếng Việt – vi Quản lý bản dịch

1243 / 4000

Đồ họa

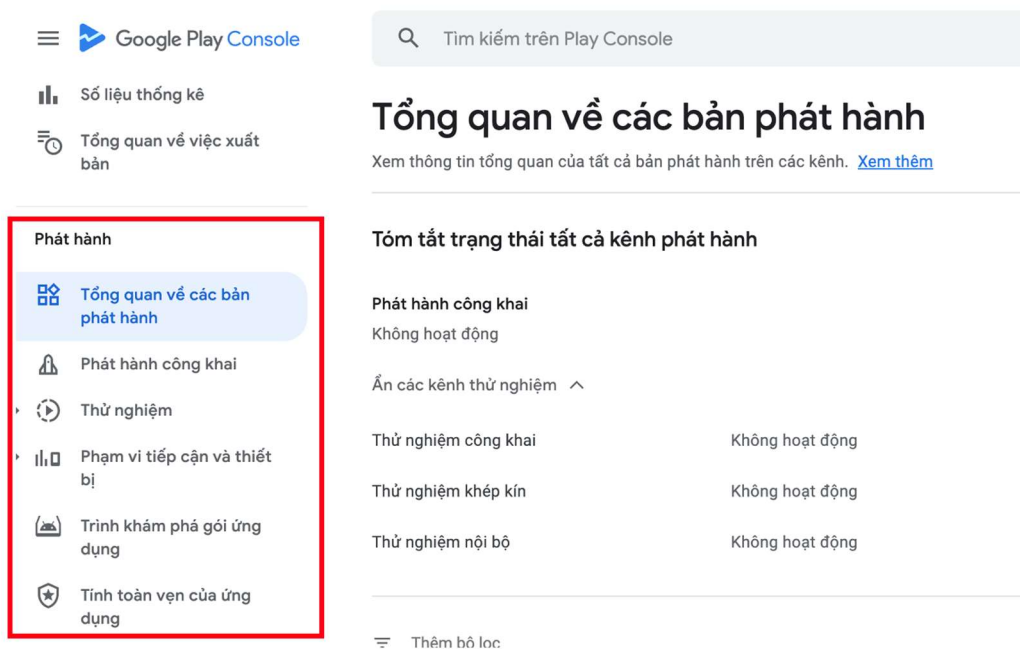
Hãy quản lý biểu tượng ứng dụng, ảnh chụp màn hình và video để quảng bá ứng dụng của bạn trên Google Play. Vui lòng xem lại [nguyên tắc về nội dung](#) trước khi tải nội dung đồ họa mới lên. Nếu bạn thêm bản dịch cho trang thông tin của mình trên Cửa hàng Play nhưng không bản địa hóa nội dung đồ họa, thì chúng tôi sẽ sử dụng nội dung đồ họa bằng ngôn ngữ mặc định của bạn.

Biểu tượng ứng dụng *

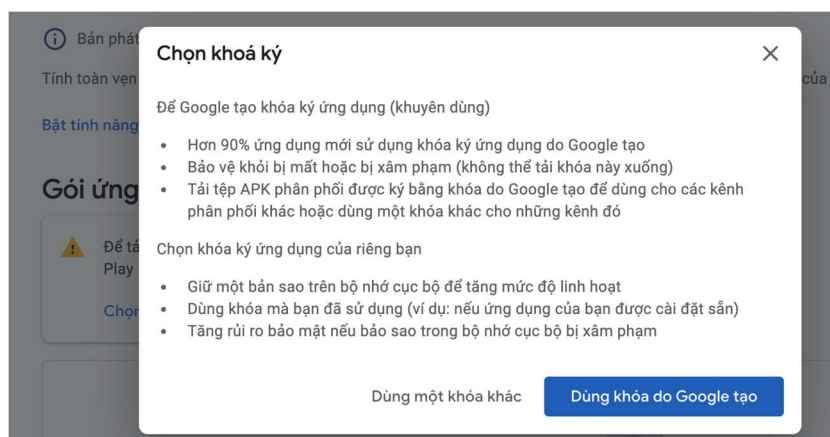
[Thay thế](#)

Bước 4: Phát hành ứng dụng

- Hình thức phát hành: Có nhiều hình thức phát hành trên Google Play, bao gồm: Phát hành công khai, Thử nghiệm (Công khai, Khép kín, Nội bộ và Đăng ký trước). Tùy vào mục đích ta chọn hình thức phát hành phù hợp với các nhân/doanh nghiệp.



- Tạo khoá ký



- Tải lên file APK hoặc AAB của ứng dụng
- Điền các thông tin chi tiết về bản phát hành

Tạo bản phát hành chính thức

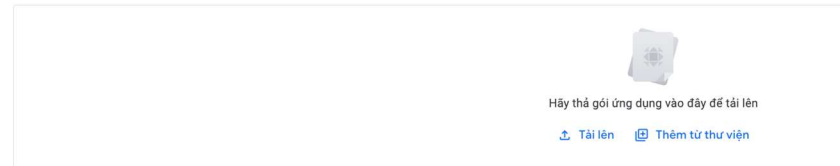
Tính toàn vẹn của ứng dụng

✓ Google Play đã ký các bản phát hành

Tính toàn vẹn và dịch vụ ký của Google Play giúp bạn đảm bảo người dùng trải nghiệm ứng dụng và trò chơi của bạn theo cách bạn muốn

[Bật tính năng bảo vệ tính toàn vẹn](#) [Thay đổi khóa ký](#)

Gói ứng dụng



Chi tiết bản phát hành

Tên bản phát hành *

0 / 50

Đây là tên nhận dạng bản phát hành và sẽ không hiển thị cho người dùng trên Google Play. Chúng tôi đã đề xuất một tên nhận dạng dựa theo gói ứng dụng hoặc APK đầu tiên của bản phát hành, nhưng bạn có thể chỉnh sửa tên đó.

Ghi chú phát hành

[Sao chép từ bản phát hành trước](#)

<vi>
Nhập hoặc dán ghi chú phát hành cho vi tại đây
</vi>

- Kiểm tra lại toàn bộ thông tin trước khi xuất bản.

Bước 5: Xuất bản ứng dụng

- Chọn quốc gia và khu vực mà ta muốn phân phối ứng dụng.
- Thiết lập giá bán (nếu là ứng dụng trả phí).

Bước 6: Đăng tải ứng dụng

- Sau khi kiểm tra và hoàn tất các bước, chọn "Tổng quan về việc xuất bản" để xuất bản ứng dụng.
- Gửi các nội dung thay đổi để được xem xét

Google Play Console

Trang tổng quan

Hộp thư đến

Số liệu thống kê

Tổng quan về việc xuất bản

Phát hành

Tổng quan về các bản phát hành

Phát hành công khai

Thử nghiệm

Thử nghiệm công khai

Tìm kiếm trên Play Console

Tổng quan về việc xuất bản

Các thay đổi đã sẵn sàng để gửi đi xem xét

Gửi 11 nội dung thay đổi để được xem xét

Đã thay đổi mục	Mô tả
Phát hành công khai	
Hutech Checkin	Bắt đầu phát hành đầy đủ
Quốc gia/khu vực	Thêm 2 quốc gia/khu vực: Lào, Việt Nam
Thử nghiệm công khai	
Quốc gia/khu vực	Thêm 1 quốc gia/khu vực: Việt Nam
Thử nghiệm kín - Alpha	

Google sẽ kiểm duyệt và sau khi được chấp thuận, ứng dụng của ta sẽ xuất hiện trên Google Play Store.

8.2.2 App Store Connect - Apple

App Store Connect là công cụ quản lý ứng dụng của Apple, cho phép các nhà phát triển iOS quản lý và phân phối ứng dụng của họ trên App Store. Dưới đây là các bước chính để xuất bản ứng dụng trên App Store:

Bước 1: Tạo tài khoản Apple Developer

- Đăng ký tài khoản nhà phát triển Apple tại Apple Developer.
- Thanh toán phí hàng năm (\$99).

Bước 2: Chuẩn bị ứng dụng

- Đảm bảo rằng ứng dụng của ta đã hoàn thiện và đã được kiểm tra kỹ lưỡng.
- Sử dụng Xcode để tạo file IPA của ứng dụng.

Bước 3: Tạo hồ sơ ứng dụng trên App Store Connect

- Truy cập App Store Connect (<https://appstoreconnect.apple.com>) và đăng nhập bằng tài khoản Apple Developer.

App Store Connect

Phu Khuyen Bui
Bui Phu Khuyen



Apps



App Analytics



Sales and Trends



Payments and
Financial Reports

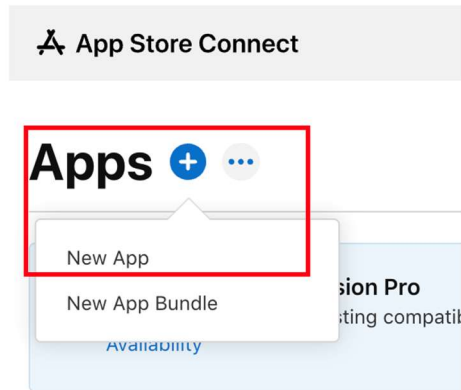


Business



Users and
Access

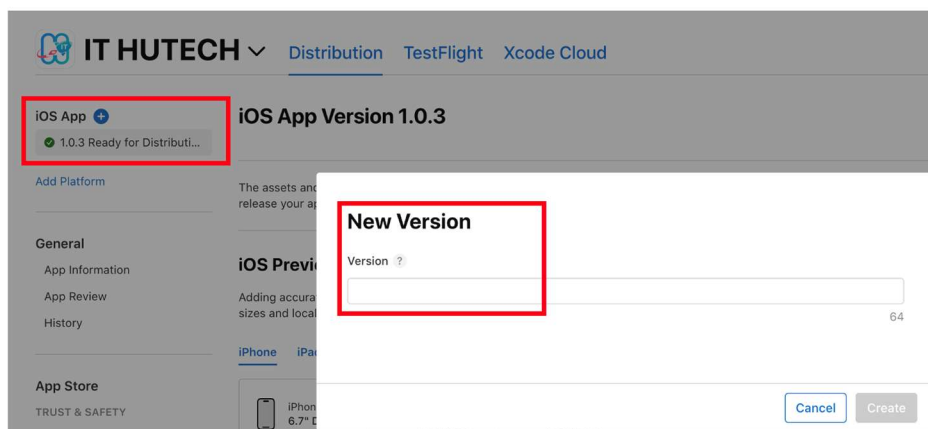
- Chọn "Apps" và nhấp vào dấu cộng (+) → New App để tạo ứng dụng mới.



- Nhập các thông tin về ứng dụng: Platforms (nền tảng), Name (tên ứng dụng), Primary Language (ngôn ngữ sử dụng chính), Bundle ID, SKU và User Access cho ứng dụng.

Bước 4: Cấu hình thông tin cho bản phát hành iOS

- Truy cập vào ứng dụng đã tạo ở bước 3
- Chọn hình thức phát hành: có 2 hình thức phát hành trên iOS: Phát hành chính thức (Distribution), Phát hành thử nghiệm (TestFlight).
- Tạo 1 phiên bản mới bằng cách nhấp vào dấu cộng (+) tại iOS App. Sau đó điền thông tin phiên bản. Lưu ý: Việc đánh số phiên bản phải tuân theo quy ước về phiên bản phần mềm



- Nhập các thông tin chi tiết cho ứng dụng trên phiên bản này: Mô tả, tính năng mới, từ khoá tìm kiếm, ảnh chụp màn hình, cấu hình nâng cao bản phát hành...
- Đảm bảo các thông tin này tuân thủ các nguyên tắc của App Store.

Bước 4: Thiết lập một số các thông tin khác

Ngoài việc cấu hình thông tin bản phát hành mới, chúng ta cần bổ sung một số thông tin về của ứng dụng (ở mục General → App Information) như tên ứng dụng, danh mục, số tuổi sử dụng, chính sách bảo mật... theo yêu cầu của Apple

Bước 5: Phân phối ứng dụng

- Kiểm tra lại toàn bộ thông tin và chọn bản Build đã được gửi lên từ Transporter.
- Chọn "Add for Review/Submit for Review" để gửi ứng dụng cho Apple kiểm duyệt.

Sau khi ứng dụng được kiểm duyệt và chấp thuận, ứng dụng của ta sẽ xuất hiện trên App Store.

TÓM TẮT

Trong bài học này, chúng ta đã tìm hiểu về quy trình xuất bản và phân phối ứng dụng di động trên hai nền tảng Android và iOS. Đối với Android, quy trình bao gồm các bước từ tạo tài khoản nhà phát triển, chuẩn bị ứng dụng, ký ứng dụng, thu nhỏ mã nguồn, đến đăng tải và xuất bản trên Google Play Store. Đối với iOS, quy trình bao gồm việc đăng ký Bundle ID, tạo hồ sơ ứng dụng trên App Store Connect, cấu hình dự án Xcode, tạo biểu tượng và launch image, tạo build archive và upload lên App Store Connect và cuối cùng là phát hành ứng dụng trên App Store. Bài học cung cấp kiến thức chi tiết và cụ thể để người học có thể thực hiện xuất bản ứng dụng một cách dễ dàng và hiệu quả.

BÀI TẬP

Câu 1: Quản lý phiên bản ứng dụng trong Flutter được thực hiện thông qua file nào?

- A. pubspec.yaml
- B. build.gradle

C. AndroidManifest.xml

D. Info.plist

Câu 2: Lệnh nào dùng để tạo bản release của ứng dụng Android dưới dạng APK?

A. flutter build apk --release

B. flutter build appbundle

C. flutter run --release

D. flutter build ios --release

Câu 3: Trong quá trình ký ứng dụng Android, file nào chứa thông tin về keystore?

A. key.properties

B. build.gradle

C. AndroidManifest.xml

D. pubspec.yaml

Câu 4: Công cụ nào được sử dụng để quản lý và phân phối ứng dụng iOS trên App Store?

A. Google Play Console

B. App Store Connect

C. Xcode

D. Transporter

Câu 5: Lệnh nào dùng để tạo bản release của ứng dụng iOS?

A. flutter build apk --release

B. flutter build ipa

C. flutter run --release

D. flutter build ios --release

TÀI LIỆU THAM KHẢO

1. Khoa Công nghệ Thông tin (2021). Tài liệu học tập học phần “Lập trình trên thiết bị di động”. HUTECH
2. Dart documentation. (n.d.). Retrieved from <https://dart.dev/guides>
3. Flutter documentation. (n.d.). Retrieved from <https://docs.flutter.dev/>
4. Rap Payne (2019), Beginning App Development with Flutter: Create Cross-Platform Mobile Apps
5. Atul Vashisht (2023), Flutter Architectures: Write code with a good architecture